[Reminder] Midterm course feedback survey: 10/13 - 10/27
Feedback is welcome!

Deep Learning for Computer Vision

114-1/Fall 2025

https://cool.ntu.edu.tw/courses/53063 (NTU COOL)

http://vllab.ee.ntu.edu.tw/dlcv.html (Public website)

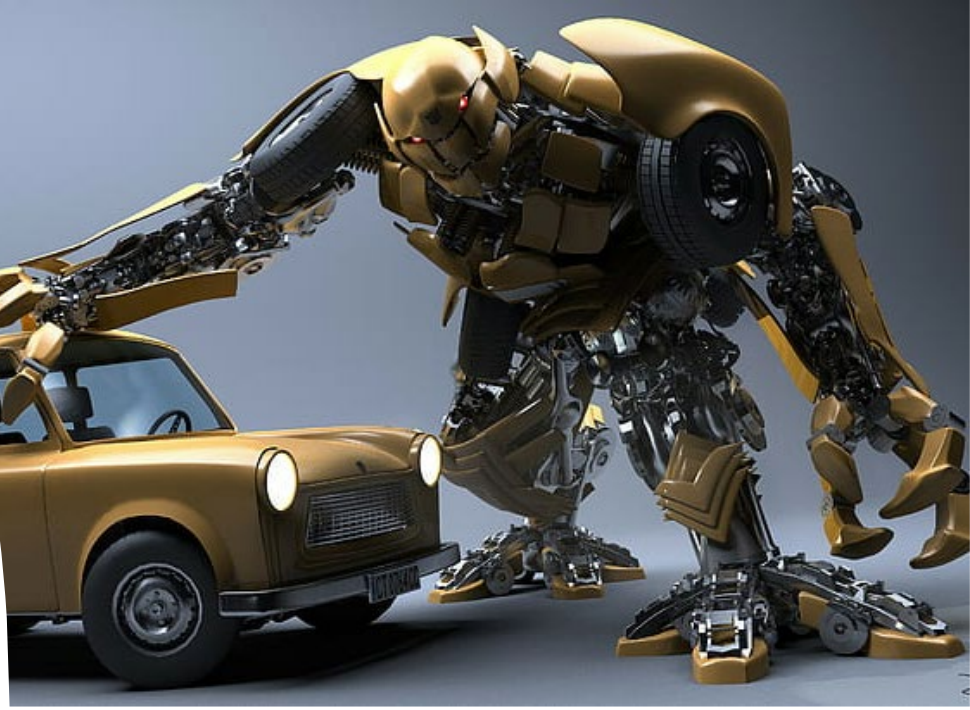
Yu-Chiang Frank Wang 王鈺強, Professor

Dept. Electrical Engineering, National Taiwan University

2025/10/15

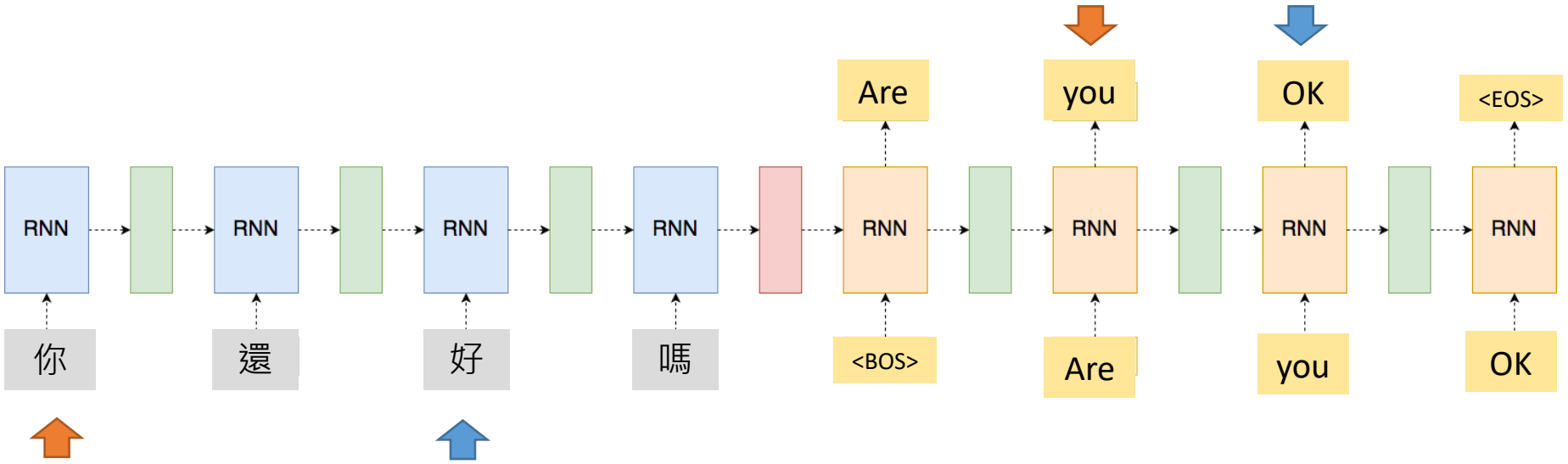
What to Be Covered?

- Transformer
- Transformer for Visual Analysis
  - Vision Transformer (ViT)
  - SSL & Beyond
- Vision-Language Model
  - Image2Text
  - Text2Image
- Pretraining vs. Finetuning
  - Parameter-Efficient Finetuning (PEFT)



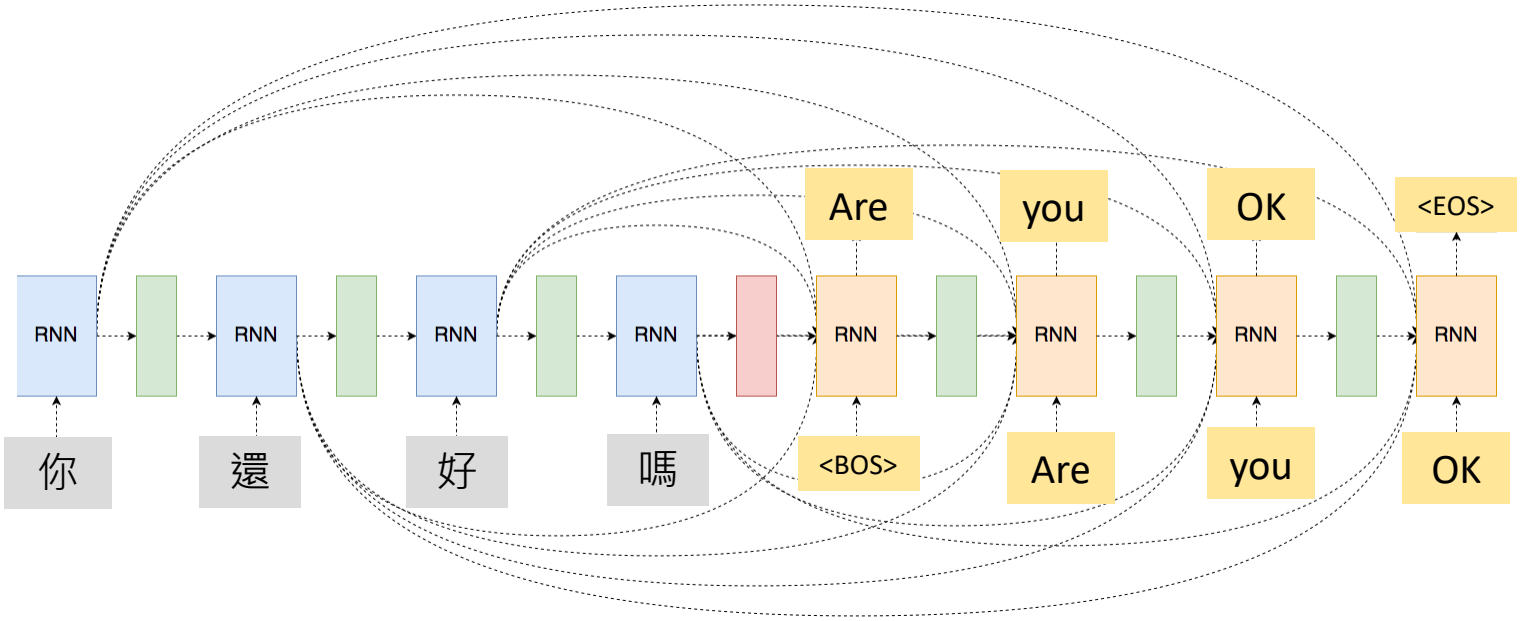
Recap: What's the Potential Problem for RNN?

- Hidden state vector carries information across time steps
- Outputs at different time steps have particular meanings; however, no synchrony between **input** and **output seqs**



What's the Potential Problem? (cont'd)

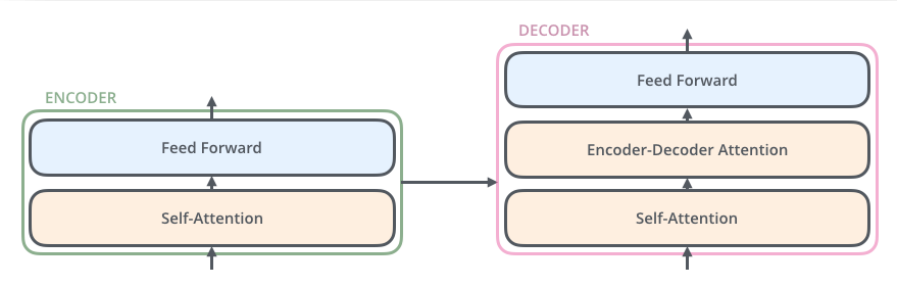
- Connecting every hidden state between encoder and decoder?

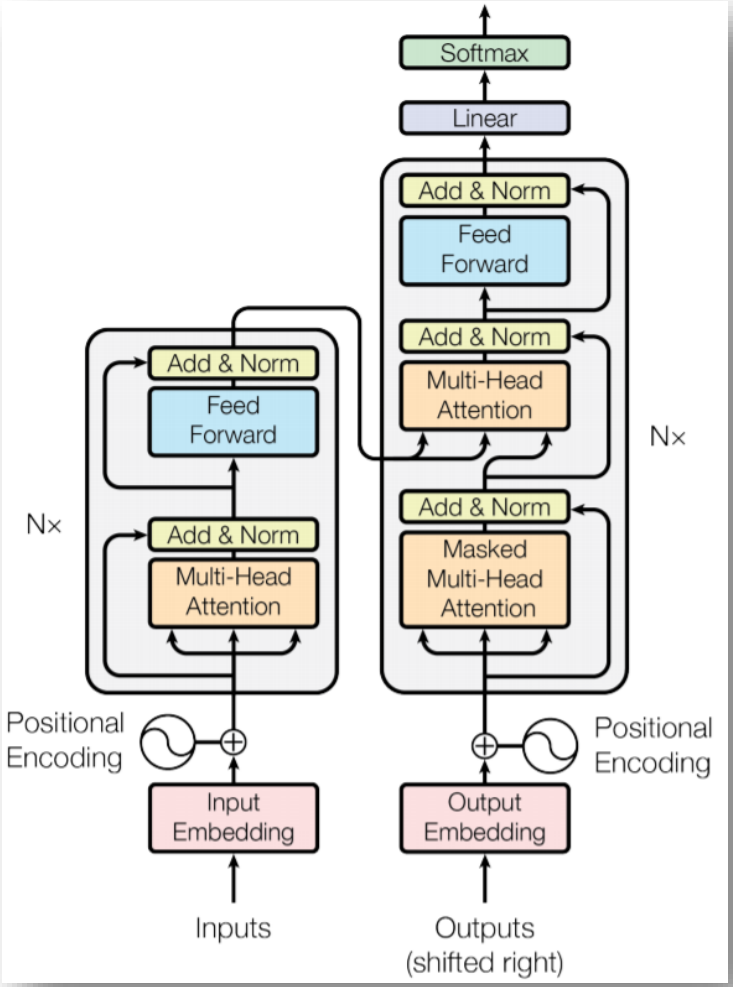


- Infeasible!
  - Both inputs and outputs are with varying sizes.
  - Overparameterized!
  - Possible solution: **attention**

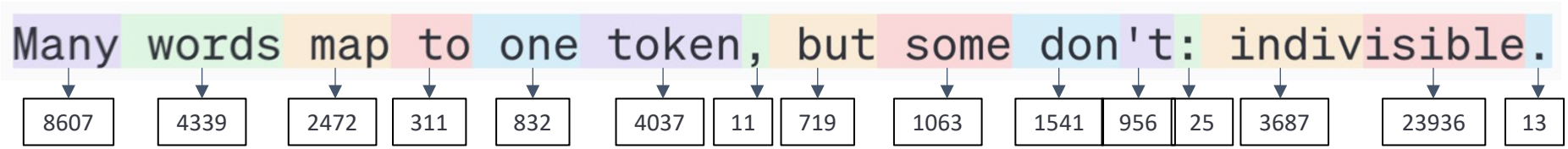
Solution #2: Transformer

- “Attention is all you need”, NIPS/NeurIPS 2017
- Self-attention for text translation
- Say goodbye to CNN & RNN
- More details available at:
  https://www.youtube.com/watch?v=rcWMRA9E5RI
  http://jalammar.github.io/illustrated-transformer/

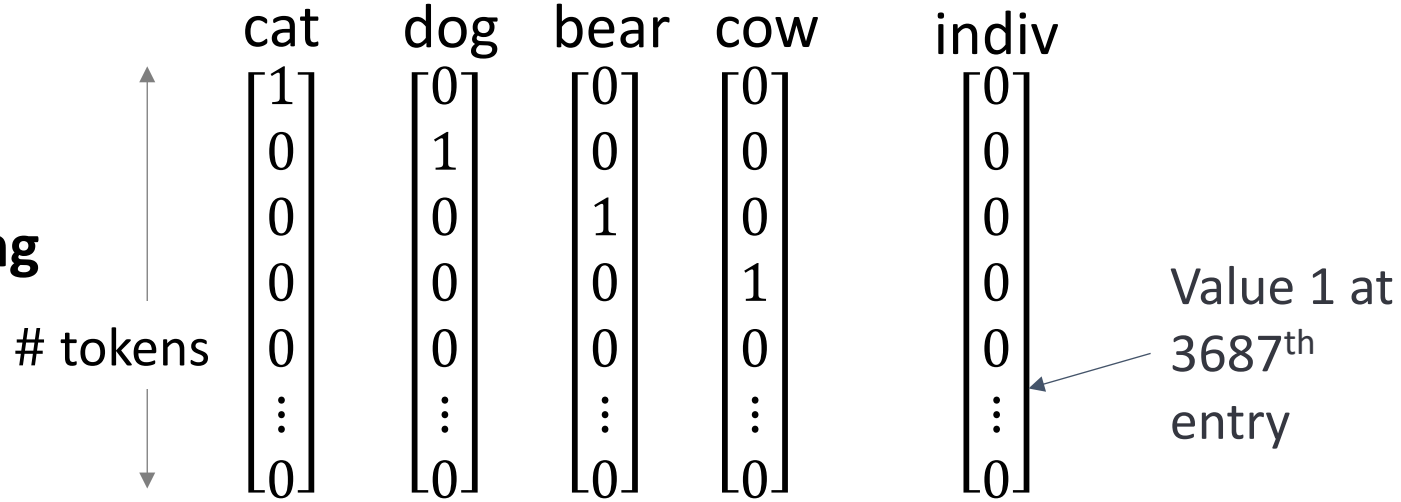




Tokenization

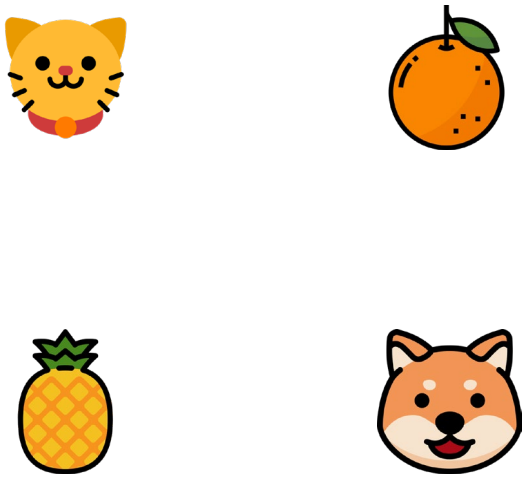


One-hot encoding



TOKEN EMBEDDING

One-hot encoding

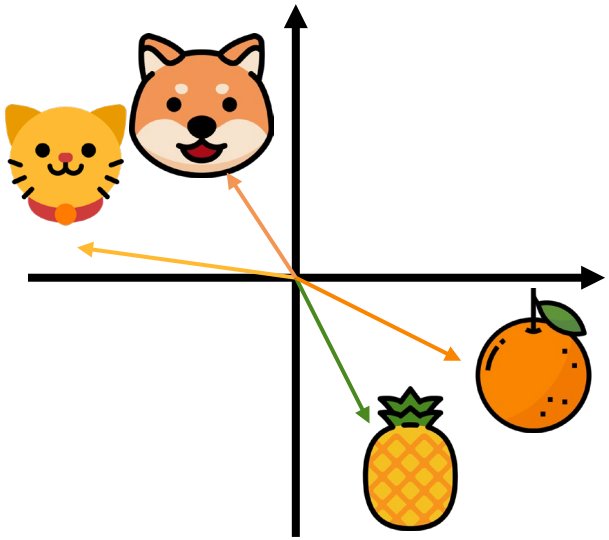


cat $\begin{bmatrix}1\\0\\0\\0\\0\\\vdots\\0\end{bmatrix}$ dog $\begin{bmatrix}0\\1\\0\\0\\0\\\vdots\\0\end{bmatrix}$ bear $\begin{bmatrix}0\\0\\1\\0\\0\\\vdots\\0\end{bmatrix}$ cow $\begin{bmatrix}0\\0\\0\\1\\0\\\vdots\\0\end{bmatrix}$ indiv $\begin{bmatrix}0\\0\\0\\0\\0\\\vdots\\0\end{bmatrix}$

Value 1 at 3687th entry

Slide credit: Prof. J.-B. Huang @ U. Maryland

TOKEN EMBEDDING



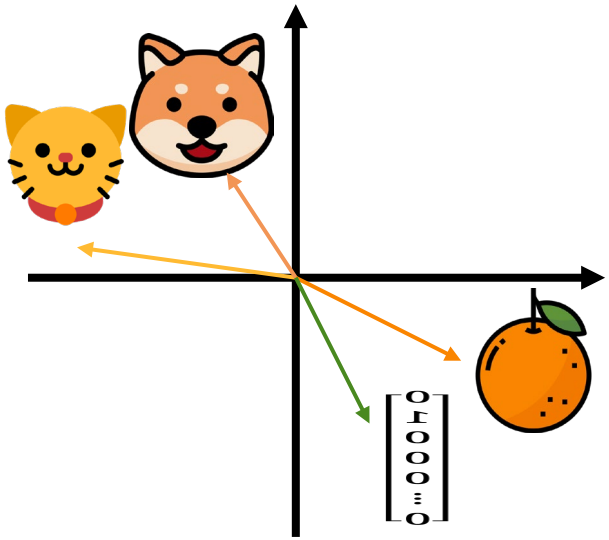
Embedding Space

cat $\begin{bmatrix}1\\0\\0\\0\\0\\\vdots\\0\end{bmatrix}$ dog $\begin{bmatrix}0\\1\\0\\0\\0\\\vdots\\0\end{bmatrix}$ bear $\begin{bmatrix}0\\0\\1\\0\\0\\\vdots\\0\end{bmatrix}$ cow $\begin{bmatrix}0\\0\\0\\1\\0\\\vdots\\0\end{bmatrix}$ indiv $\begin{bmatrix}0\\0\\0\\0\\0\\\vdots\\0\end{bmatrix}$

Value 1 at 3687th entry

Slide credit: Prof. J.-B. Huang @ U. Maryland

TOKEN EMBEDDING



Embedding Space

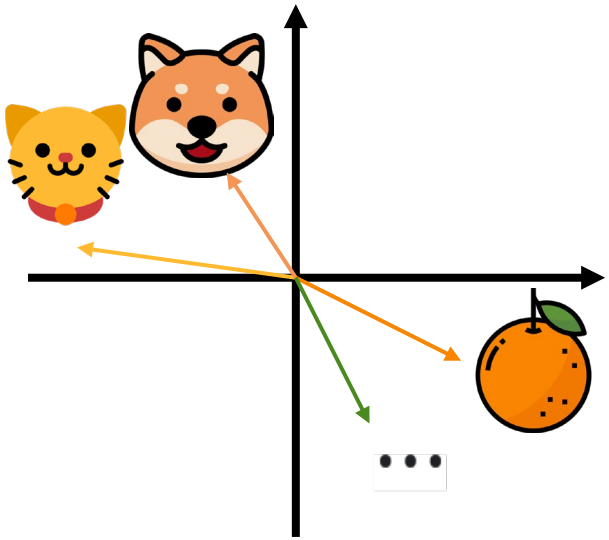
$$d\begin{bmatrix}0.5\\2.7\\1.2\\\vdots\\0.2\end{bmatrix} = d\begin{bmatrix} & \leftarrow \text{\# tokens} \rightarrow & \\ & \boldsymbol{W_E} & \end{bmatrix}\begin{matrix}\\\\\\\\\\\\\end{matrix}\begin{matrix}\text{dog}\\\begin{bmatrix}0\\1\\0\\0\\0\\\vdots\\0\end{bmatrix}\end{matrix}$$

Embedded token

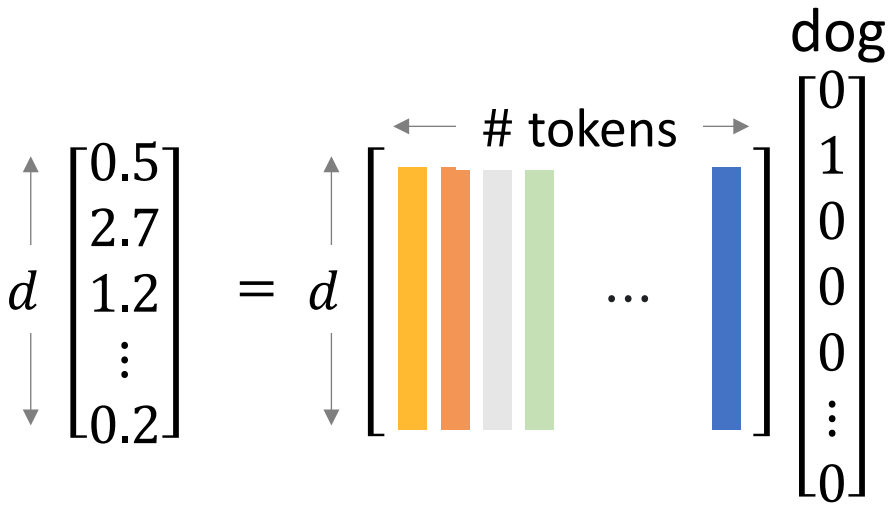
Embedding Matrix

Slide credit: Prof. J.-B. Huang @ U. Maryland

TOKEN EMBEDDING



Embedding Space

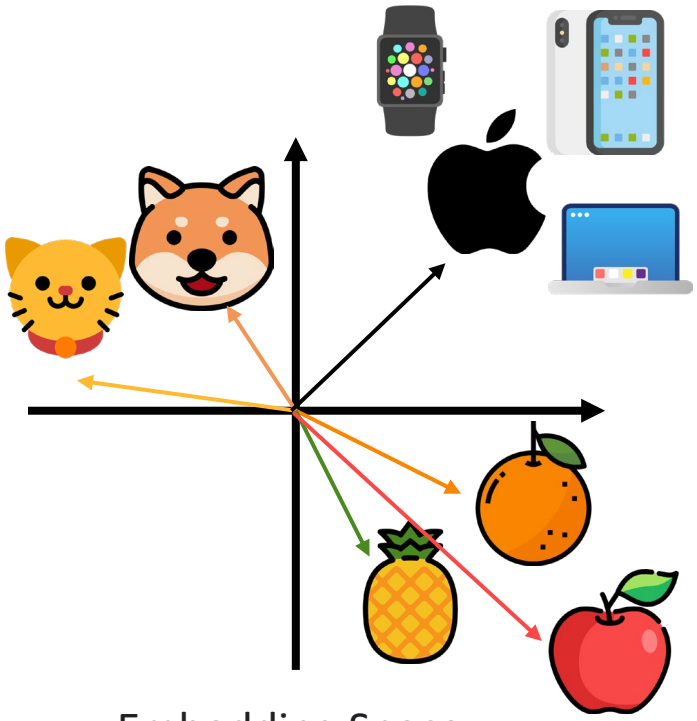


Embedded token

Embedding Matrix

Slide credit: Prof. J.-B. Huang @ U. Maryland

TOKEN EMBEDDING



Apple

Embedding Space

$$d \begin{bmatrix} 0.5 \\ 2.7 \\ 1.2 \\ \vdots \\ 0.2 \end{bmatrix} = d \begin{bmatrix} & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \end{bmatrix}$$

tokens

dog

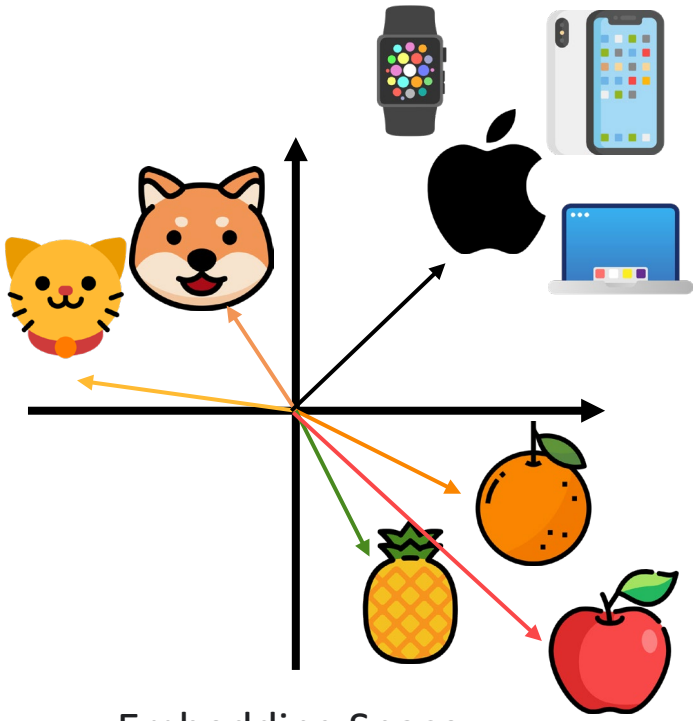
$$\begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Embedded token

Embedding Matrix

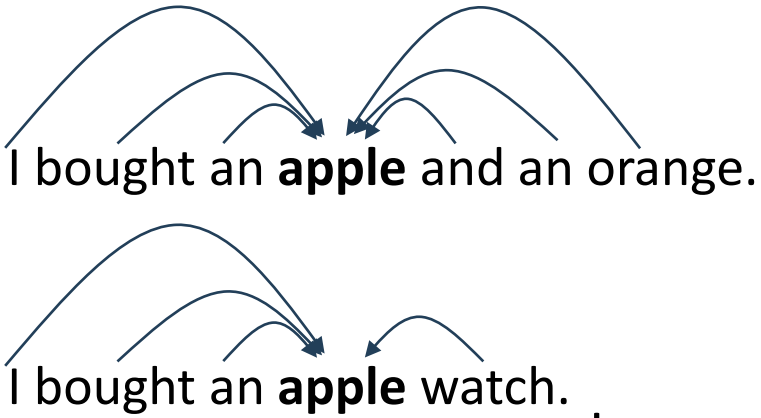
Slide credit: Prof. J.-B. Huang @ U. Maryland

TOKEN EMBEDDING



Embedding Space

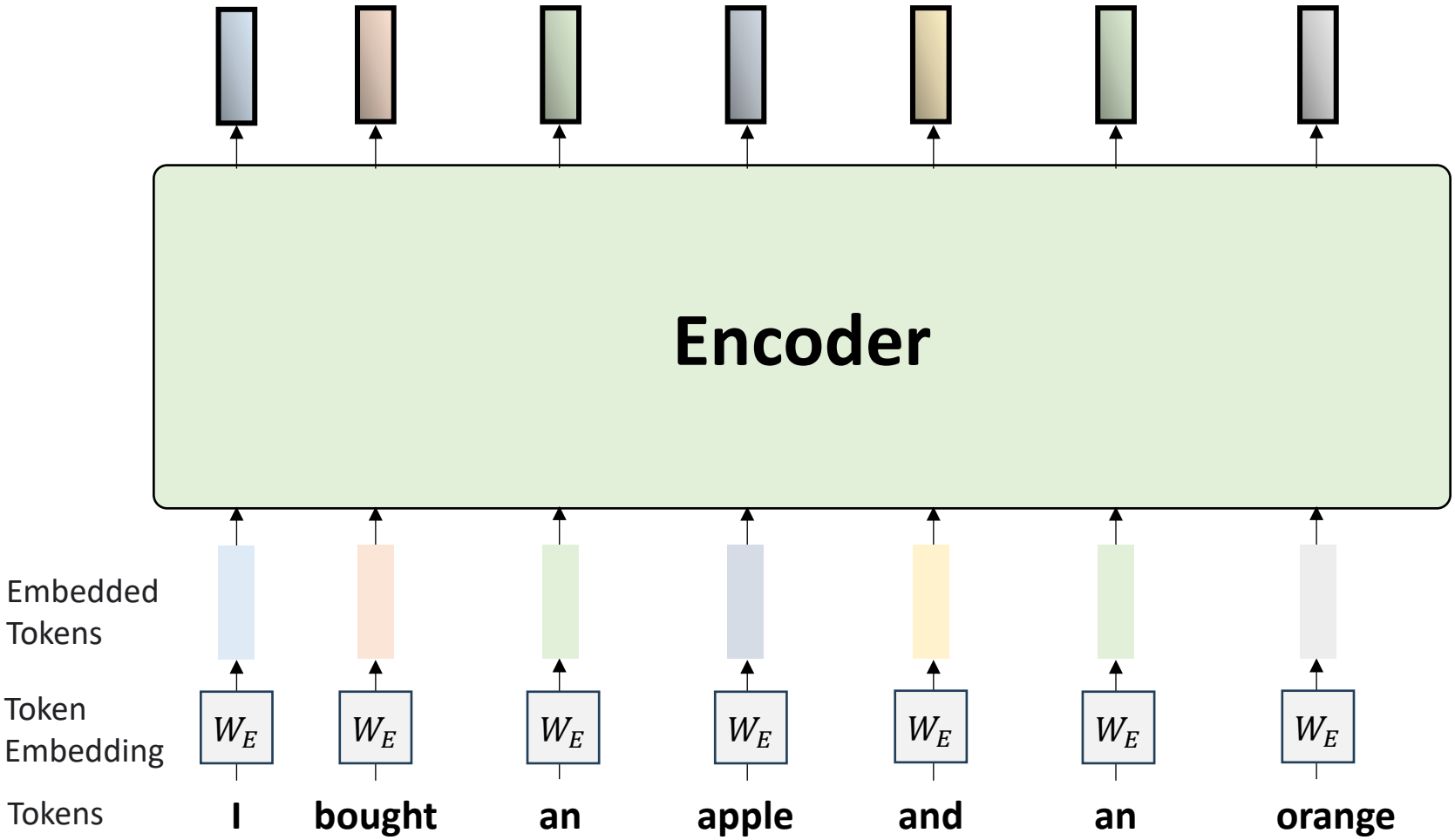
Apple



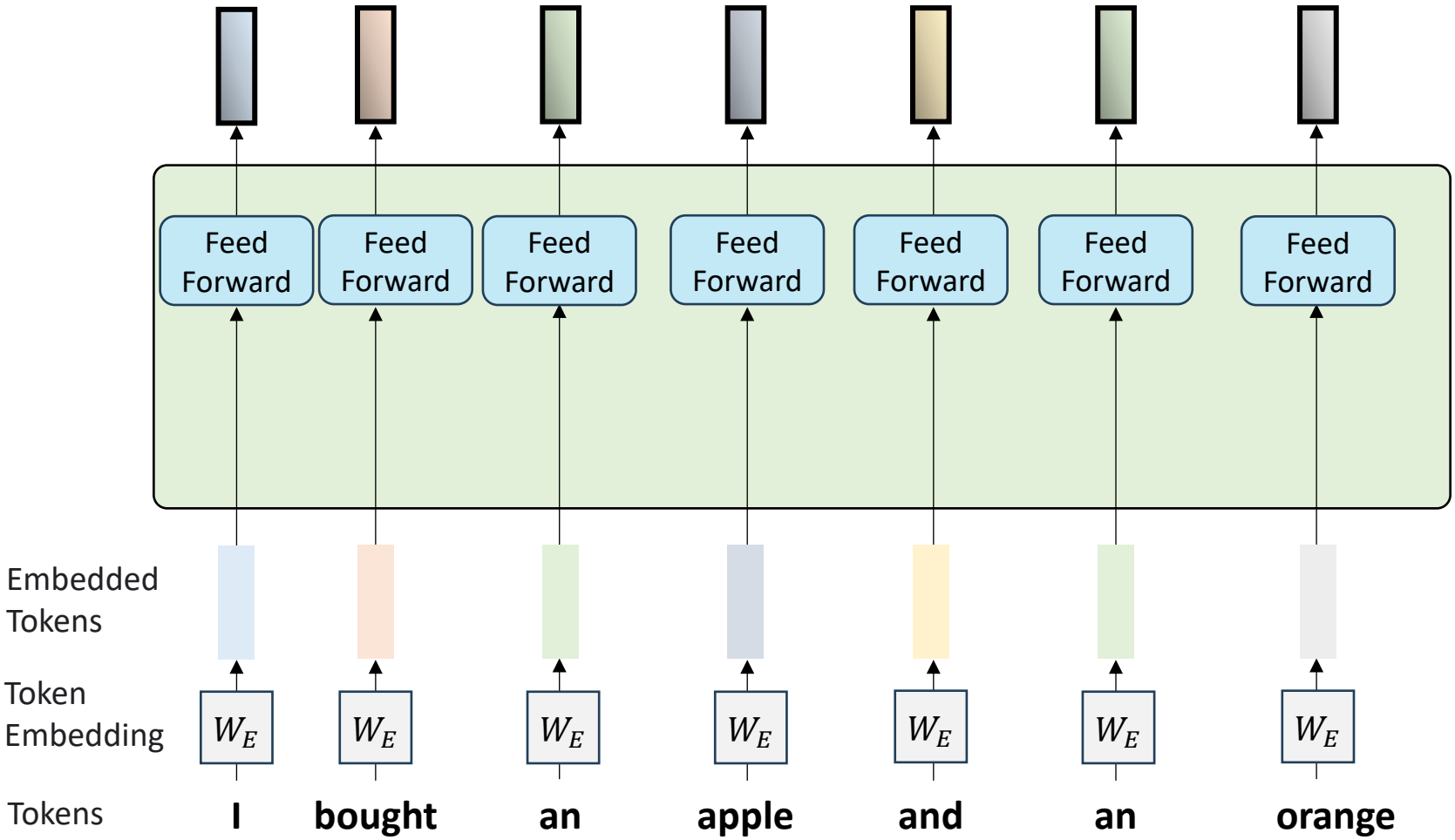
$$d \begin{bmatrix} 0.5 \\ 2.7 \\ 1.2 \\ \vdots \\ 0.2 \end{bmatrix} = d \begin{bmatrix} \leftarrow \text{\# tokens} \rightarrow \\ \cdots \end{bmatrix} \begin{bmatrix} \text{dog} \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Embedded token

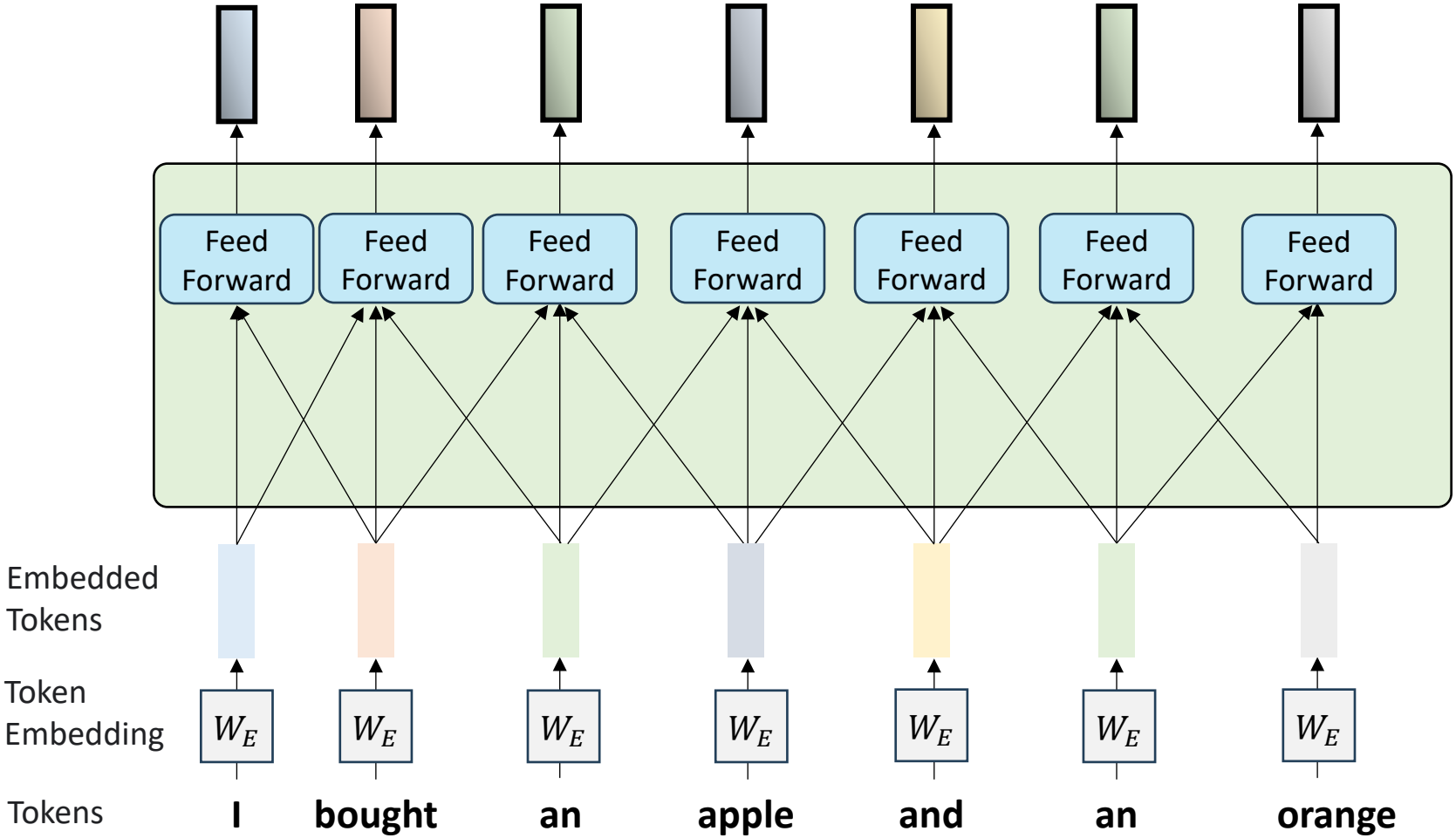
Embedding Matrix



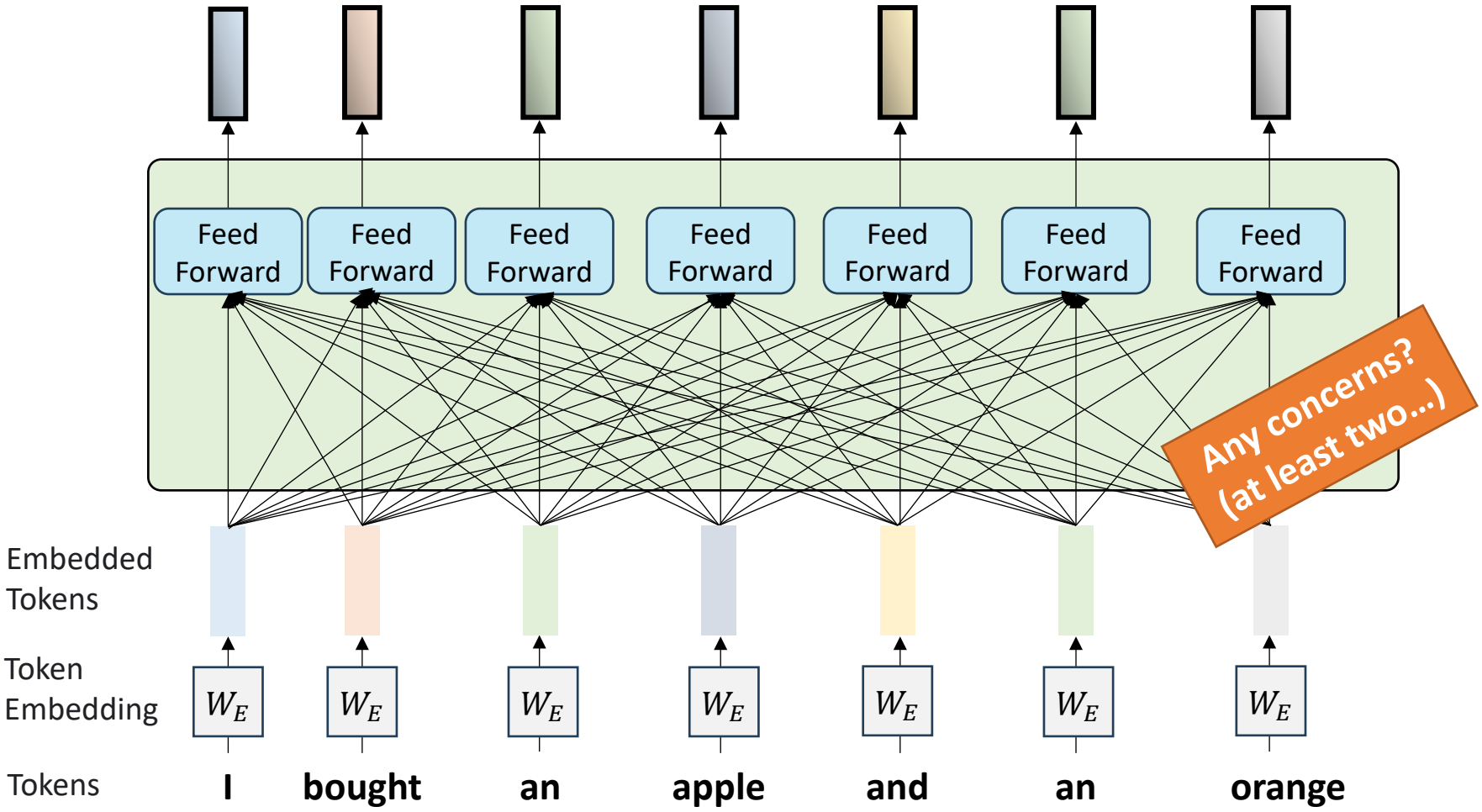
Slide credit: Prof. J.-B. Huang @ U. Maryland



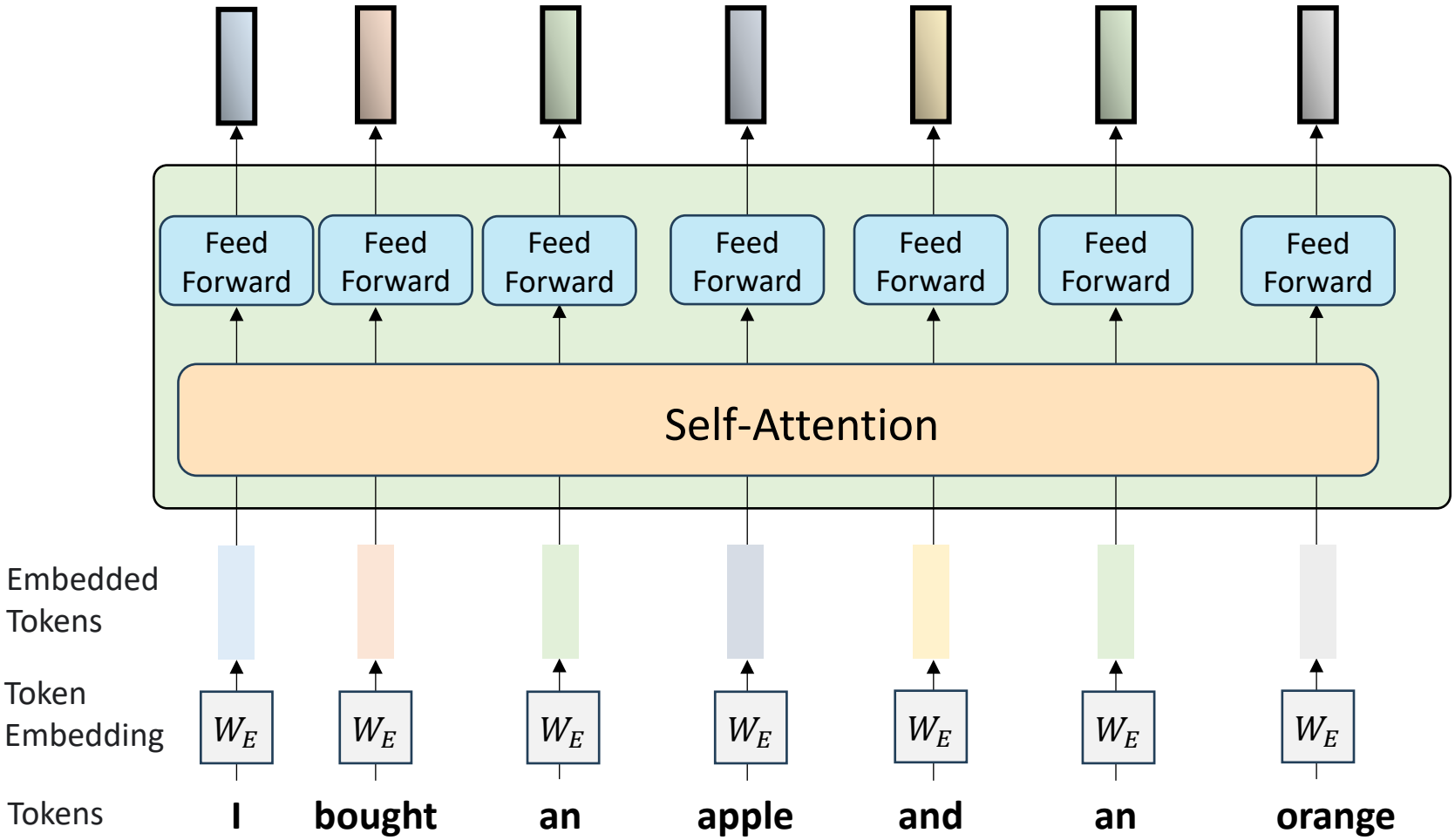
Slide credit: Prof. J.-B. Huang @ U. Maryland



Slide credit: Prof. J.-B. Huang @ U. Maryland

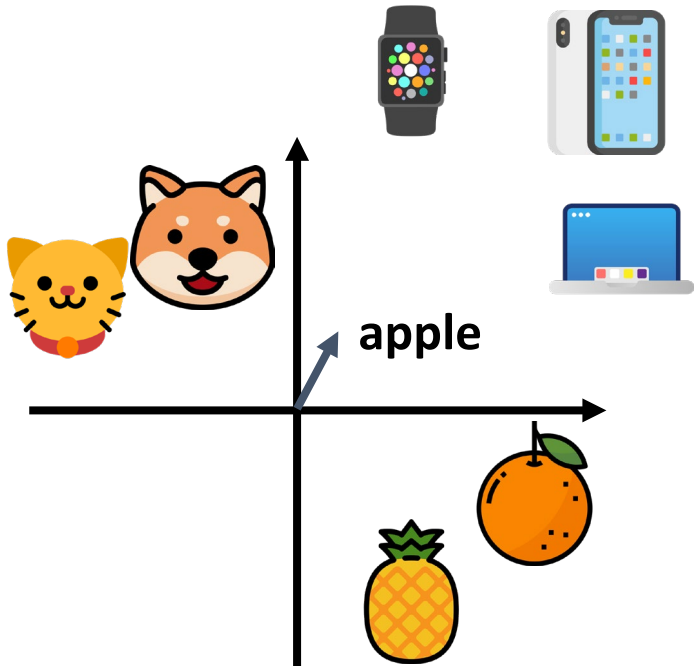


Slide credit: Prof. J.-B. Huang @ U. Maryland



Slide credit: Prof. J.-B. Huang @ U. Maryland

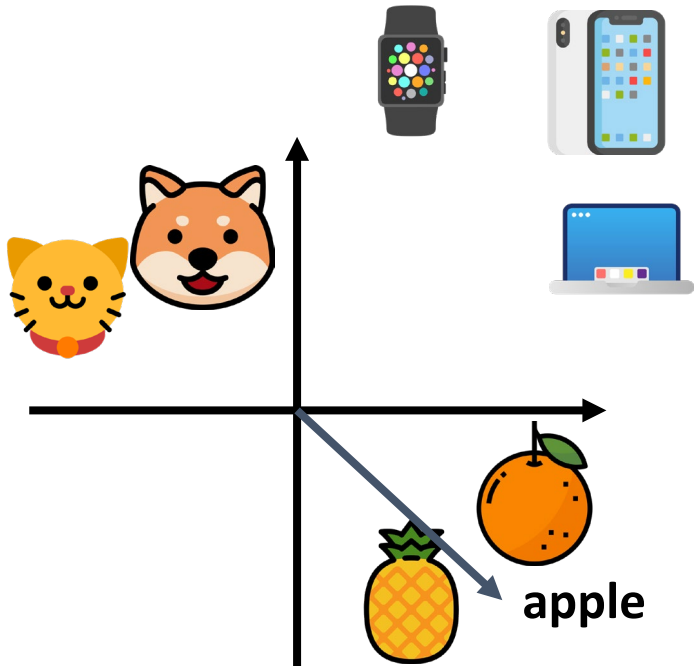
Self-Attention



Embedded Tokens

Tokens **I** **bought** **an** **apple** **and** **an** **orange**

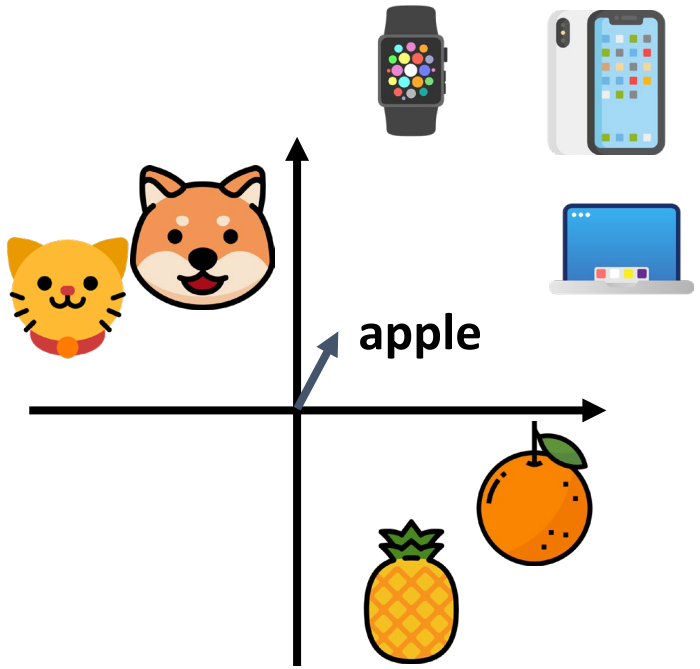
Self-Attention



Embedded Tokens

Tokens **I** **bought** **an** **apple** **and** **an** **orange**

Self-Attention

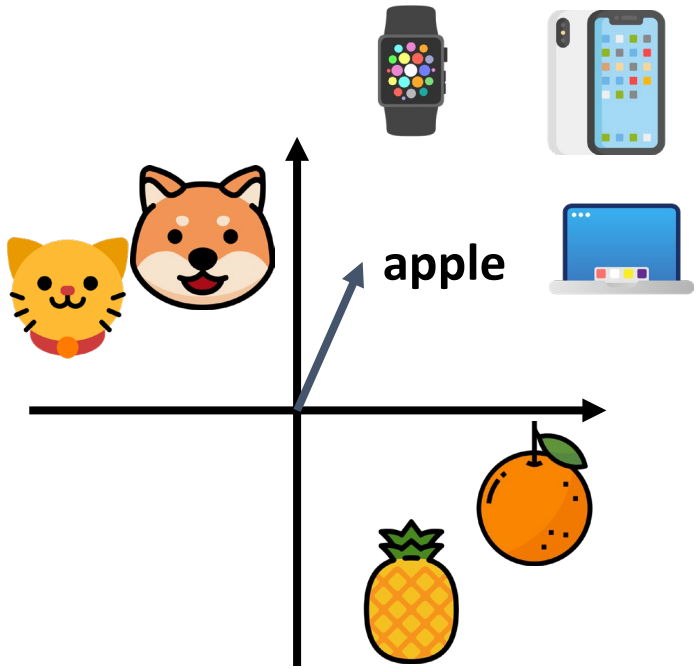


Embedded Tokens

Tokens **I** **bought** **an** **apple** **watch**

Slide credit: Prof. J.-B. Huang @ U. Maryland

Self-Attention

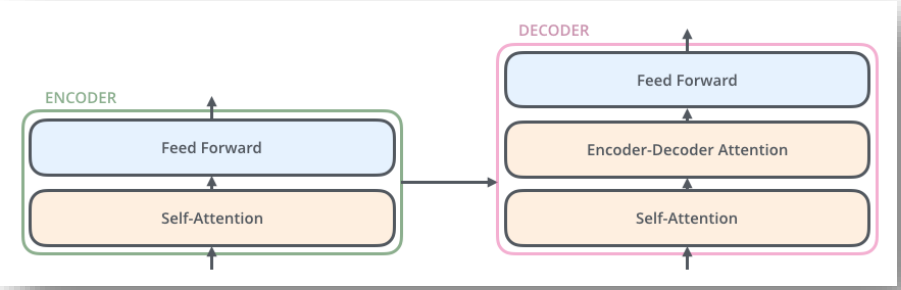


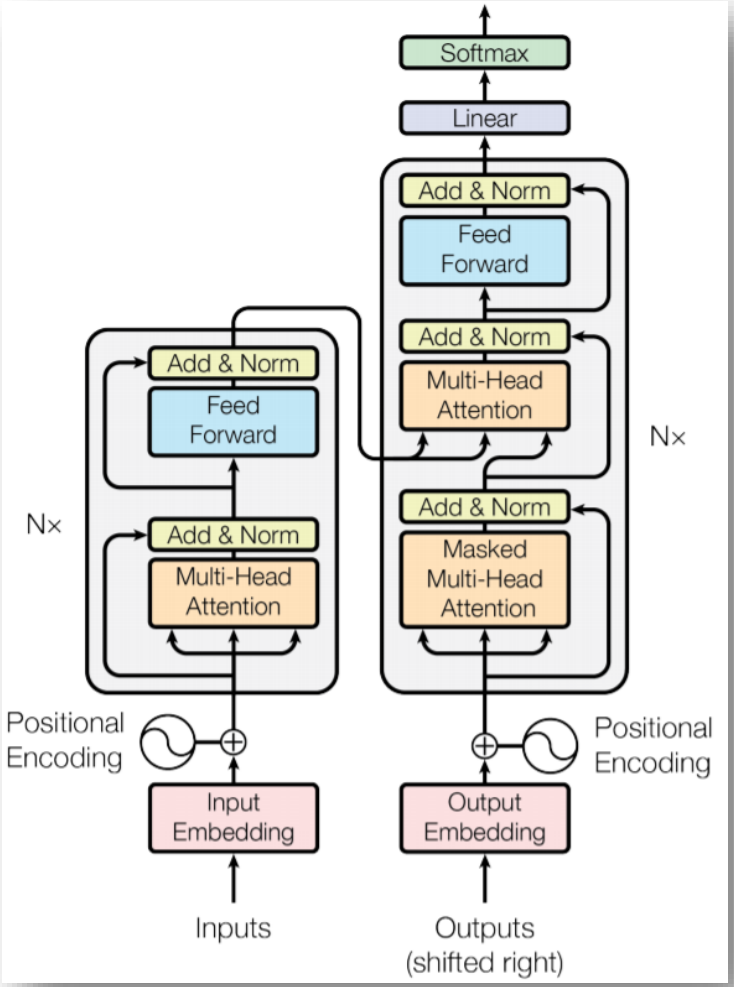
Embedded Tokens

Tokens **I** **bought** **an** **apple** **watch**

Solution #2: Transformer

- “Attention is all you need”, NIPS/NeurIPS 2017
- Self-attention for text translation
- More details available at:
  https://www.youtube.com/watch?v=rcWMRA9E5RI
  http://jalammar.github.io/illustrated-transformer/

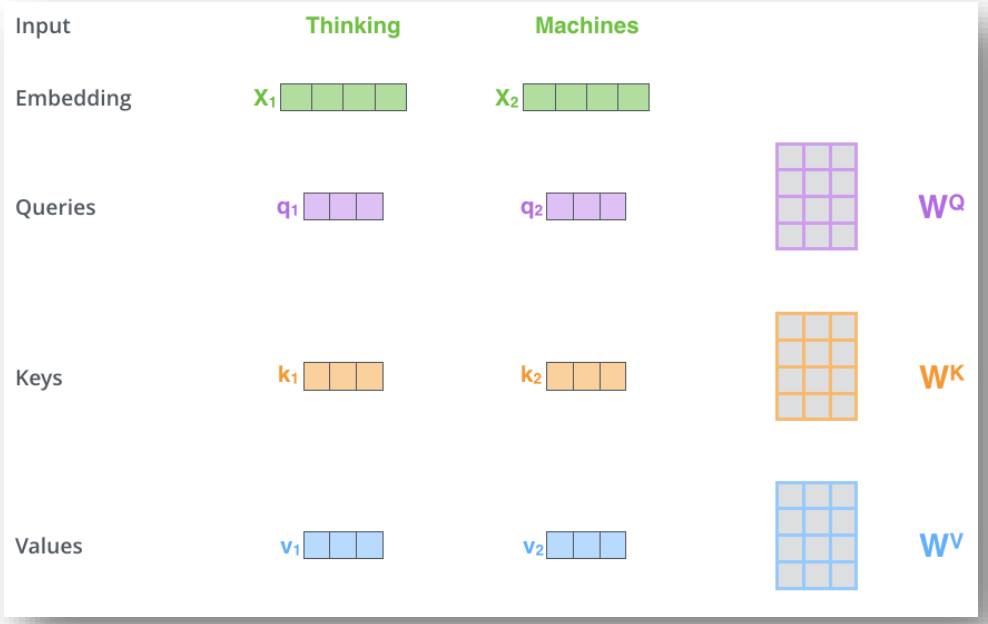


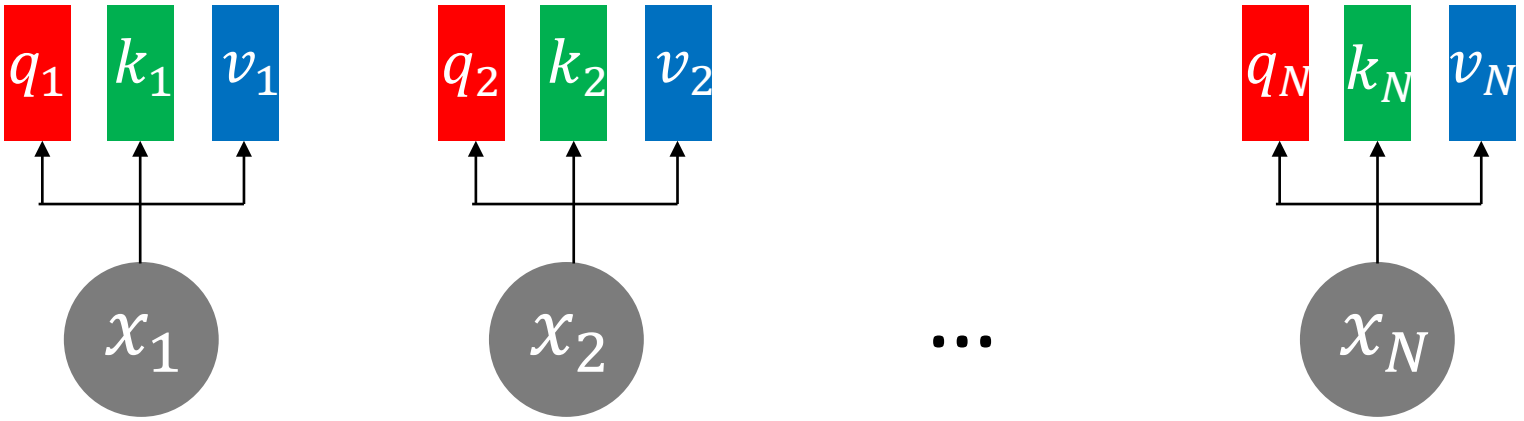


Self-Attention (1/5)

- Query ***q***, key ***k***, value ***v*** vectors are learned from each input ***x***

$$q_i = W^Q x_i$$
$$k_i = W^K x_i$$
$$v_i = W^V x_i$$

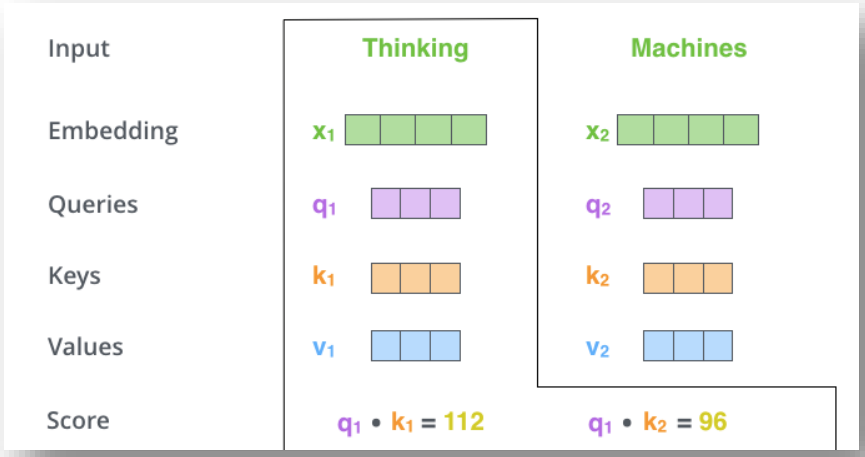


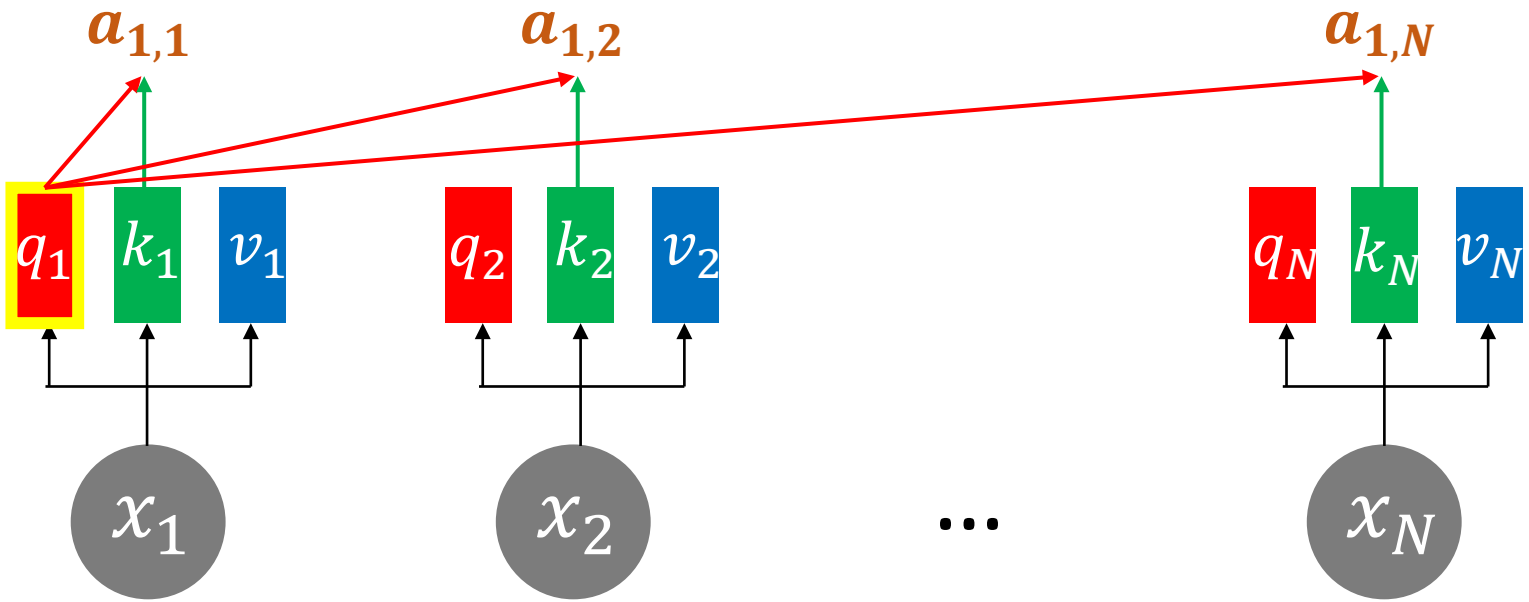


Self-Attention (2/5)

- Relation between each input is modeled by inner-product of query ***q*** and key ***k***.

$$a_{1,i} = \frac{q_1 \cdot k_i}{\sqrt{d}}, \text{ where } a \in R, q, k \in R^d$$

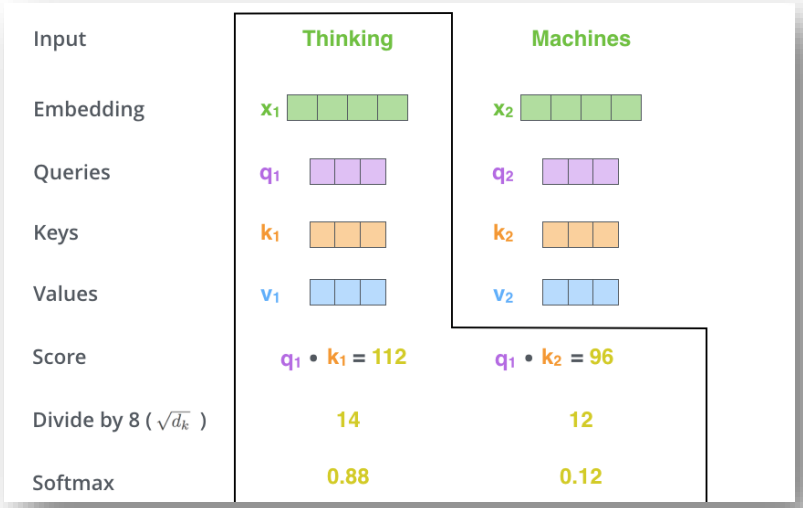


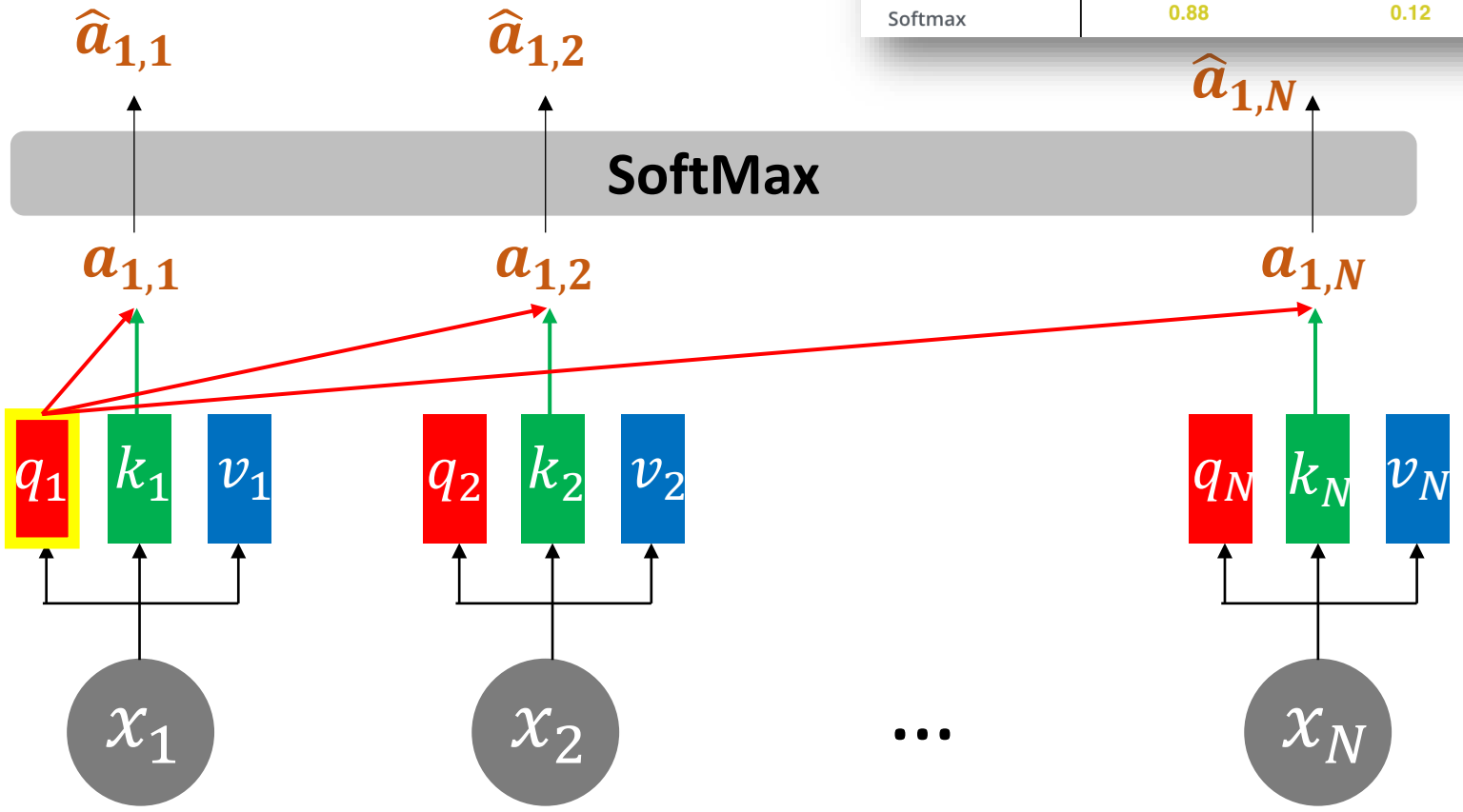


Self-Attention (3/5)

- SoftMax is applied:

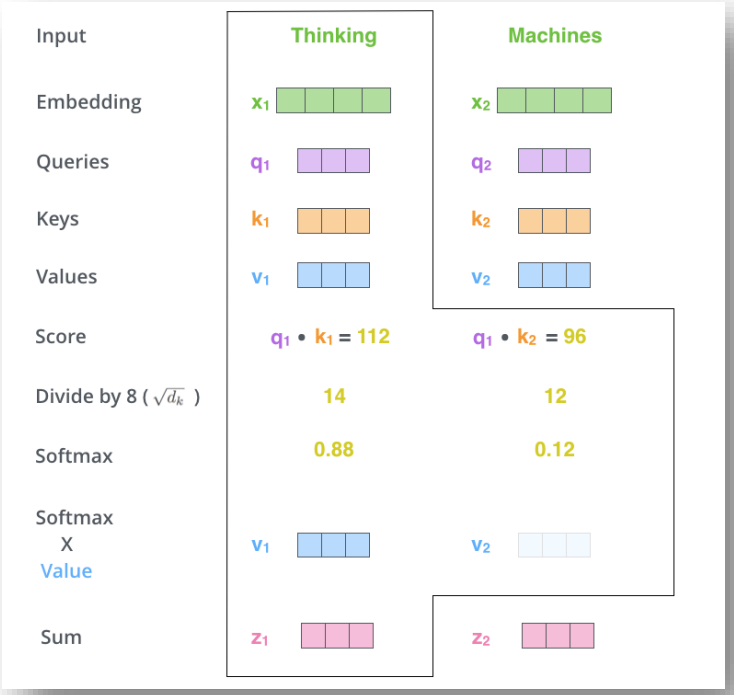
$0 \le \hat{a}_i = e^{a_i} / \sum_j^{\mathrm{N}} e^{a_j} \le 1$, for I =1, …, N

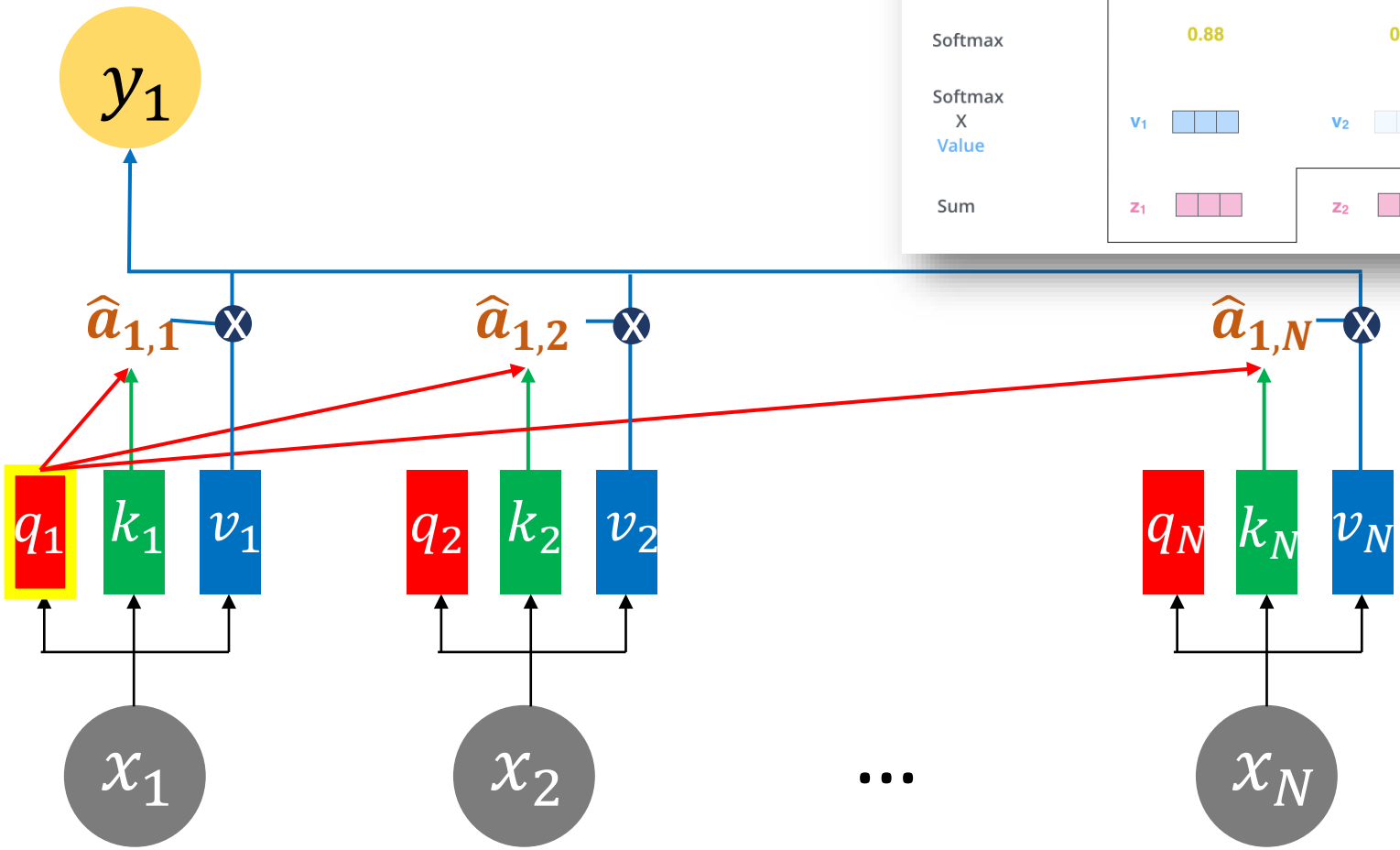




Self-Attention (4/5)

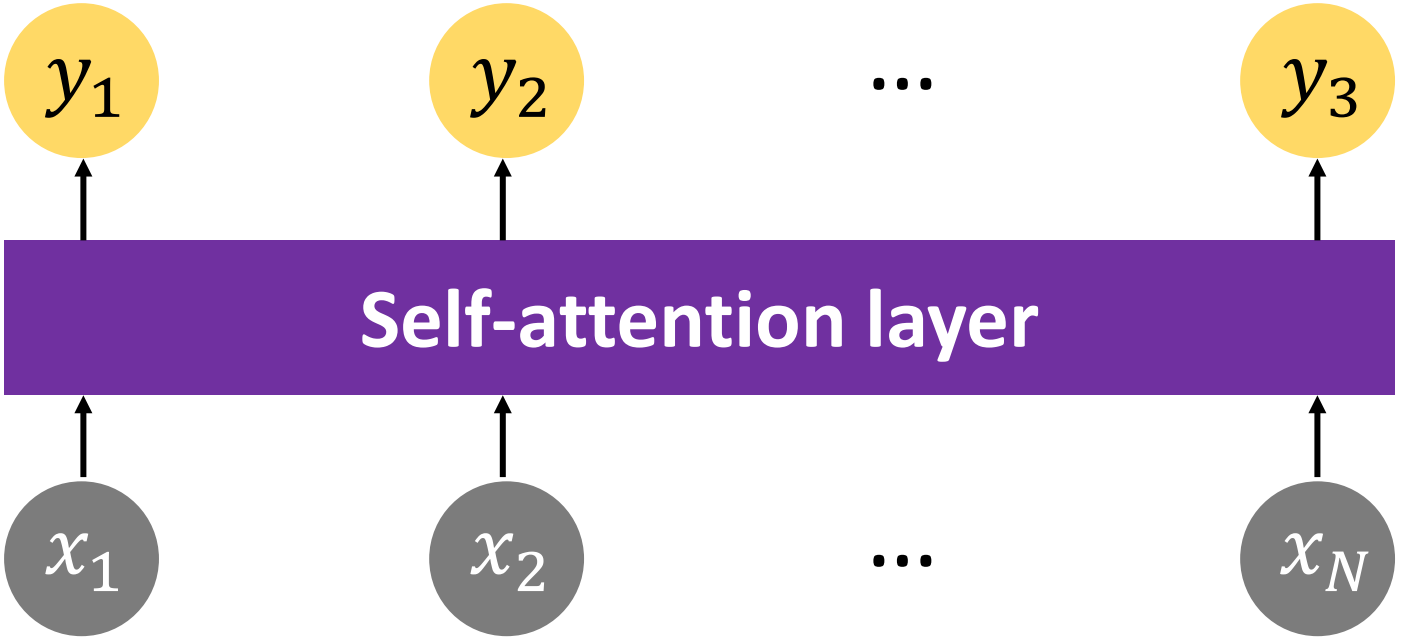
- Value vectors ***v*** are aggregated with attention weight $\hat{a}$, i.e., $y_1 = \sum_i^N \hat{a}_i \cdot v_i$



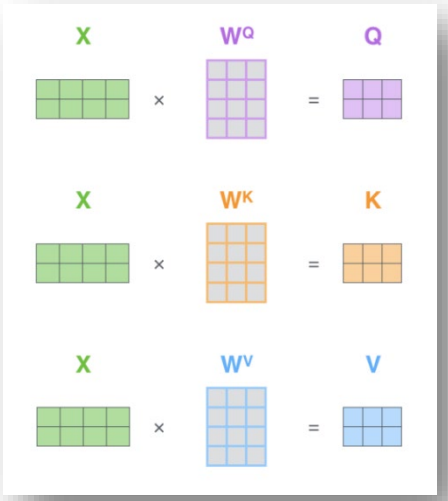


Self-Attention (5/5)

- All y_i can be computed **in parallel**
- Each y_i considers $x_1 \sim x_N$, modeling their **long-distance dependencies**.
- Global feature can be obtained by **average-pooling** over $y_1 \sim y_N$



Self-Attention: Implementation



- Input sequence can be represented as a N x d_{in} matrix
- * denotes matrix multiplication

$x_i \in R^{d_{in}}$

x_1
x_2
x_N

Input matrix

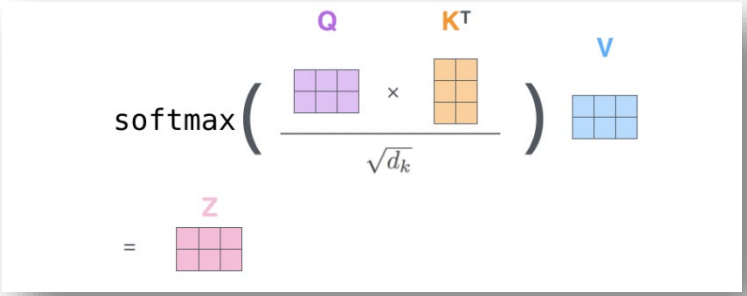
$= N \; d_{in}$

$* \; d_{in} \; W^Q \rightarrow N \; Q \; d$

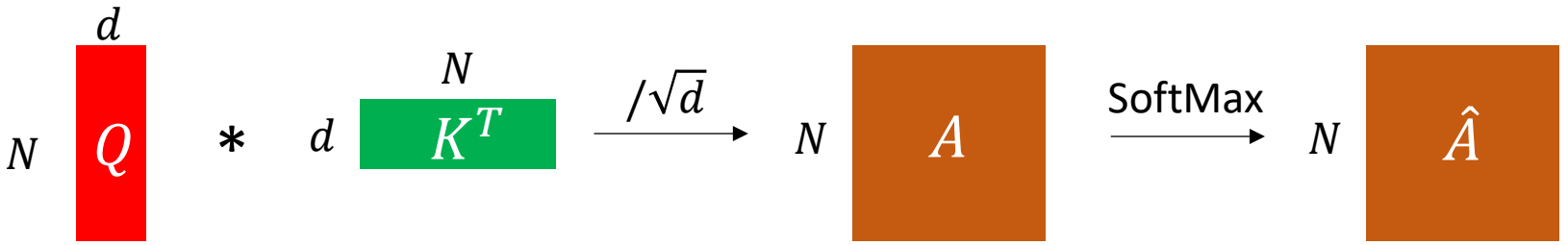
$* \; d_{in} \; W^K \rightarrow N \; K \; d$

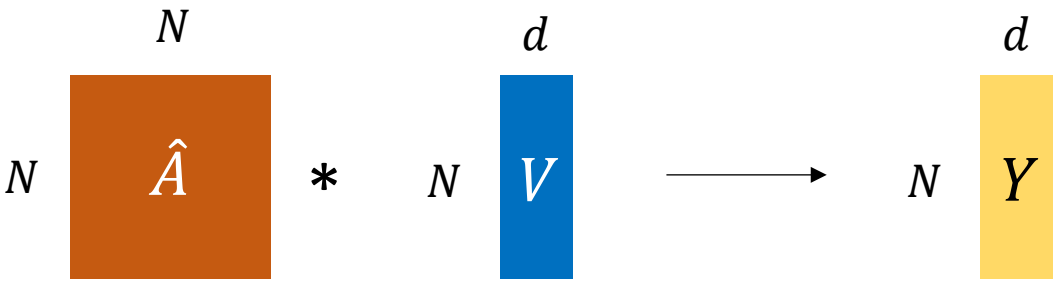
$* \; d_{in} \; W^V \rightarrow N \; V \; d$

Self-Attention: Implementation



- Output matrix **Y**
- All operations are **matrix multiplication**, can be parallelized on GPU.

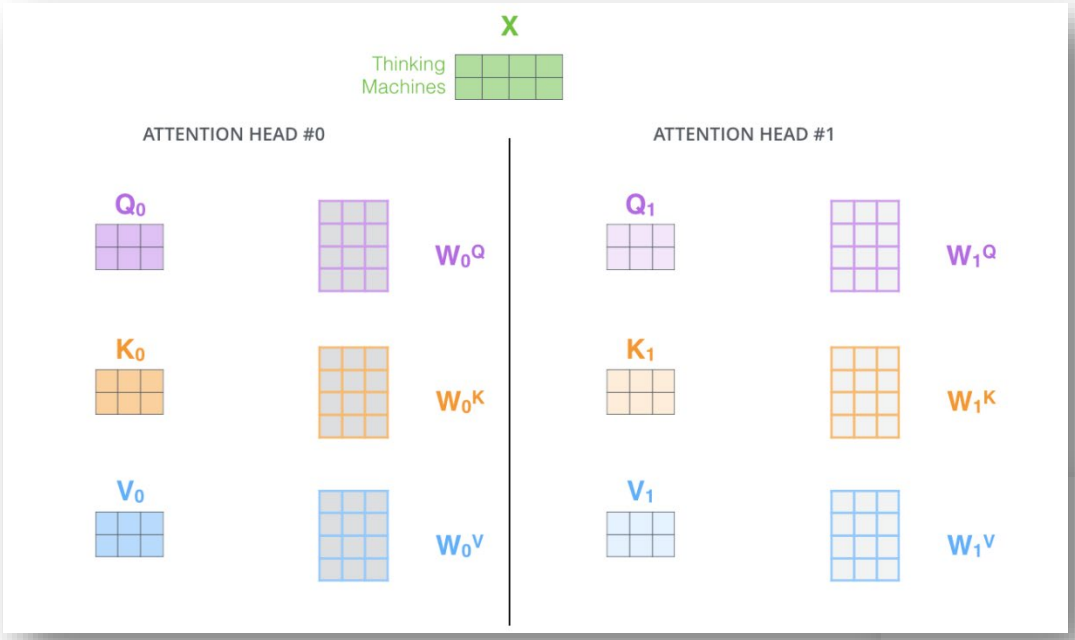


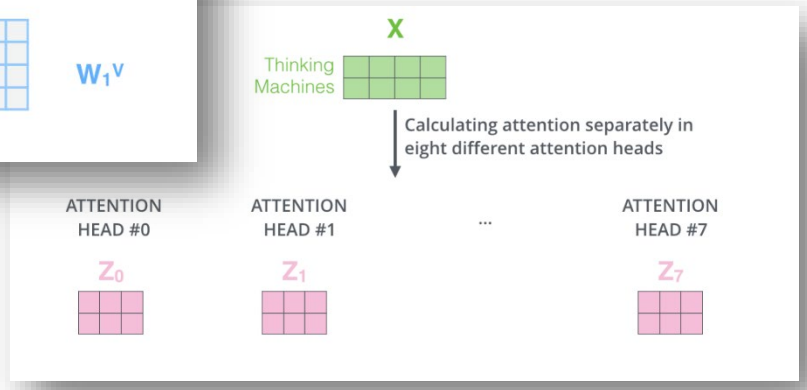


$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

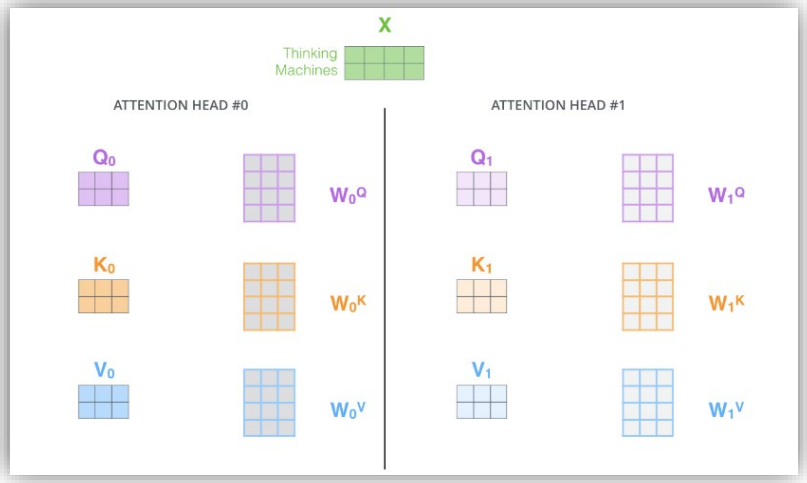
Multi-Head Self-Attention (1/4)

- Perform self-attention at different subspaces, implying performing attention over different input feature types (e.g., representations, modalities, positions, etc.)

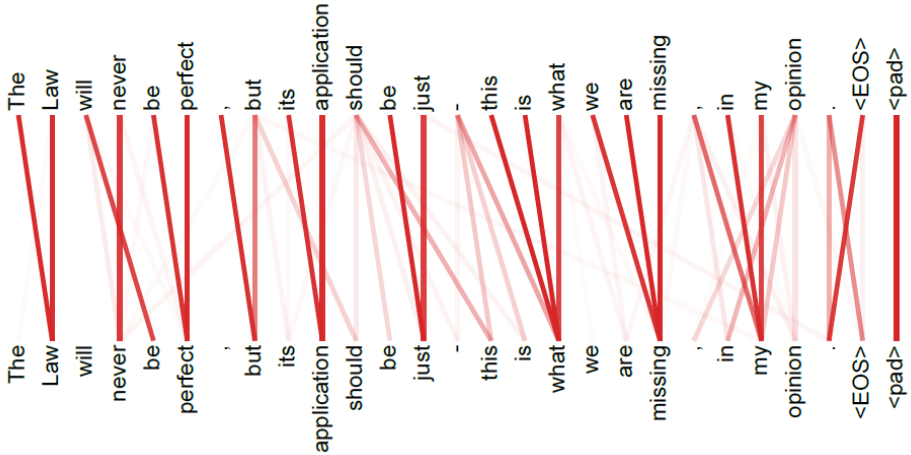




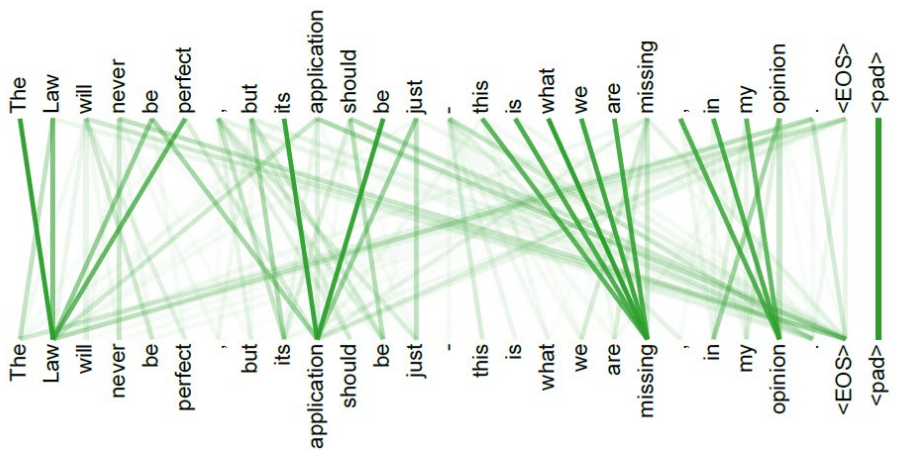
Multi-Head Self-Attention (2/4)



- Perform self-attention at different subspaces, implying performing attention over different input feature types
- See example below



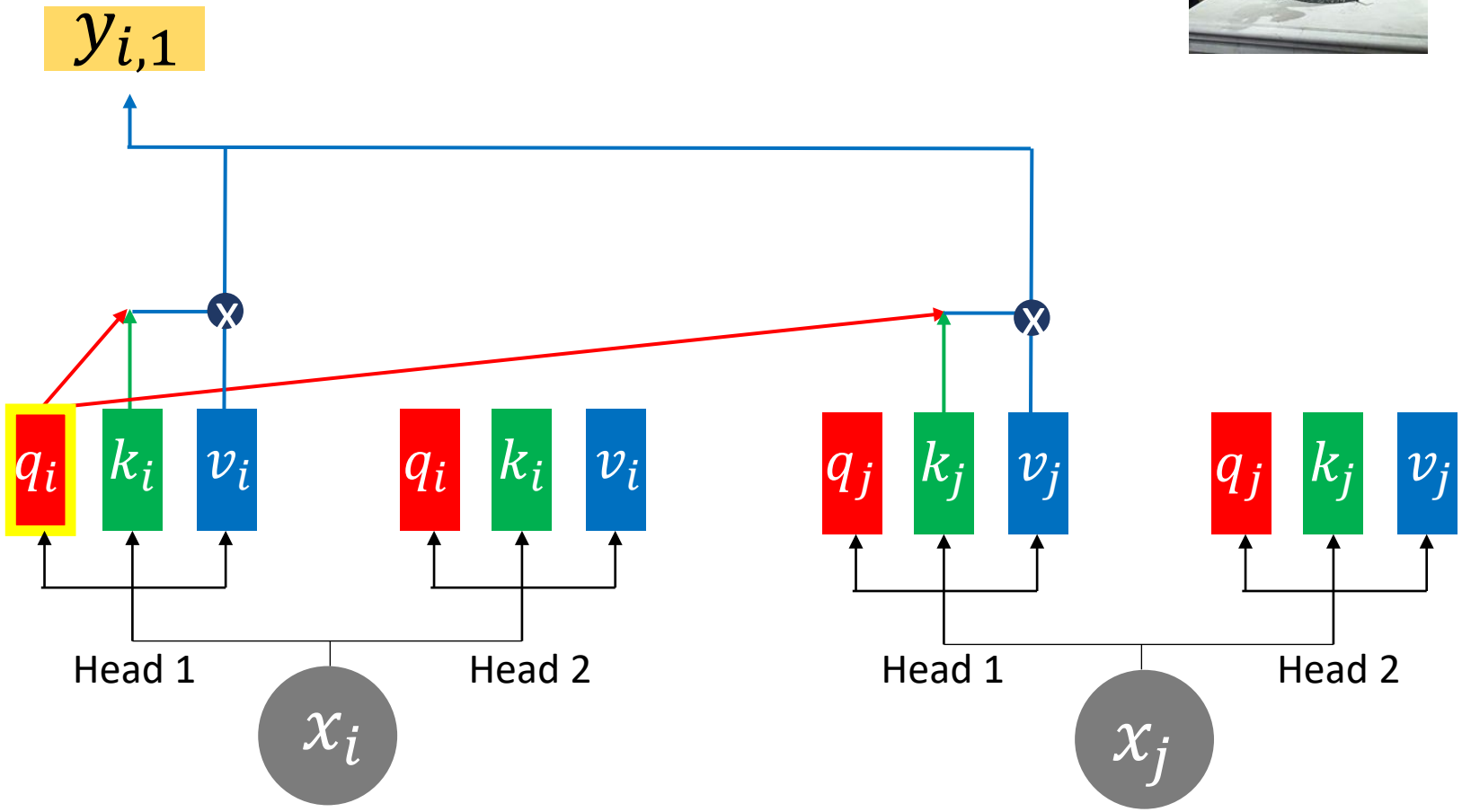
Attention weights of Head 1



Attention weights of Head 2

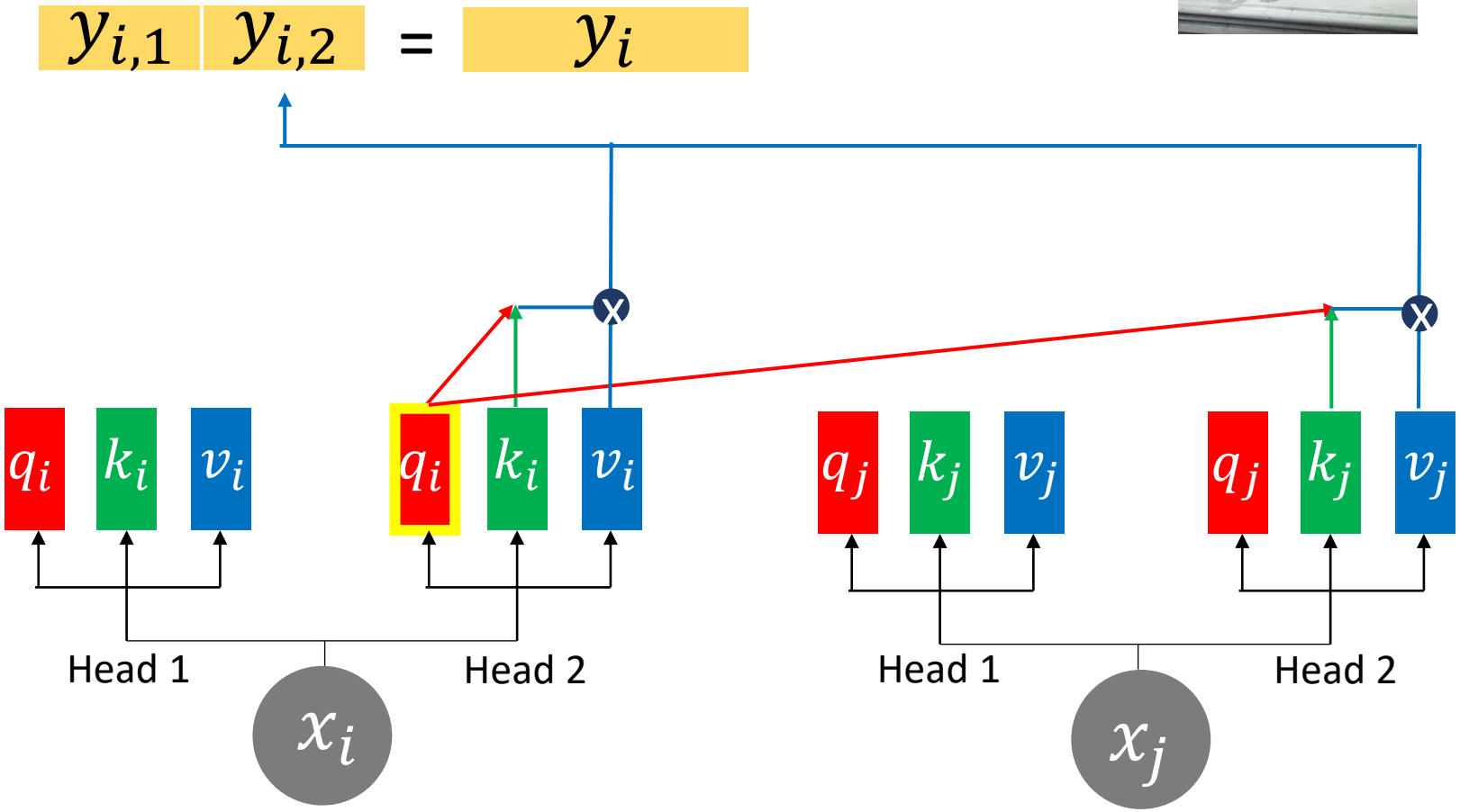
Multi-Head Self-Attention (3/4)

- A 2-head example, output of two heads are concatenated.

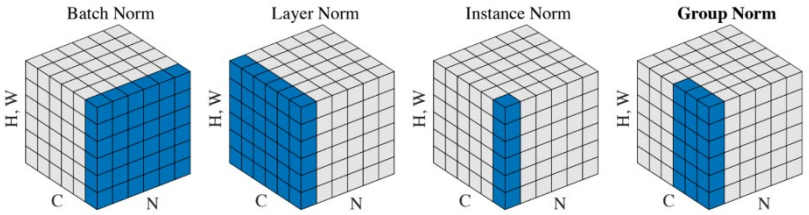


Multi-Head Self-Attention (4/4)

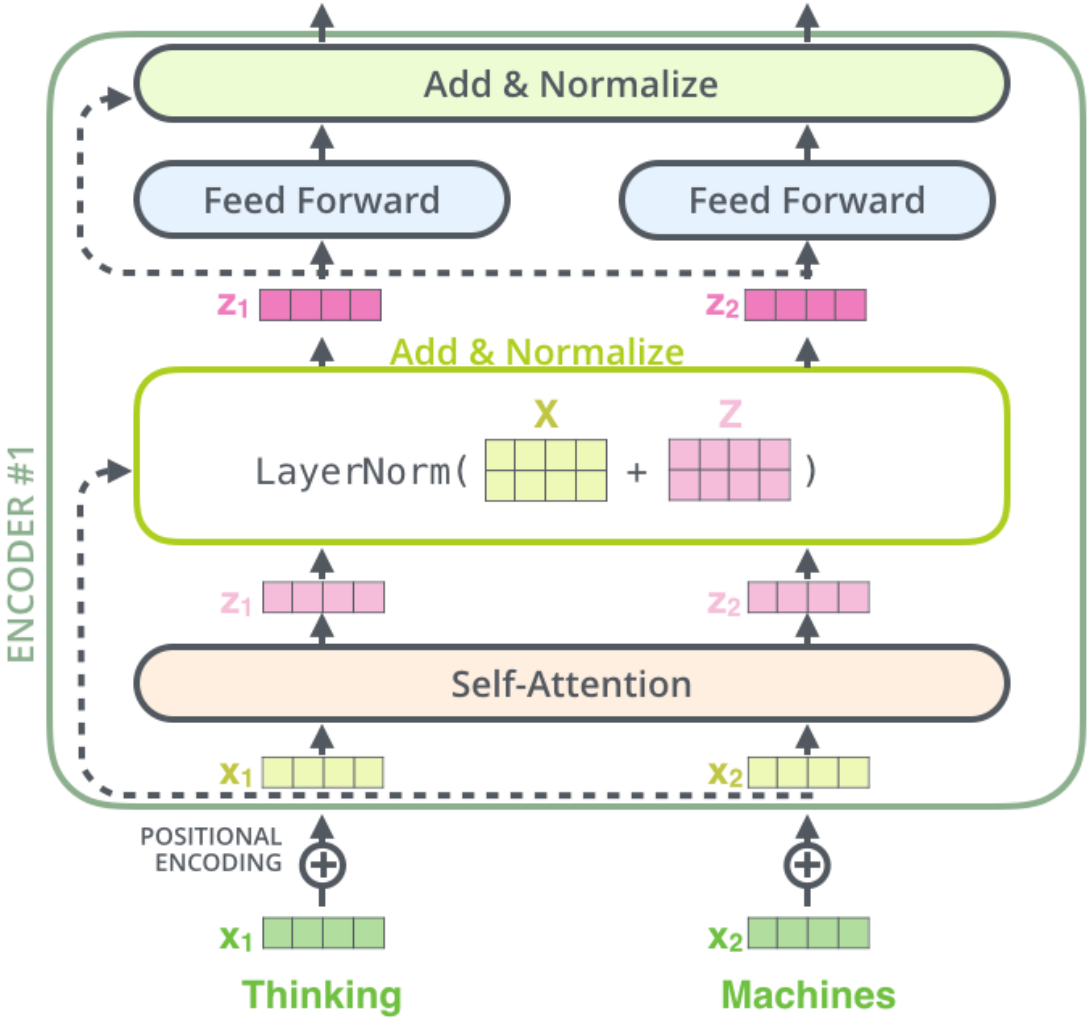
- A 2-head example, output of two heads are concatenated.



The Residuals

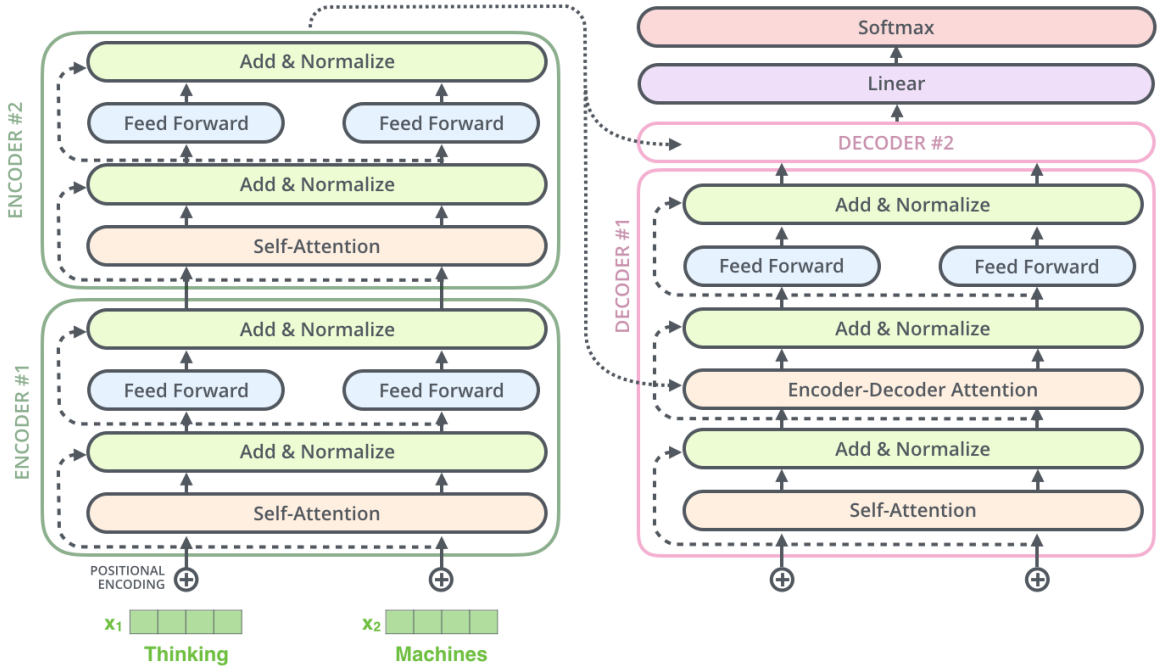


- A **residual connection** followed by layer normalization



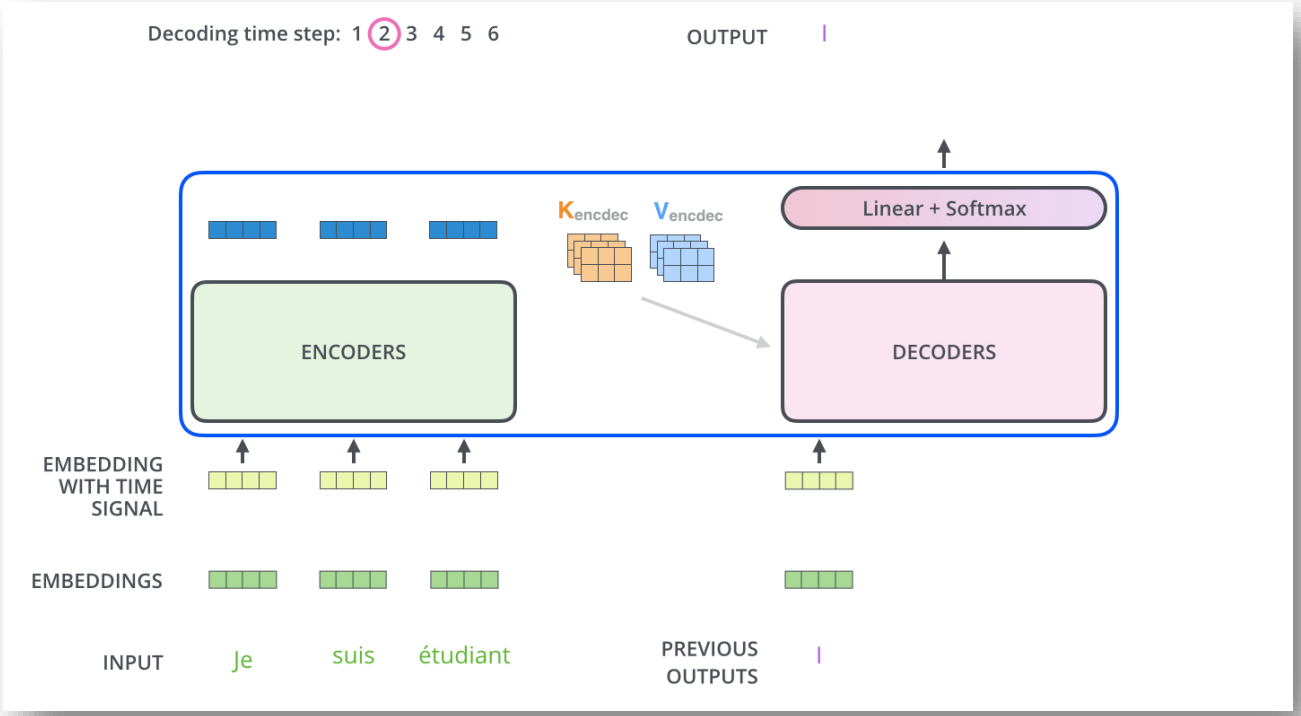
The Decoder in Transformer

- Encoder-decoder attention
  - Q from self-attn in decoder, K & V from encoder outputs
- Masked multi-head attention
  - Design similar to that of encoder, except for decoder #1 which takes additional inputs (of GT/predicted word embeddings).



The Decoder in Transformer (cont'd)

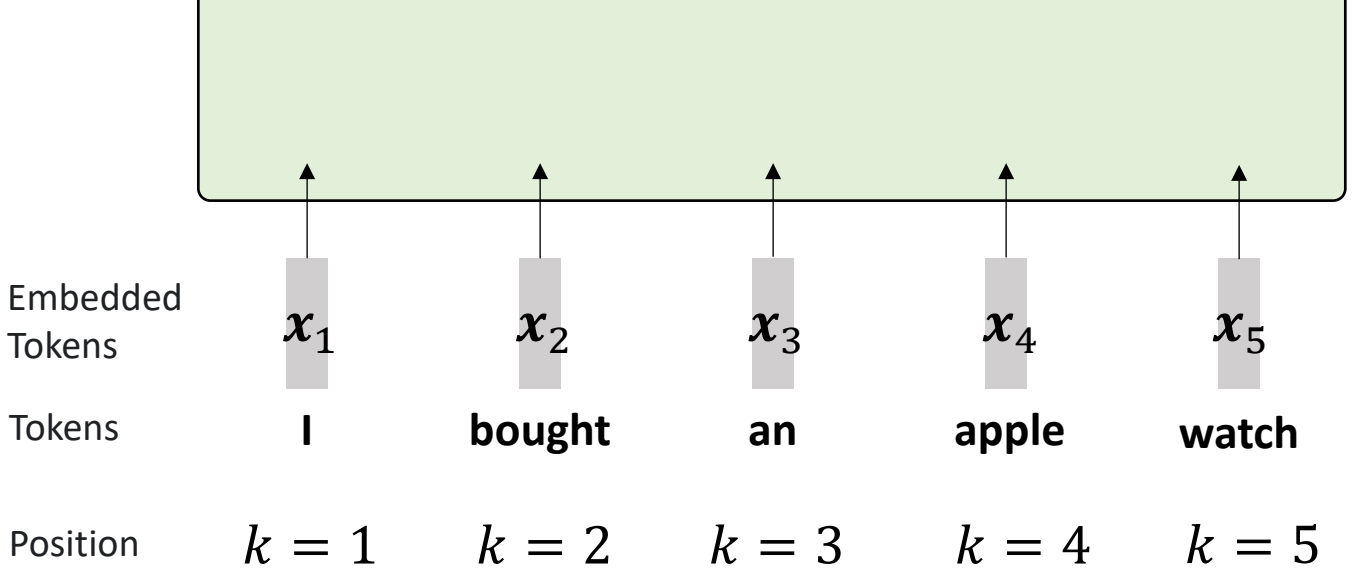
- Encoder-decoder attention
  - Q from self-attn in decoder, K & V from encoder outputs
- Masked multi-head attention
  - Design similar to that of encoder, except for decoder #1 which takes additional inputs (of GT/predicted word embeddings).
  - Mask unpredicted tokens during softmax: why?



Overview of Decoding in Transformer

- Encoder/Decoder Cross-Attention + Decoder self-attention

https://medium.com/ml-note/autoencoder-%E4%B8%89-self-attention-transformer-c37f719d222



Position k

Angular frequency

$w_i = N^{-2i/d}$

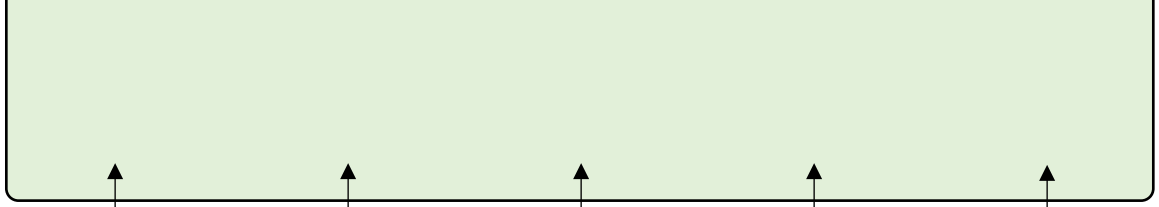
$N = 100{,}000$

$$d \begin{bmatrix} \sin(w_0 k) \\ \cos(w_0 k) \\ \sin(w_1 k) \\ \cos(w_1 k) \\ \vdots \\ \vdots \\ \sin\left(w_{\frac{d}{2}-1} k\right) \\ \cos\left(w_{\frac{d}{2}-1} k\right) \end{bmatrix}$$

← Fast oscillating

← Slow oscillating

Slide credit: Prof. J.-B. Huang @ U. Maryland

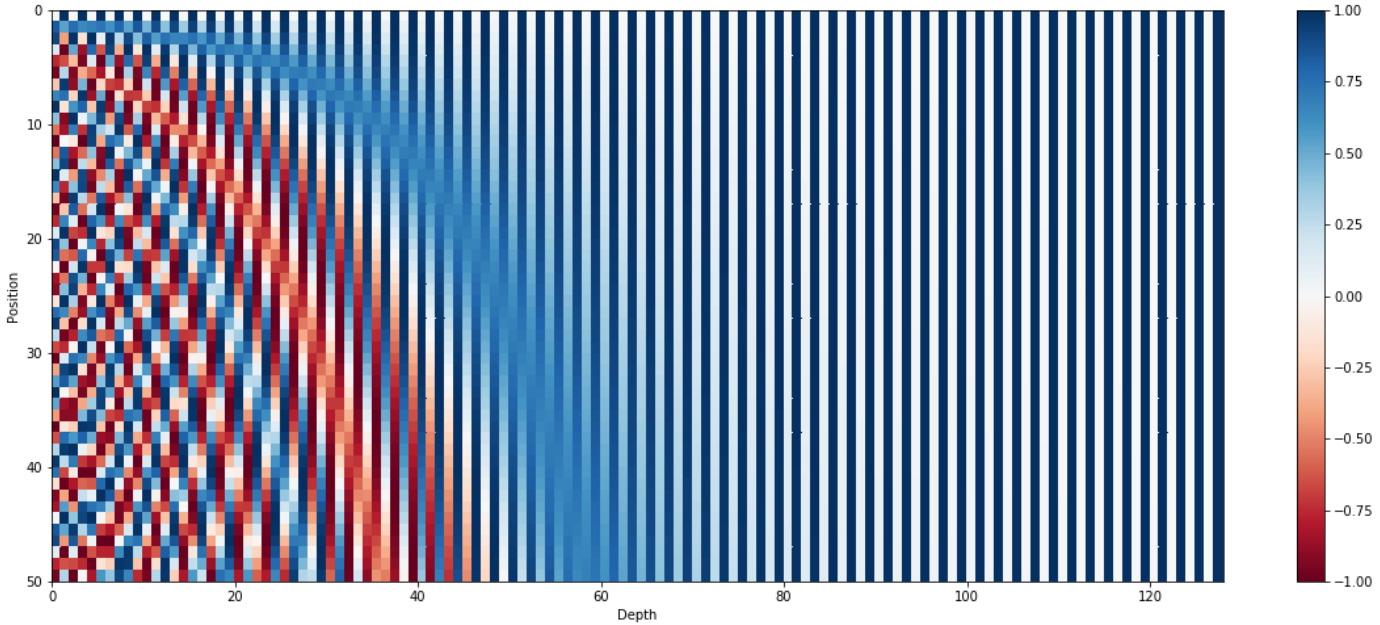


Embedded Tokens $\boldsymbol{x}_1$ $\boldsymbol{x}_2$ $\boldsymbol{x}_3$ $\boldsymbol{x}_4$ $\boldsymbol{x}_5$

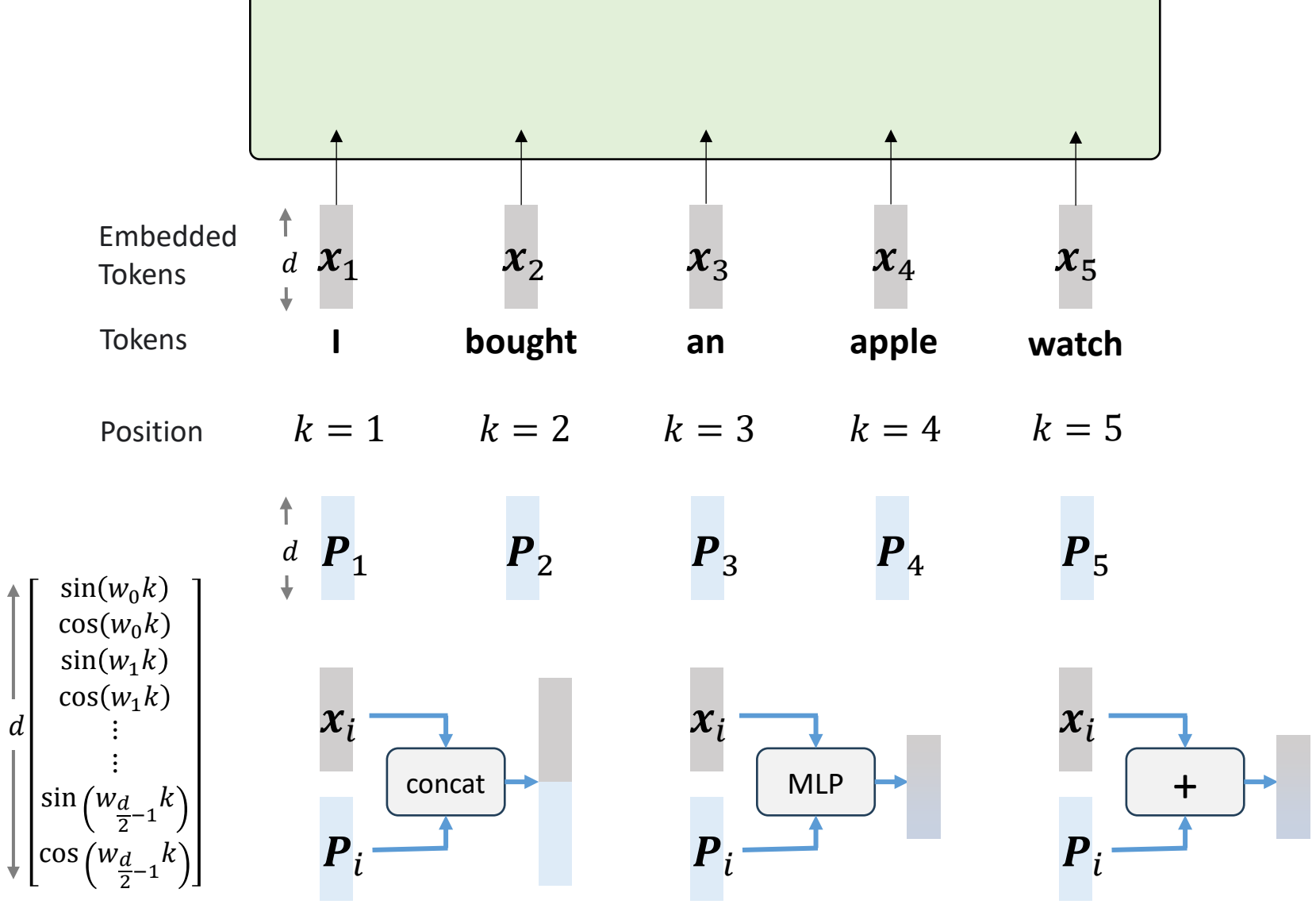
Tokens **I** **bought** **an** **apple** **watch**

Position $k = 1$ $k = 2$ $k = 3$ $k = 4$ $k = 5$

$$d \begin{bmatrix} \sin(w_0 k) \\ \cos(w_0 k) \\ \sin(w_1 k) \\ \cos(w_1 k) \\ \vdots \\ \vdots \\ \sin\left(w_{\frac{d}{2}-1} k\right) \\ \cos\left(w_{\frac{d}{2}-1} k\right) \end{bmatrix}$$



Slide credit: Prof. J.-B. Huang @ U. Maryland



Slide credit: Prof. J.-B. Huang @ U. Maryland

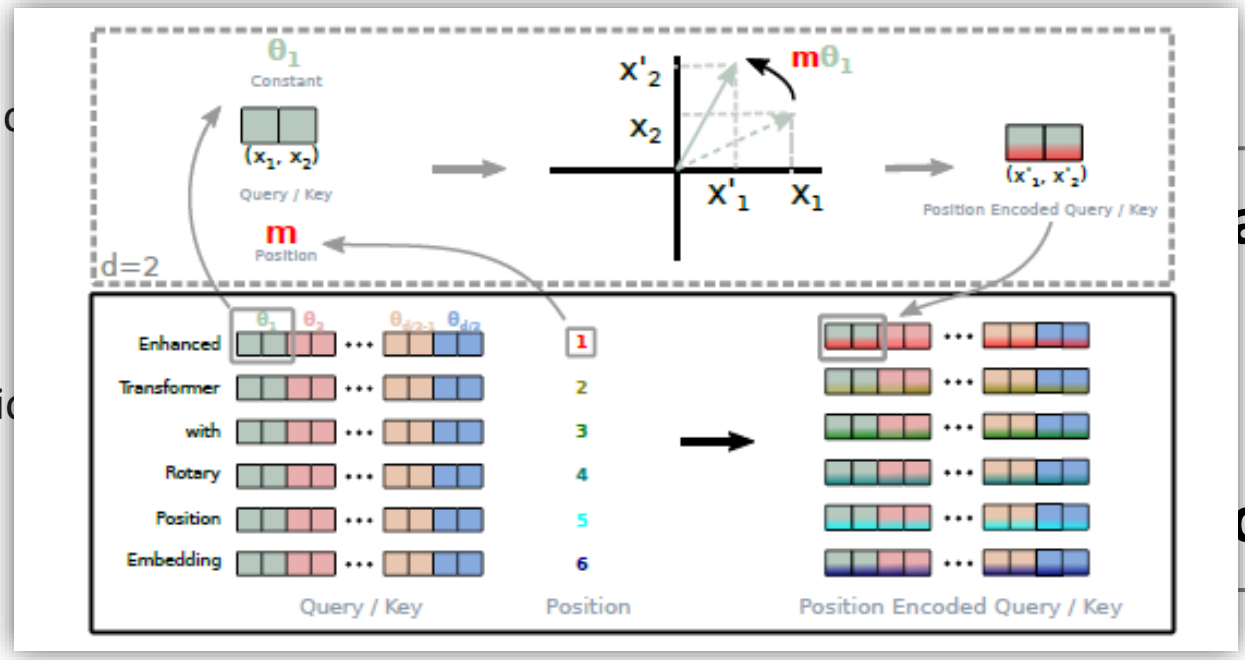
Position	1	2	3	4	5	6
	I	**walk**	**my**	**dog**	**every**	**day**

Position	1	2	3	4	5	6
	every	**day**	**I**	**walk**	**my**	**dog**

Slide credit: Prof. J.-B. Huang @ U. Maryland

Positi

Positi



ay

og

- From *absolute* to *relative* positional embedding

  "RoFormer: Enhanced Transformer with Rotary Position Embedding", arxiv 2021

Slide credit: Prof. J.-B. Huang @ U. Maryland

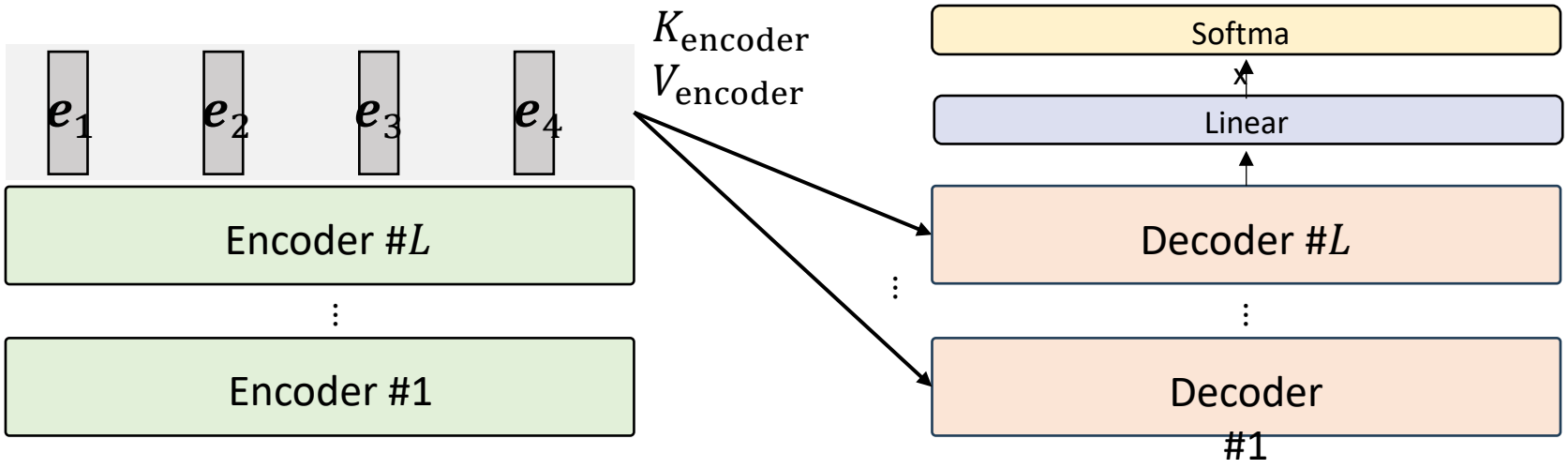
Encoder-Decoder Transformer

Examples:

Attention is all you need, T5, BART.

Good for:

Machine translation, summarization. QA (when input/target are sufficiently different)



Slide credit: Prof. J.-B. Huang @ U. Maryland

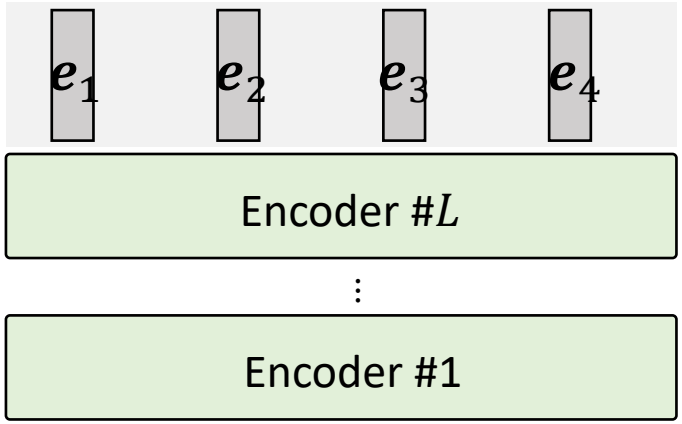
Encoder-Decoder Transformer

Examples:

Attention is all you need, T5, BART.

Good for:

Machine translation, summarization. QA (when input/target are sufficiently different)



Slide credit: Prof. J.-B. Huang @ U. Maryland

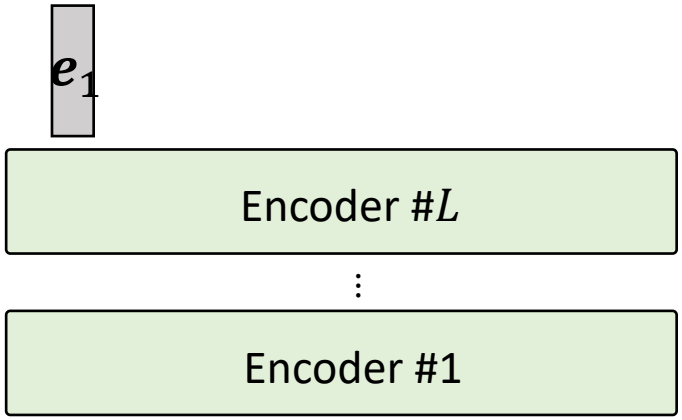
Encoder-only Transformer

Examples:

BERT, RoBERTa, DeBERTa, X-BERT

Good for:

Classification, sequence tagging,
sentiment analysis, etc.
(Understand text, but not generate them)



Slide credit: Prof. J.-B. Huang @ U. Maryland

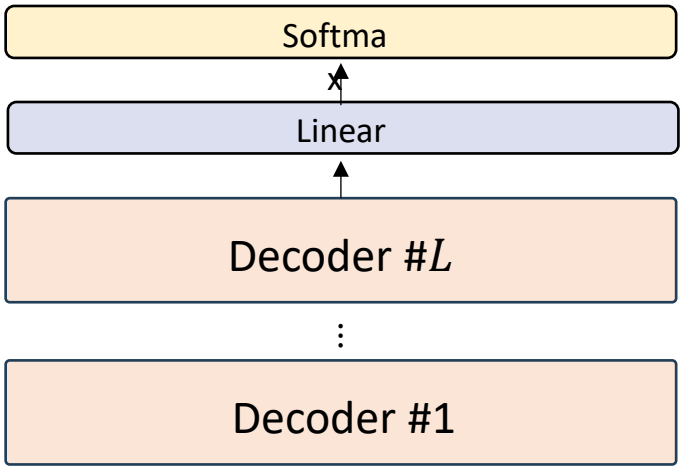
Decoder-only Transformer

Examples:

GPT-X (OpenAI), PaLM (Google), LLaMA (Meta)
BLOOM (BigScience)

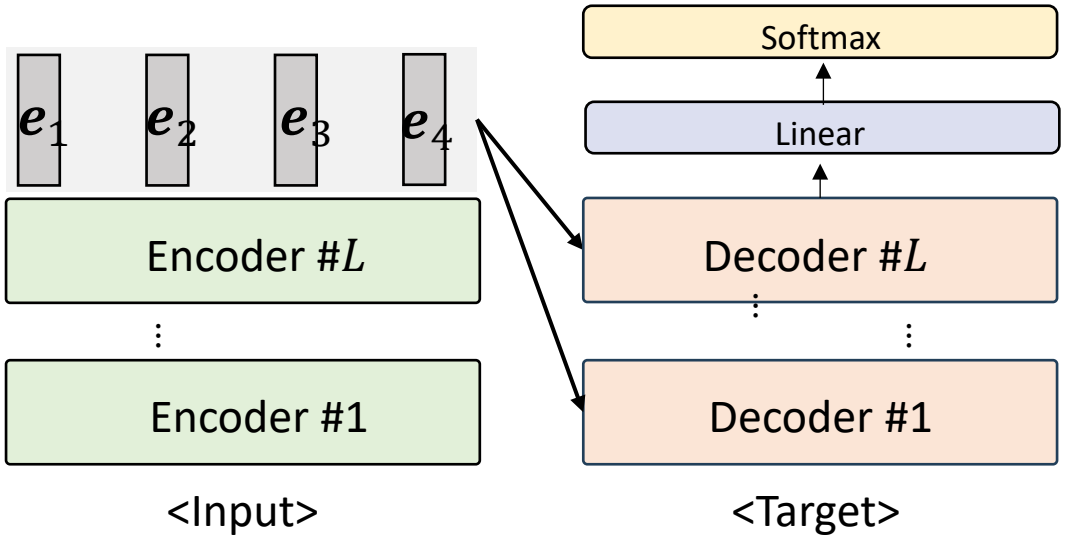
Good for:

Text generation, multi-round conversation, etc.



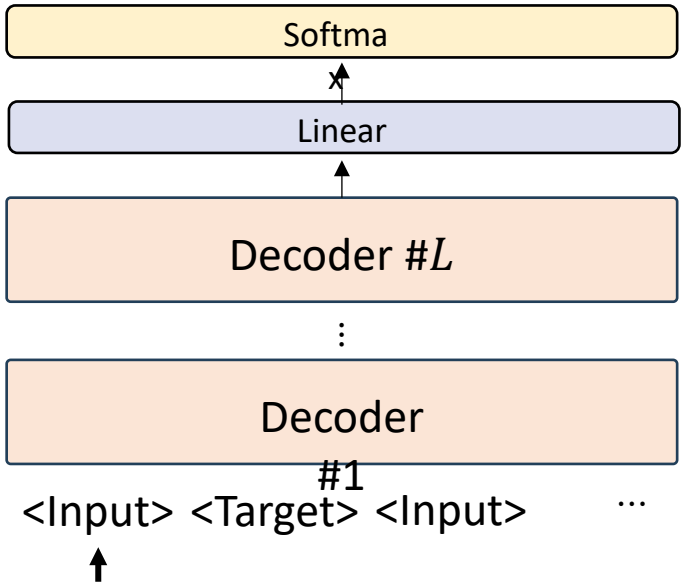
Slide credit: Prof. J.-B. Huang @ U. Maryland

Encoder-Decoder Transformer



Different parameters for encoder/decoder

Decoder-only Transformer



Shared parameters

Slide credit: Prof. J.-B. Huang @ U. Maryland

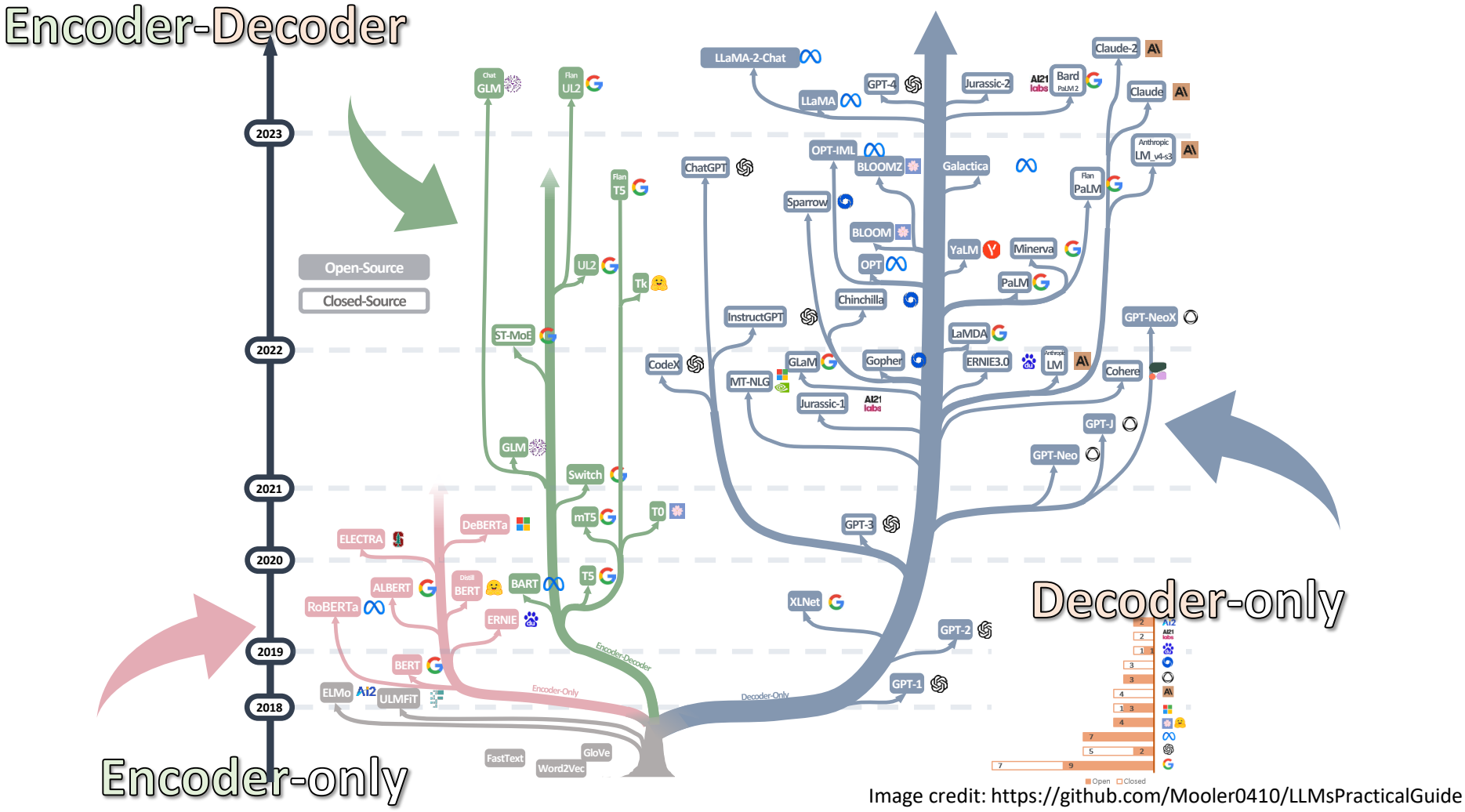
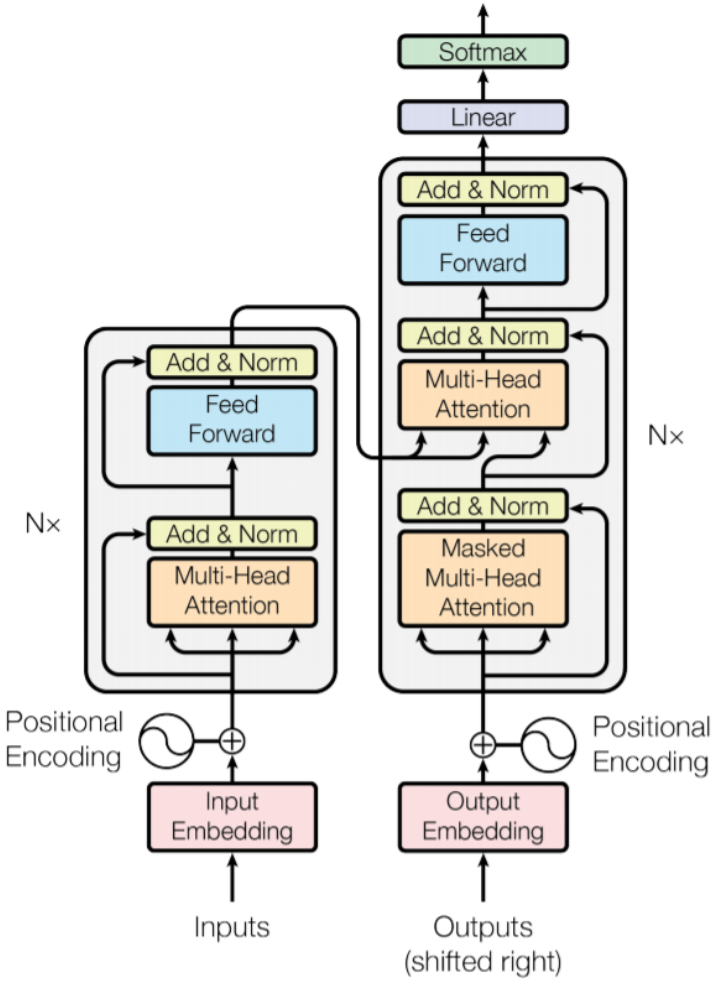


Image credit: https://github.com/Mooler0410/LLMsPracticalGuide

Slide credit: Prof. J.-B. Huang @ U. Maryland

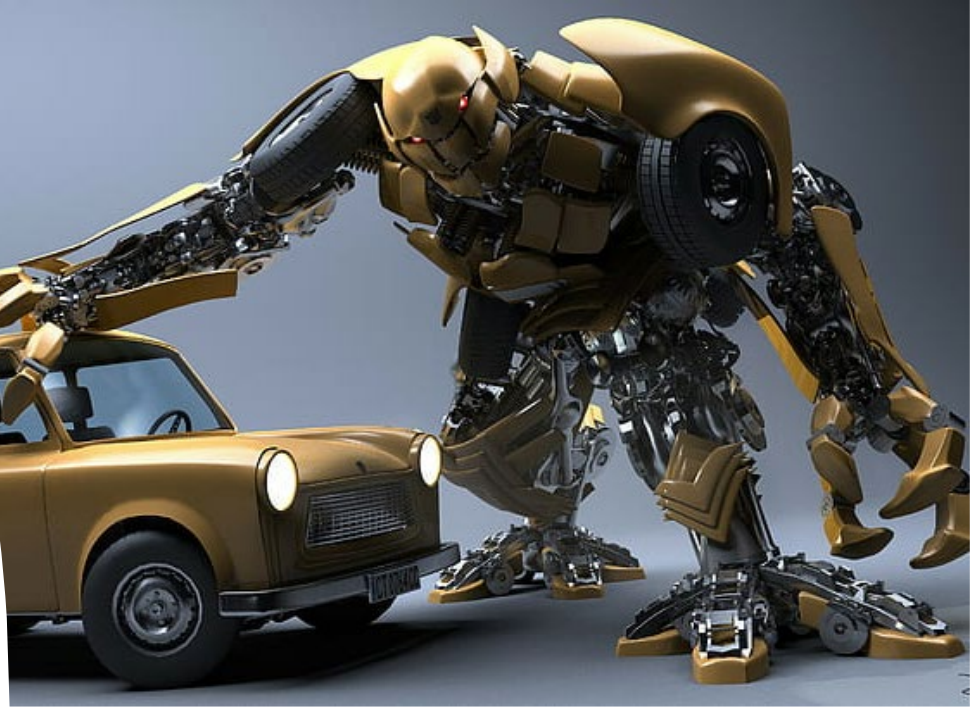
Transformer is promising, but…

- Concerns of Transformer?
  - Computation
  - Space/memory
- Potential solutions
  - Sparse Transformer
  - Linformer
  - Linearized attention, etc.
  - Ever heard of *Mamba*?



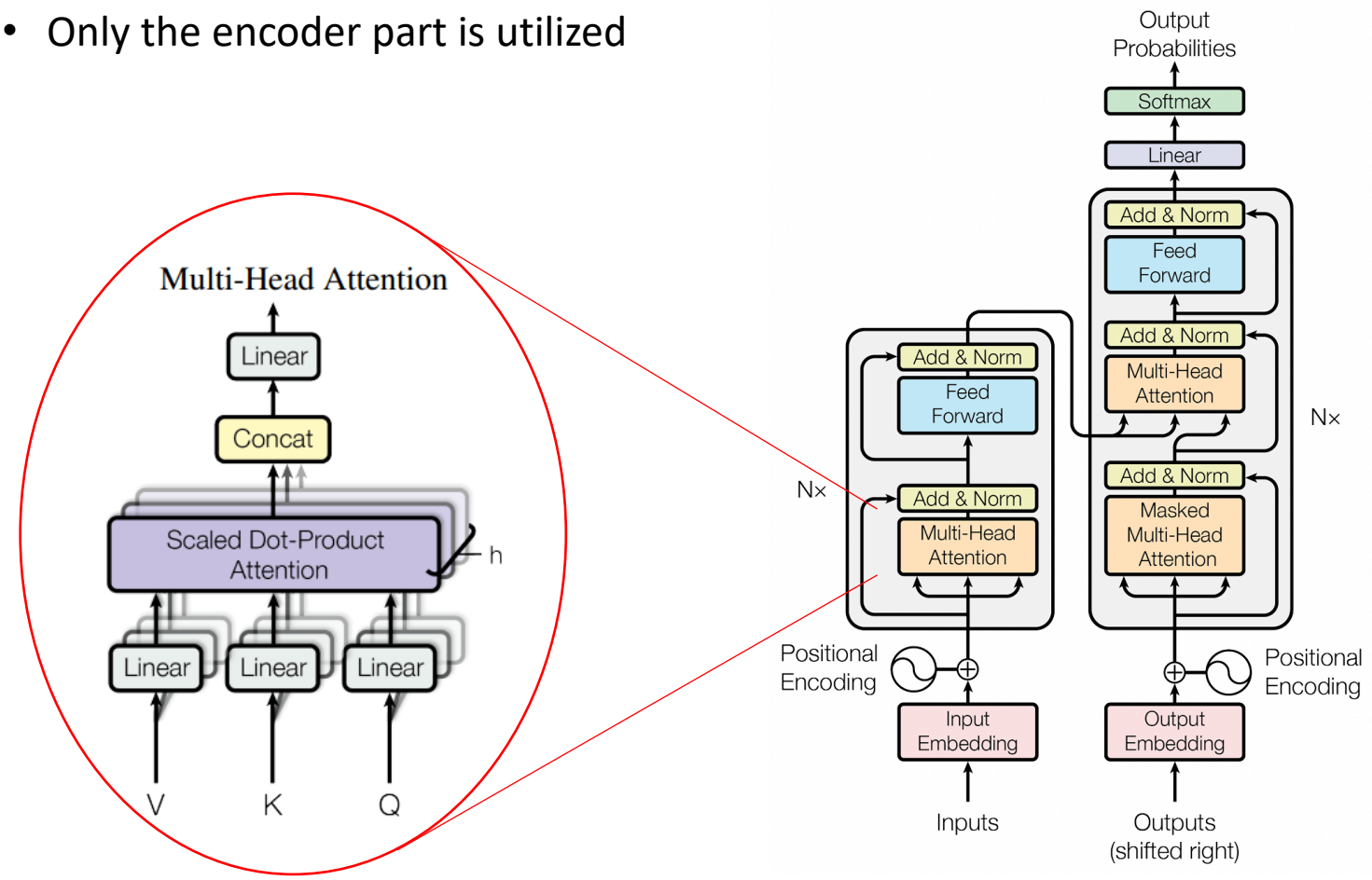
What to Be Covered?

- Transformer
- Transformer for Visual Analysis
  - Vision Transformer (ViT)
  - SSL & Beyond
- Vision-Language Model
  - Image2Text
  - Text2Image
- Pretraining vs. Finetuning
  - Parameter-Efficient Finetuning (PEFT)



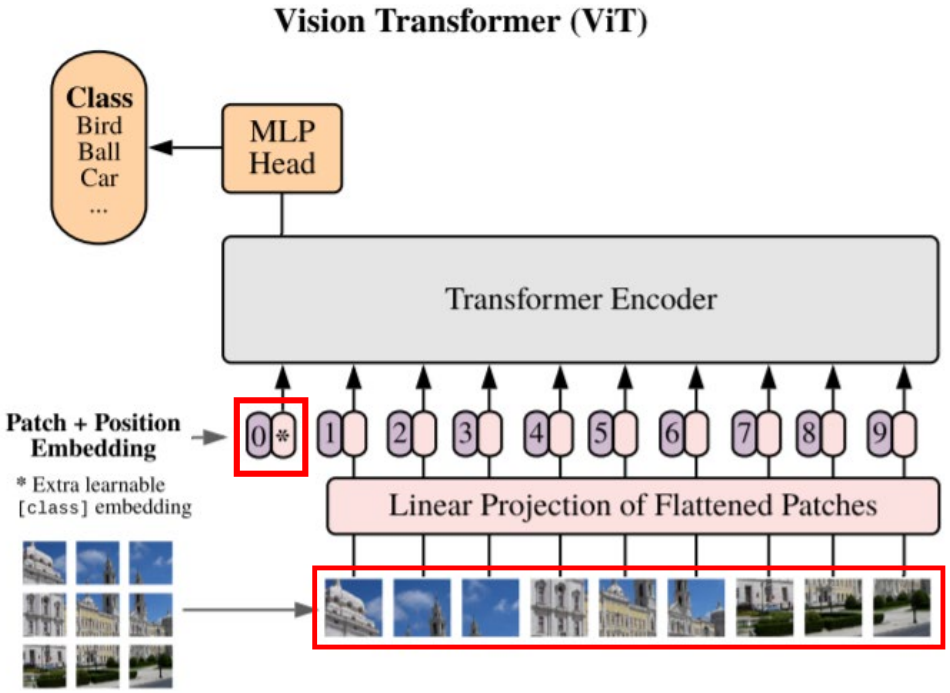
Vision Transformer

- "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR, 2021. (Google Research)
- Only the encoder part is utilized



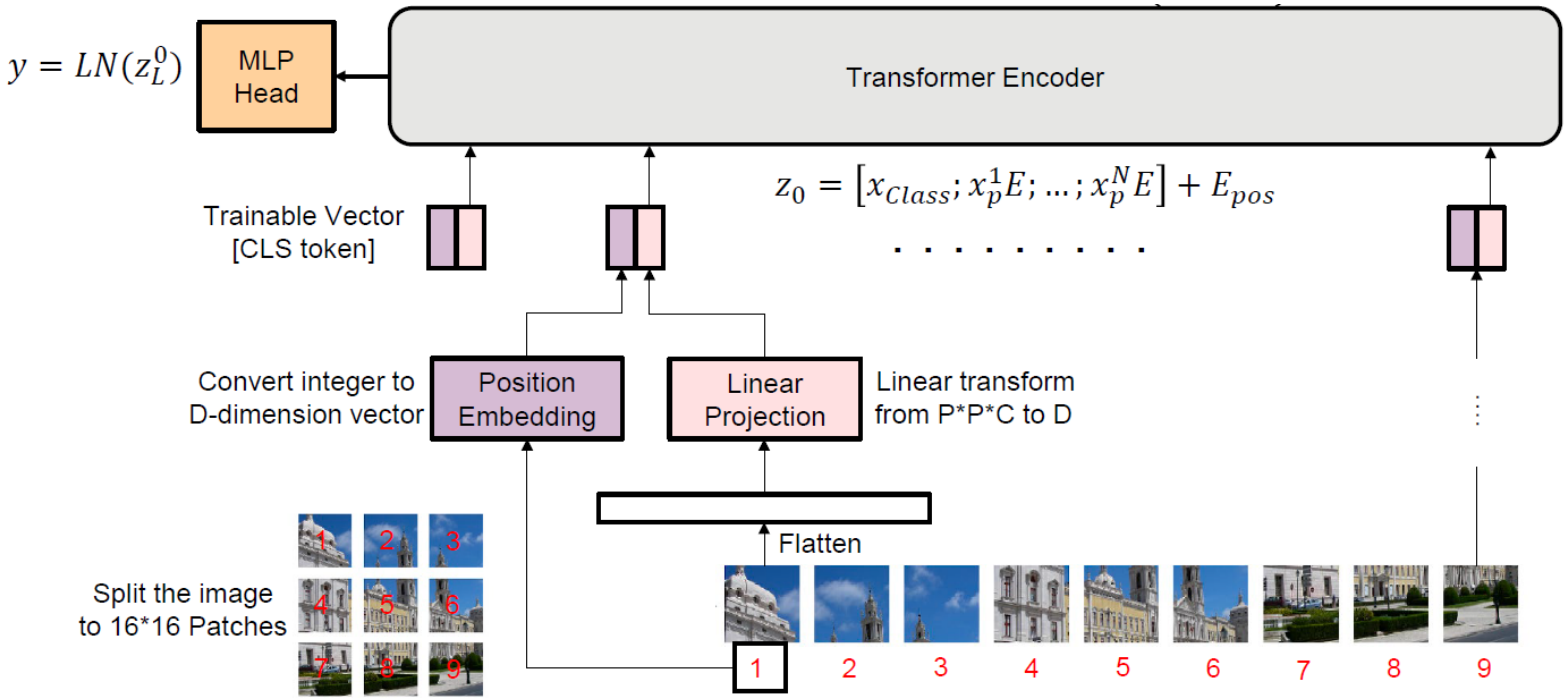
Vision Transformer (cont'd)

- Partition the input image into a **patch sequence**
- An additional **token** (*) is appended to perform attention on patches
- Both the "*" token and positional embeddings (denoted by 0, 1, 2 …) are **trainable vectors**.



Vision Transformer (cont'd)

- "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR, 2021. (Google Research)



$$\mathbf{z}_0 = [\mathbf{x}_{\text{class}}; \mathbf{x}_p^1\mathbf{E}; \mathbf{x}_p^2\mathbf{E}; \cdots; \mathbf{x}_p^N\mathbf{E}] + \mathbf{E}_{pos}, \qquad \mathbf{E} \in \mathbb{R}^{(P^2 \cdot C) \times D},\ \mathbf{E}_{pos} \in \mathbb{R}^{(N+1) \times D}$$

$$\mathbf{z}'_\ell = \text{MSA}(\text{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1}, \qquad \ell = 1 \ldots L \qquad \text{Multiheaded self-attention (MSA)}$$

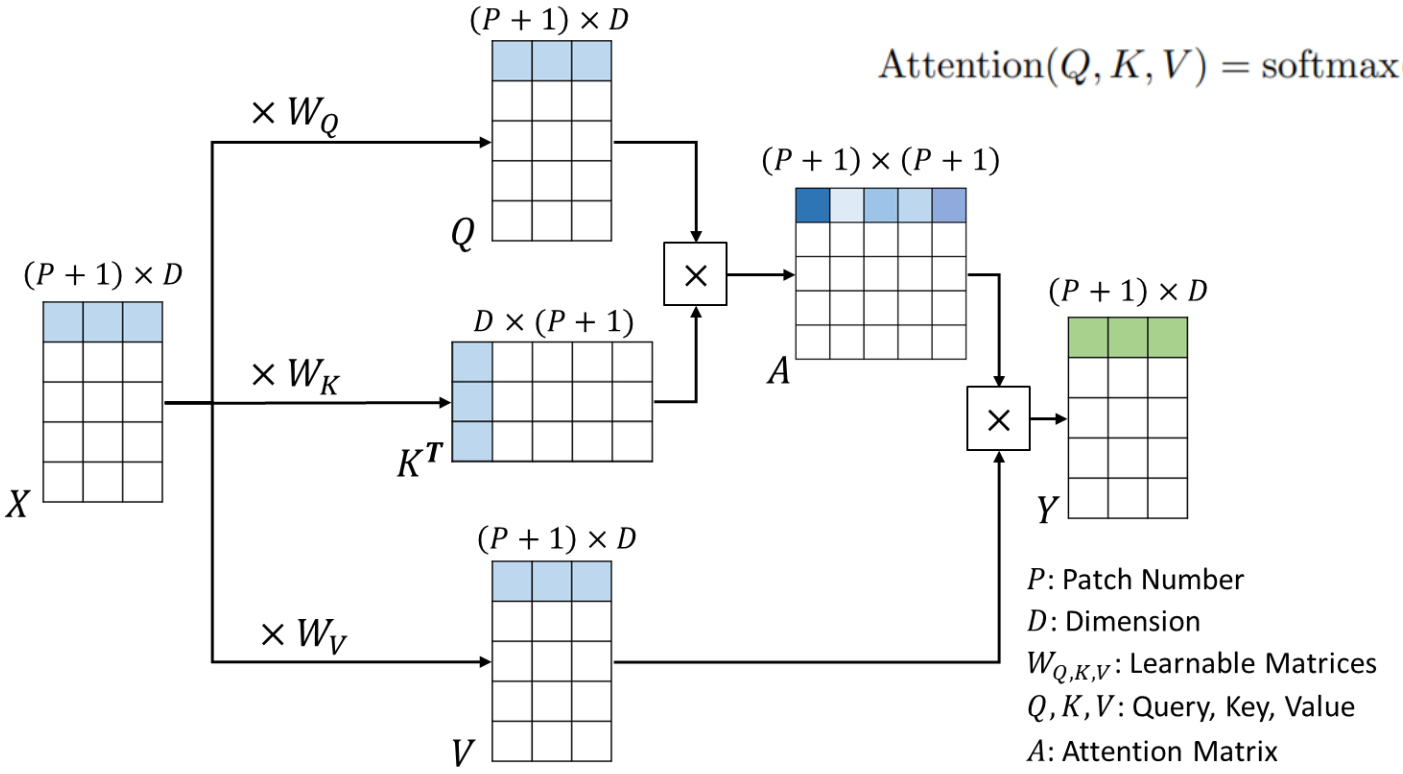
$$\mathbf{z}_\ell = \text{MLP}(\text{LN}(\mathbf{z}'_\ell)) + \mathbf{z}'_\ell, \qquad \ell = 1 \ldots L \qquad \text{Layer norm (LN)}$$

$$\mathbf{y} = \text{LN}(\mathbf{z}_L^0)$$

Figure credit: CS886 Univ. Waterloo

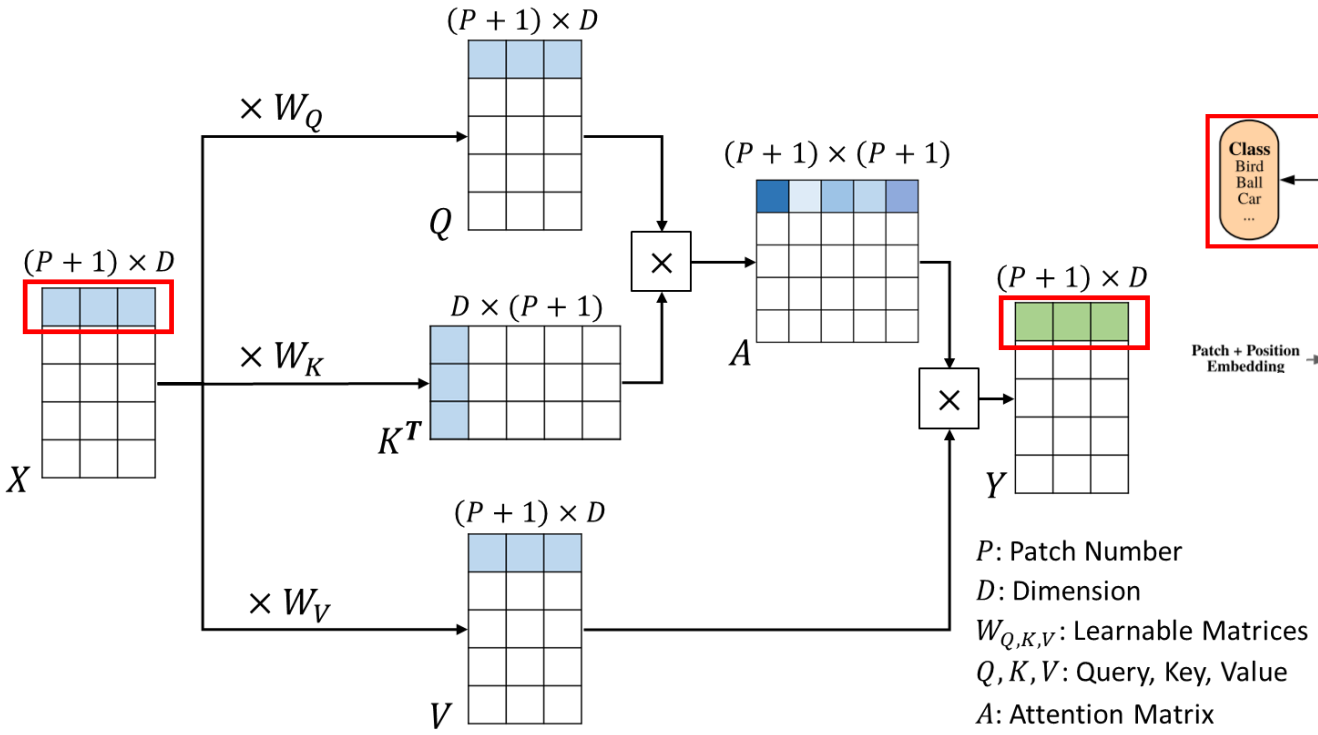
Query-Key-Value Attention in ViT

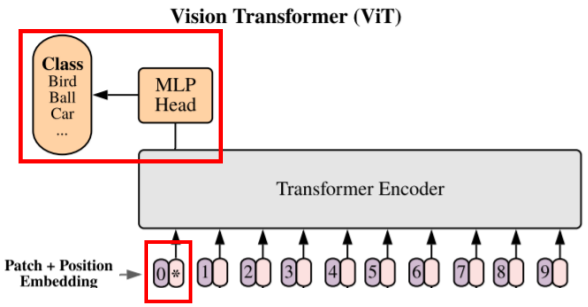
- E.g., An input image is partitioned into 4 patches, with feature dimension = 3 (i.e., P=4 and D=3).
- Note that there are (P+1) rows since we have an additional token "*".



Query-Key-Value Attention in ViT (cont'd)

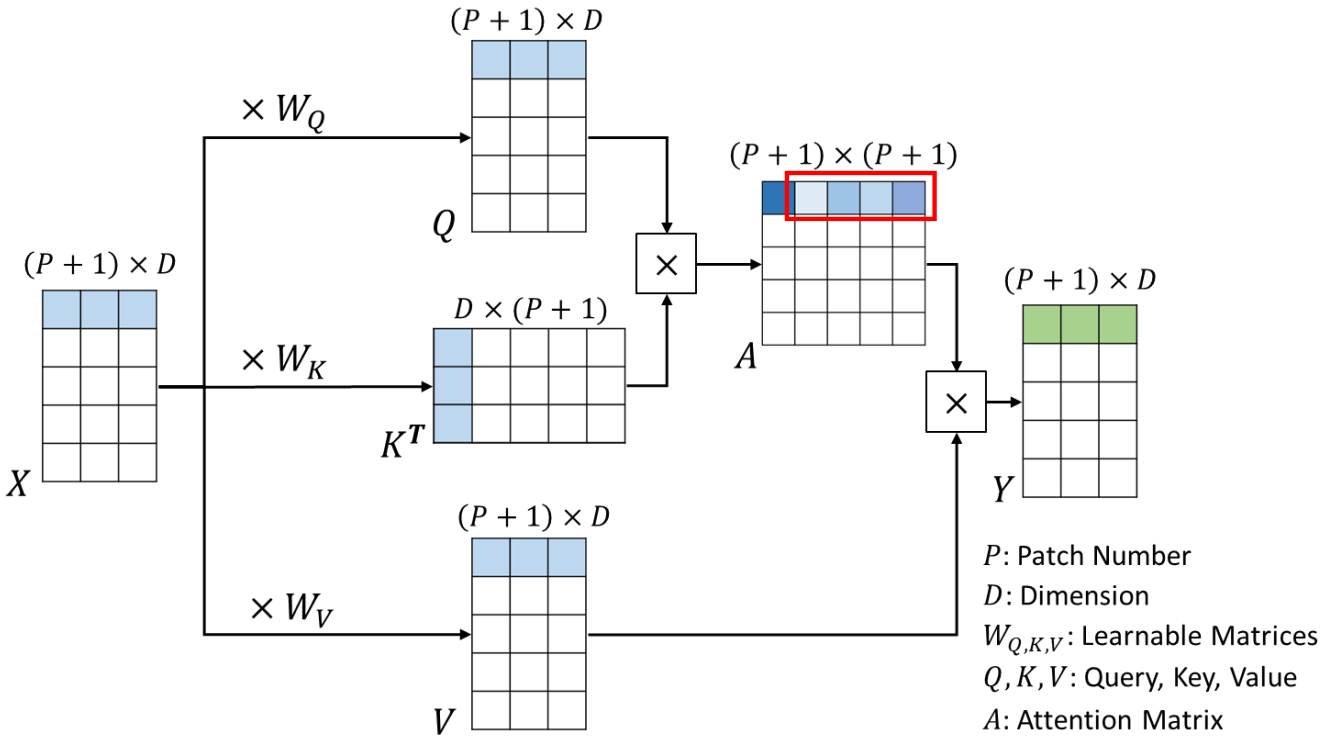
- In the standard vision transformer, we only take the **first output token** of the output sequence (the **first row** of Y) for classification purposes
- This corresponds to the output when **token "0"** serves as query





Visualization of ViT

- To visualize the **attention maps**, we take the attention scores from the **first row** of A (when token "0" serves as query)
- Note the first element is excluded, and thus there are **P scores** corresponding to the P image patches

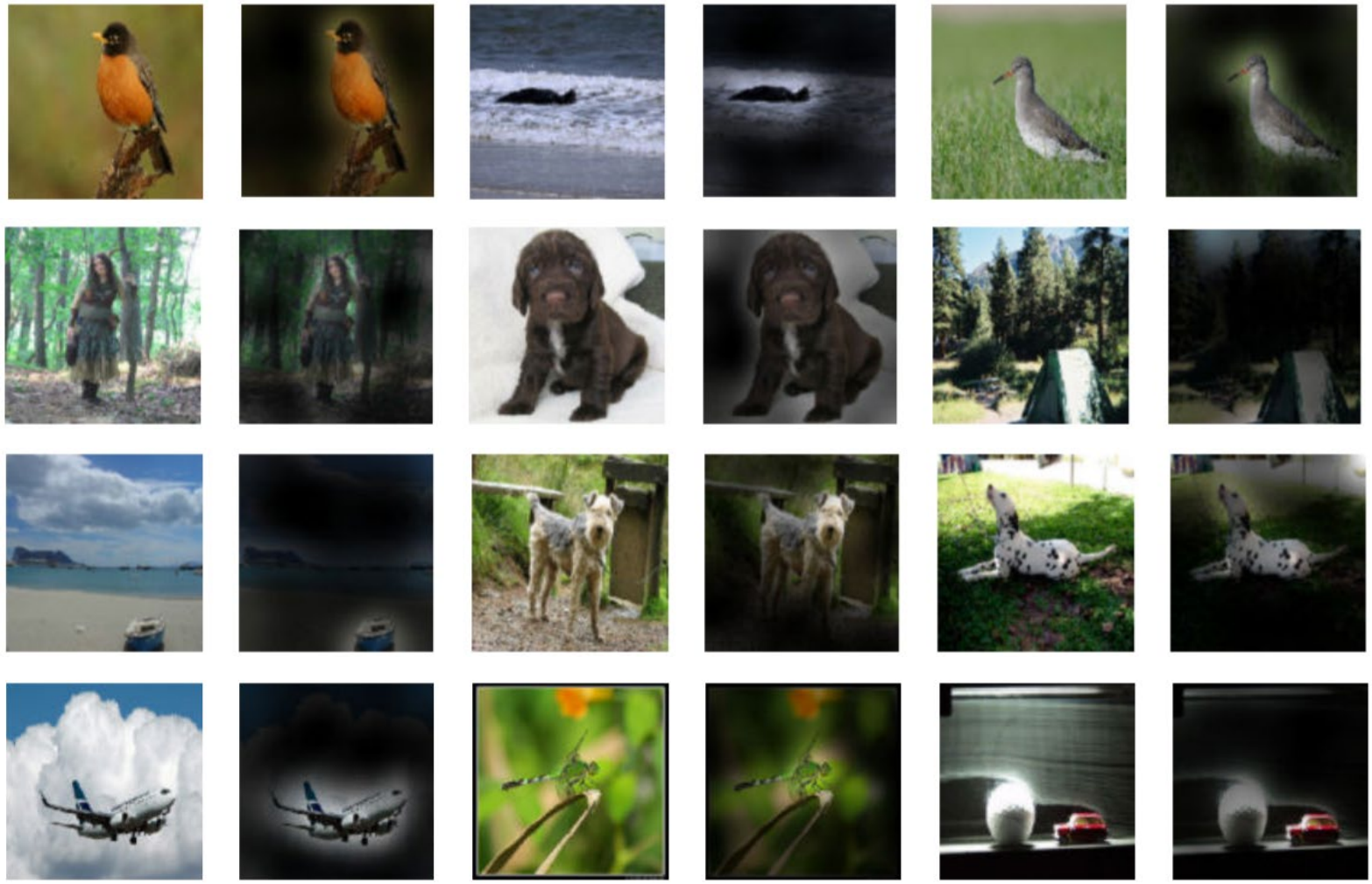


ViT Results

- ViT outperforms CNN-based models
  - Pretrained on JFT & ImageNet
  - Can be trained using TPUv3 w/ 8 cores in 30 days; faster than CNN
  - ViT for visualization/interpretability?

	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	**88.55** ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet ReaL	**90.72** ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	**99.50** ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	–
CIFAR-100	**94.55** ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	–
Oxford-IIIT Pets	**97.56** ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	–
Oxford Flowers-102	99.68 ± 0.02	**99.74** ± 0.00	99.61 ± 0.02	99.63 ± 0.03	–
VTAB (19 tasks)	**77.63** ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	–
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

Example Visualization for Object Recognition



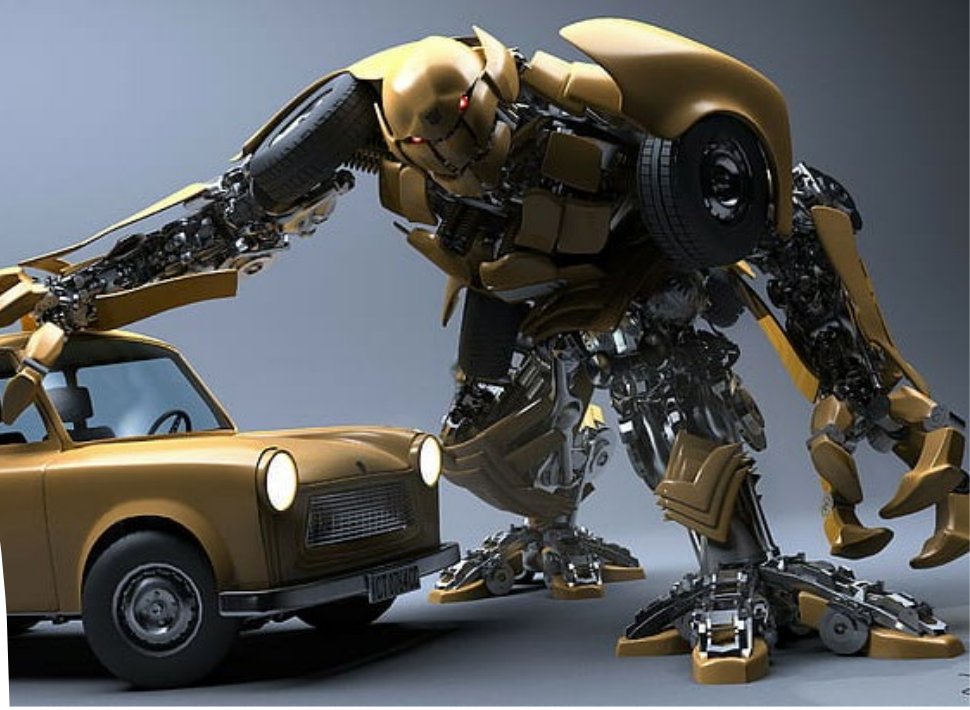
ViT Results

- ViT outperforms CNN-based models
  - Pretrained on JFT & ImageNet
  - Can be trained using TPUv3 w/ 8 cores in 30 days; faster than CNN
  - However, JFT is not publicly available…

	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	**88.55** ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet ReaL	**90.72** ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	**99.50** ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	–
CIFAR-100	**94.55** ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	–
Oxford-IIIT Pets	**97.56** ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	–
Oxford Flowers-102	99.68 ± 0.02	**99.74** ± 0.00	99.61 ± 0.02	99.63 ± 0.03	–
VTAB (19 tasks)	**77.63** ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	–
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

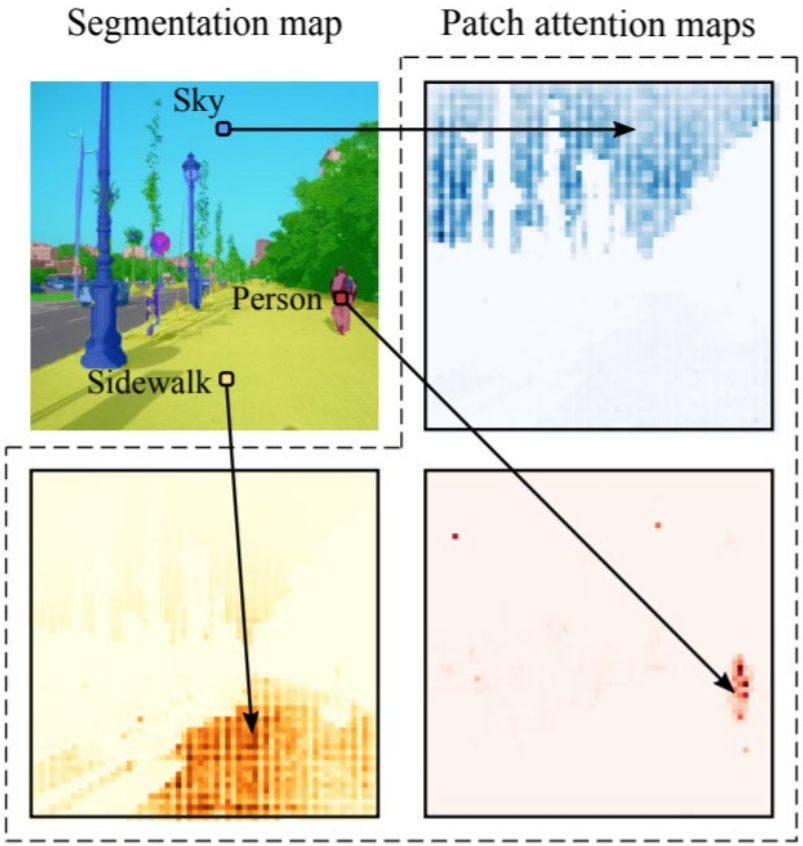
What to Be Covered?

- Transformer
- Transformer for Visual Analysis
  - Vision Transformer (ViT)
  - SSL & Beyond
- Vision-Language Model
  - Image2Text
  - Text2Image
- Pretraining vs. Finetuning
  - Parameter-Efficient Finetuning (PEFT)



Transformer for Semantic Segmentation

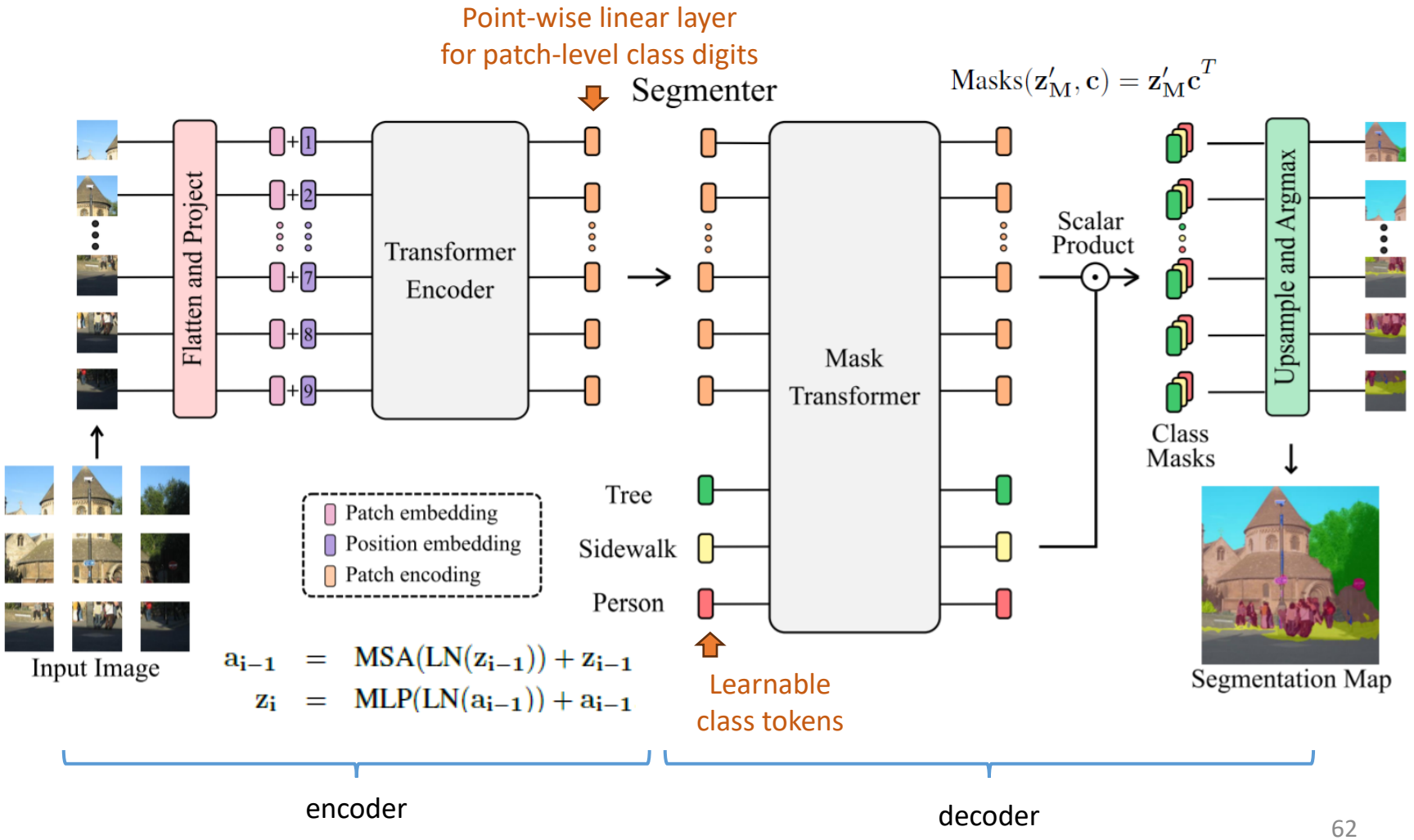
- Segmentation via attention



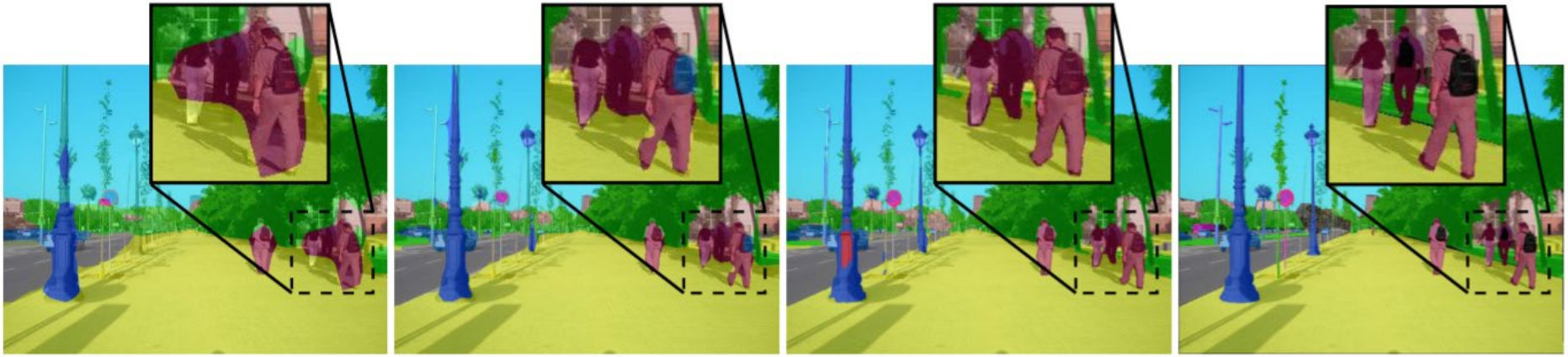
Strudel et al. "Segmenter: Transformer for Semantic Segmentation." ICCV 2021

Transformer for Semantic Segmentation (cont'd)

- Inspired by object detection models of DETR (ECCV'20), etc.



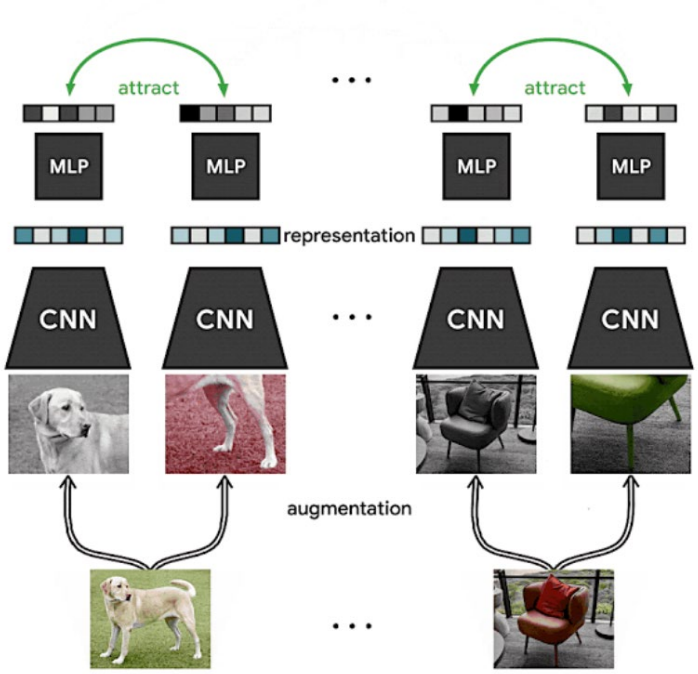
Example Visualization



(a) Patch size 32×32 (b) Patch size 16×16 (c) Patch size 8×8 (d) Ground Truth

Self-Supervised Learning (SSL)

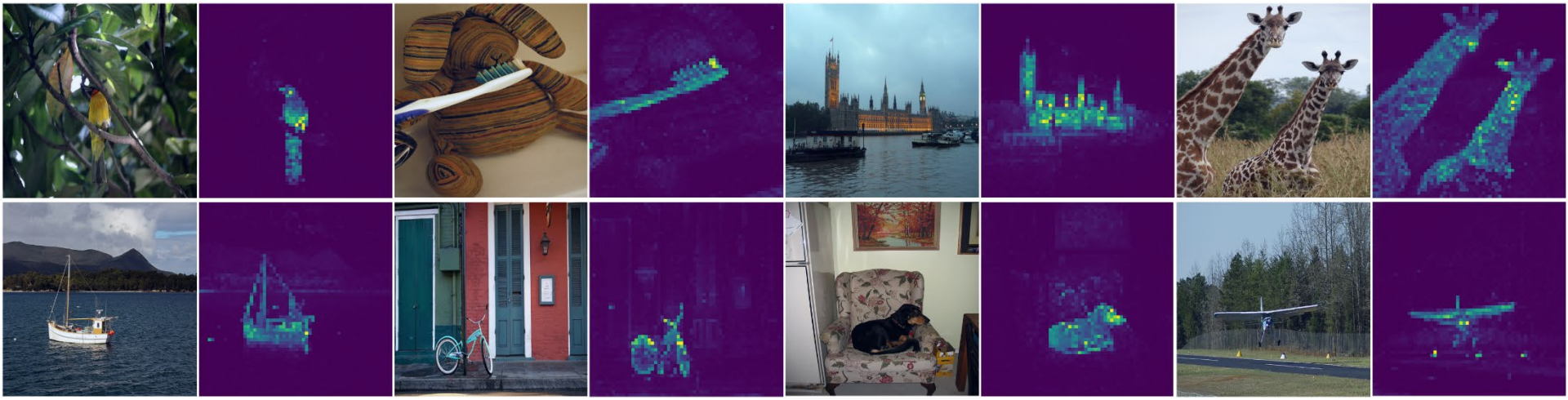
- Learning (somewhat) discriminative representations from **unlabeled** data
- Create self-supervised tasks via **data augmentation**
- Recall: SSL for CNN using image data:



Chen et al. "A simple framework for contrastive learning of visual representations." ICML 2020

Self-Supervised Learning (SSL) for Transformer (cont'd)

- SSL ViT/DeiT features contain info about semantic segmentation
- The above features are excellent k-NN classifiers
- By visualizing self-attention of the CLS token on different heads of the last layer:



Facebook AI Research & INRIA, "Emerging Properties in Self-Supervised Vision Transformers." ICCV 2021

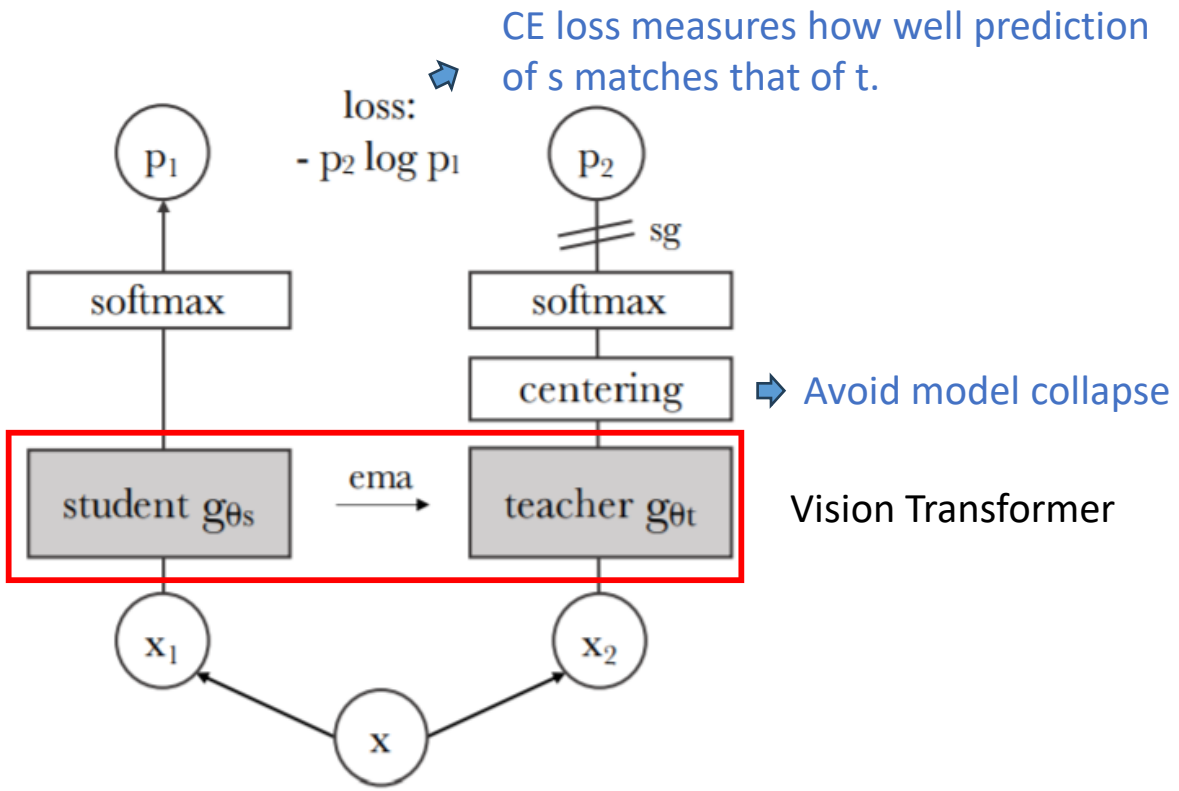
Self-Supervised Learning (SSL) for Transformer (cont'd)

- Illustration of the proposed idea:

Facebook AI Research & INRIA, "Emerging Properties in Self-Supervised Vision Transformers." ICCV 2021

Self-Distillation with No Labels (DINO)

- Vision Transformer + **SSL**
- Maximize the prediction similarity btw input & its augmented version
- Idea: a **teacher-student** network (i.e., knowledge distillation) & **EMA** (exponential moving average)

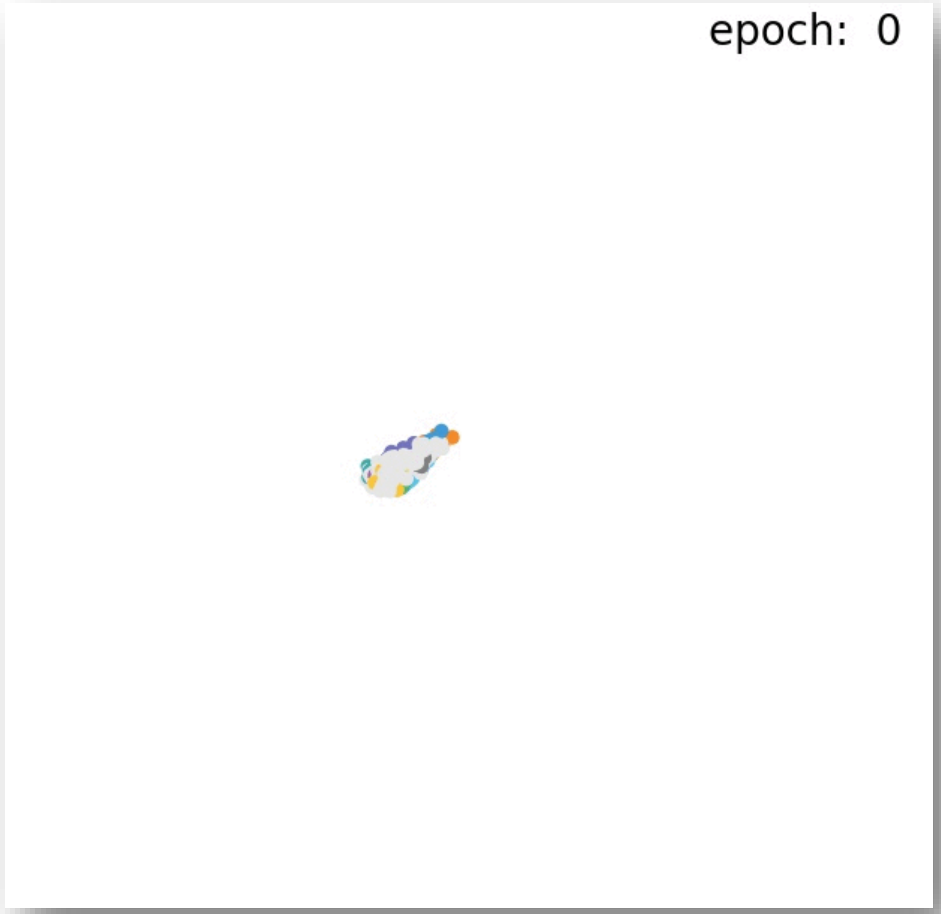


$$\theta_T \leftarrow \tau\theta_T + (1-\tau)\theta_S$$

Caron et al. "Emerging properties in self-supervised vision transformers." ICCV 2021

Self-Supervised Learning (SSL) for Transformer (cont'd)

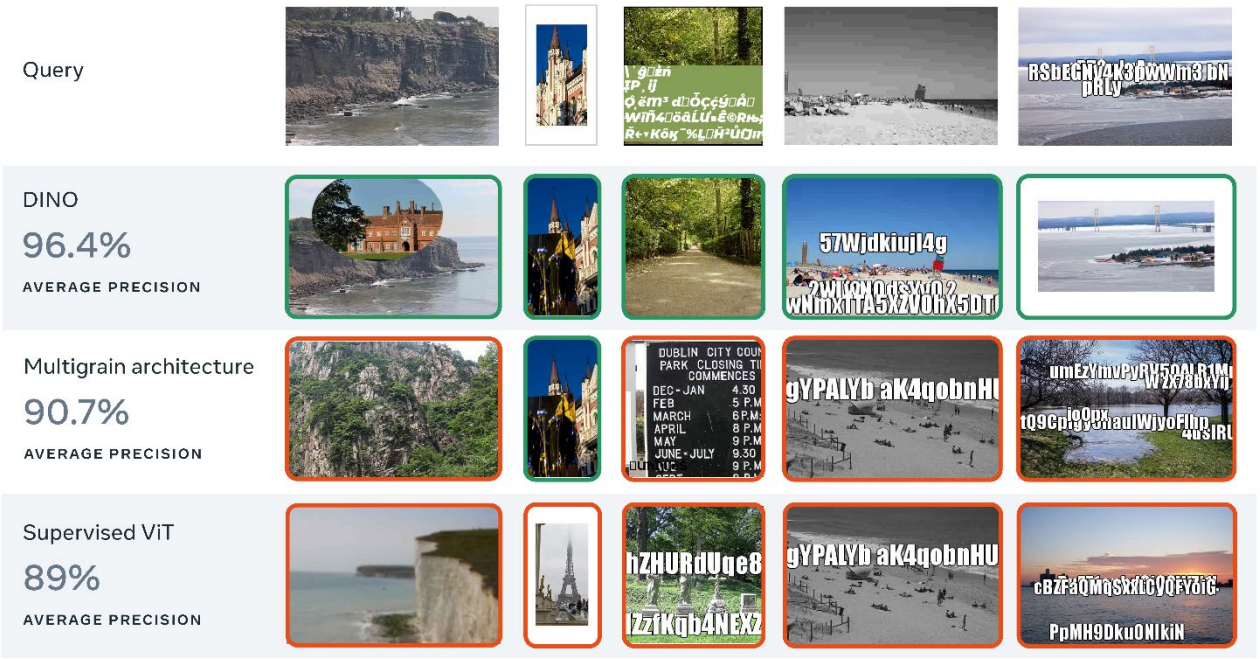
- Illustration of the learned features:



Facebook AI Research & INRIA, "Emerging Properties in Self-Supervised Vision Transformers." ICCV 2021

Highlights & Example Results of DINO

- Learning discriminative representations from **unlabeled** data
- Create self-supervised tasks via **data augmentation**



Facebook AI Research & INRIA, "Emerging Properties in Self-Supervised Vision Transformers." ICCV 2021

What to Be Covered?

- Transformer
- Transformer for Visual Analysis
  - Vision Transformer (ViT)
  - SSL & Beyond
- Vision-Language Model
  - Image2Text
  - Text2Image
- Pretraining vs. Finetuning
  - Parameter-Efficient Finetuning (PEFT)

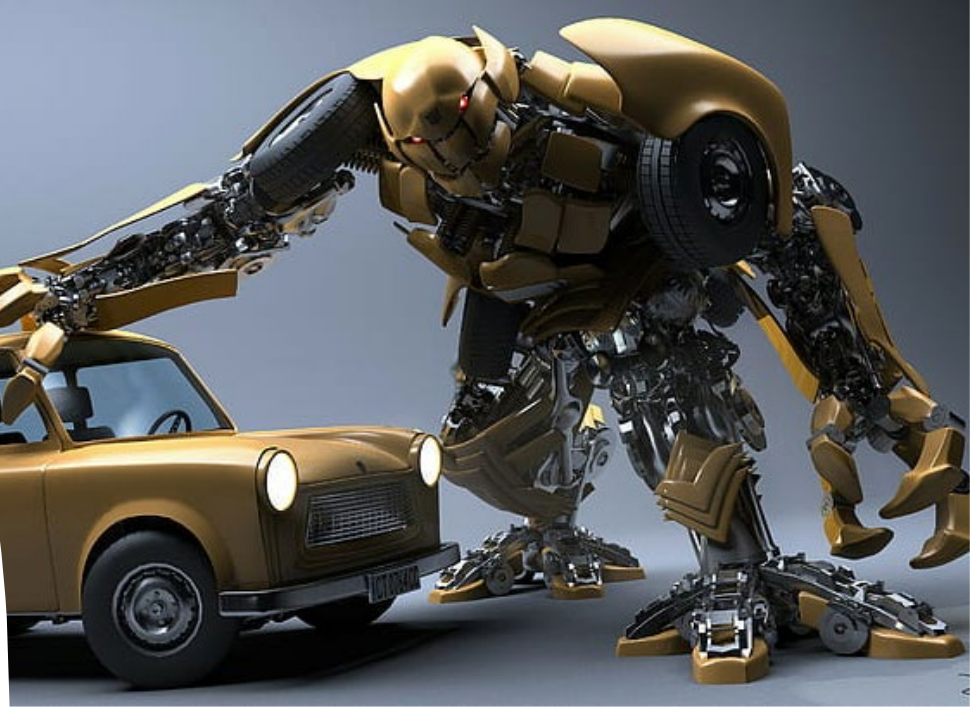


Image Captioning

Applications: semantics understanding, image-text retrieval, medical AI, etc.
How does it help GenAI? (e.g., text-to-image generation)

Image Captioning (cont'd)

- *Image Captioning: Transforming Objects into Words*, Yahoo Research, NeurIPS 2019
- Motivation: mid-level image understanding for captioning
- Framework:

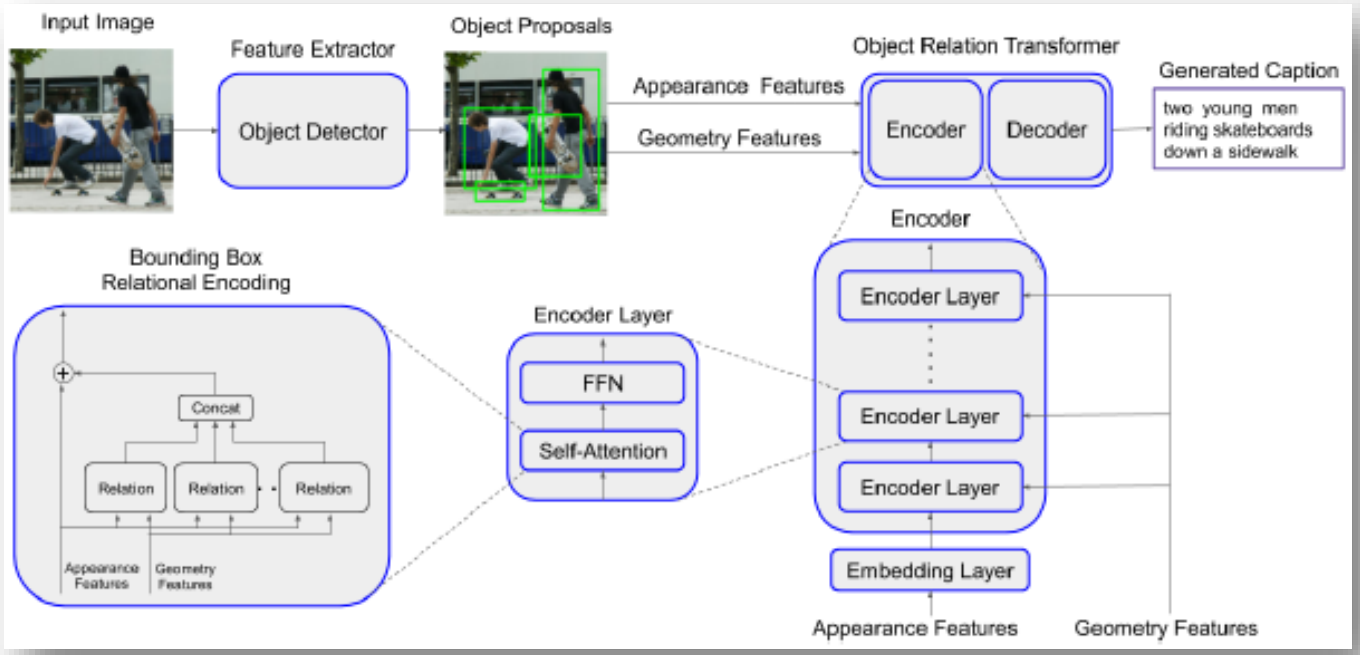
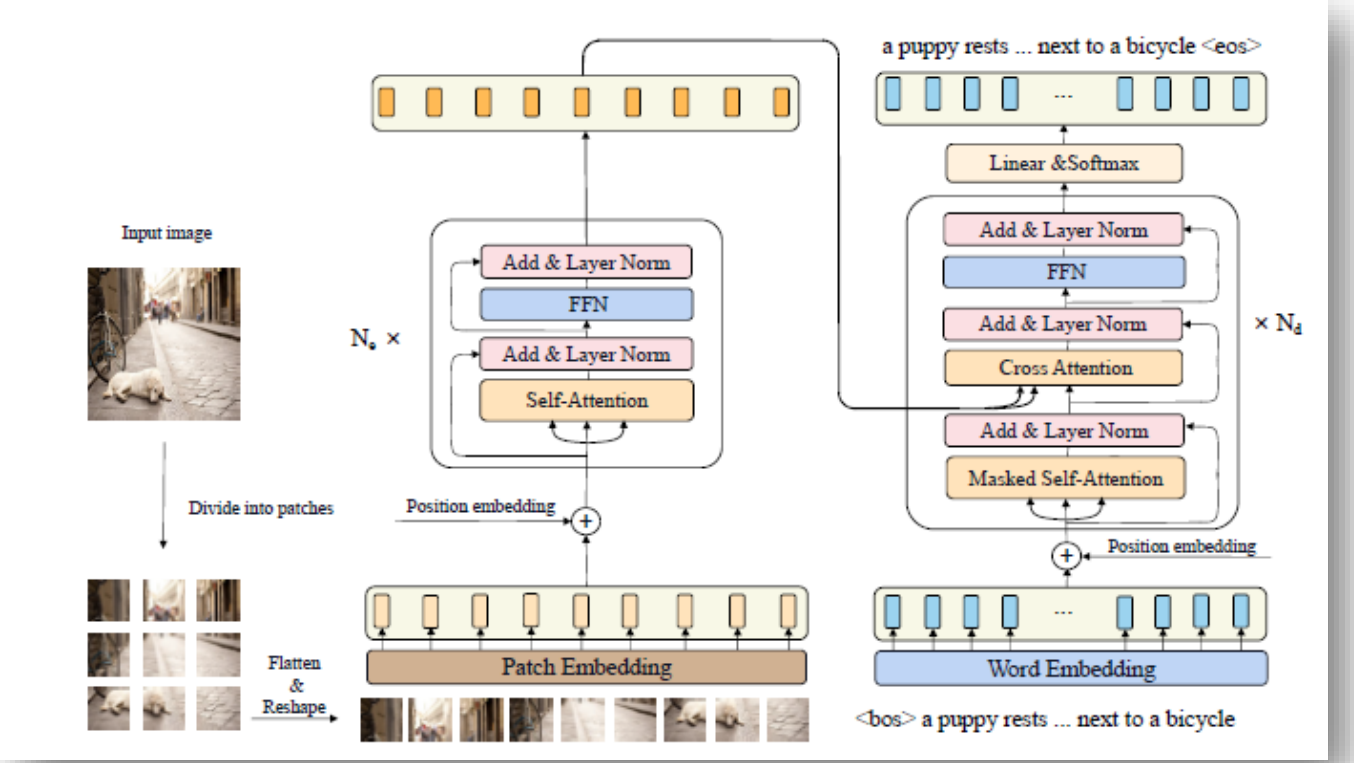


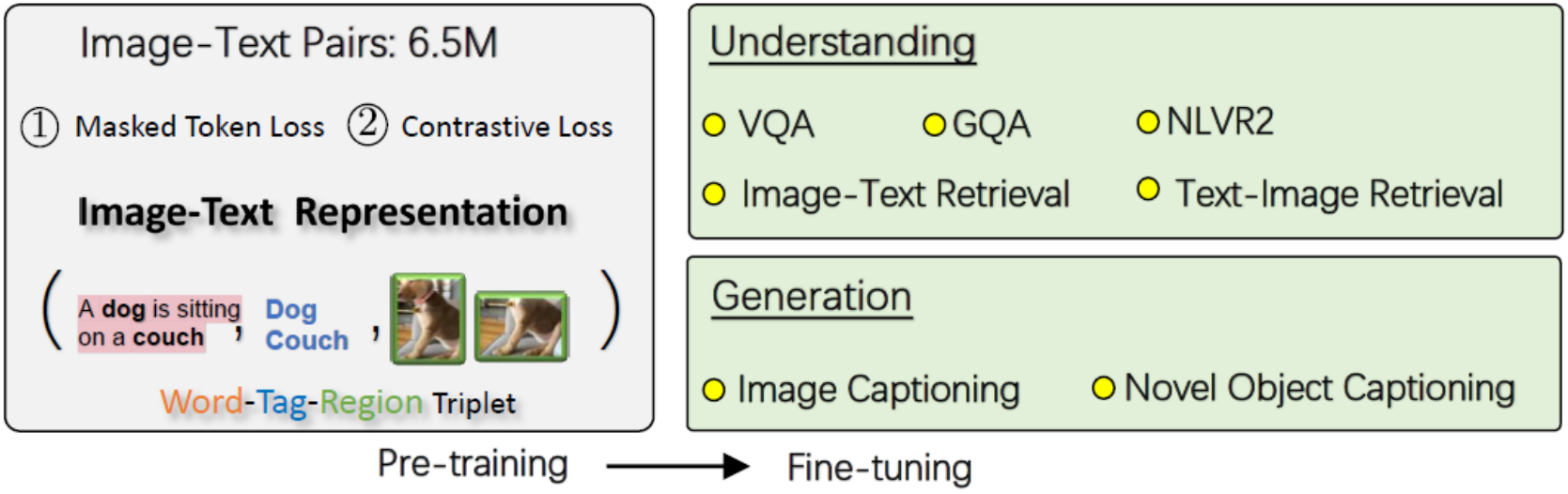
Image Captioning (cont'd)

- **Cap**tion **T**ransforme**r** (CPTR) -
  CPTR: Full Transformer Network for Image Captioning, CAS, arxiv 2021
- Motivation: patch translation for image captioning



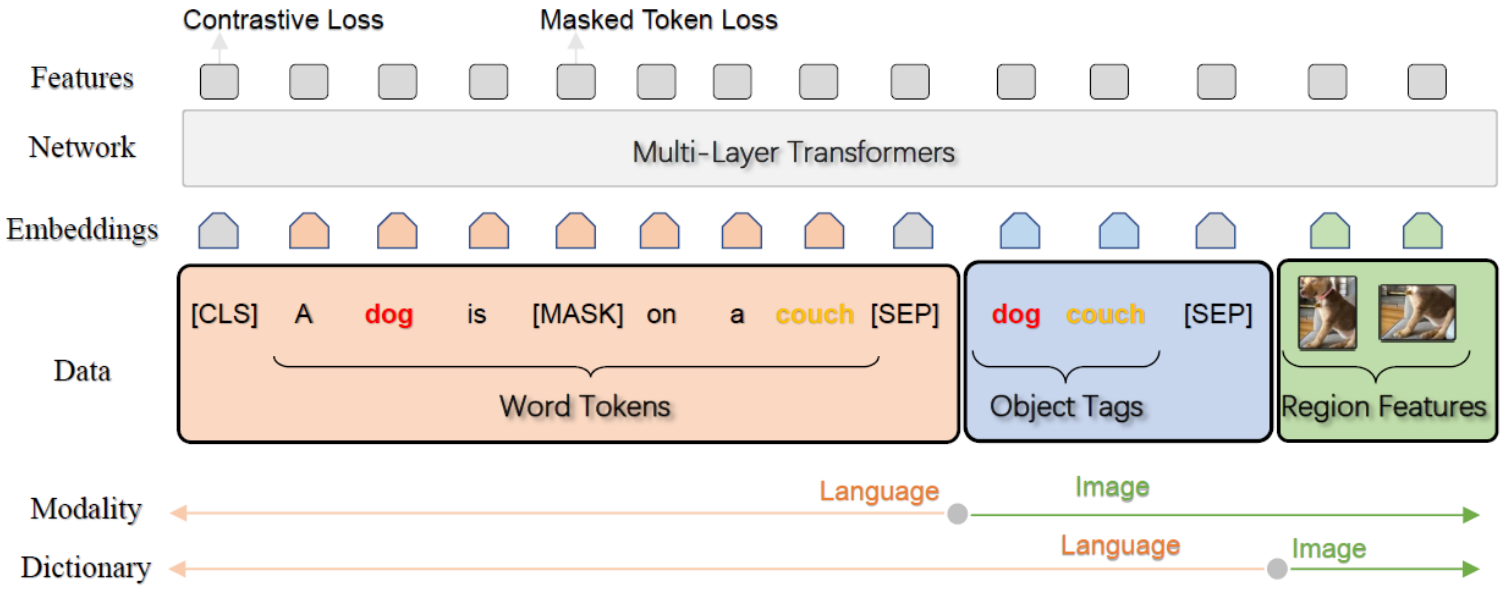
Beyond Image Captioning: Unified Vision & Language Model

- **Oscar**: **O**bject-**S**emanti**c**s **A**ligned P**r**e-training for Vision-Language Tasks, Microsoft, ECCV'20
  - Training data:
    triplets of caption-tag-region
  - Objectives:
    1. Masked token loss for words & tags
    2. Contrastive loss tags and others
  - Fine-tuning:
    5 vision & language tasks (VQA, image-text retrieval, image captioning, NOC, etc.)



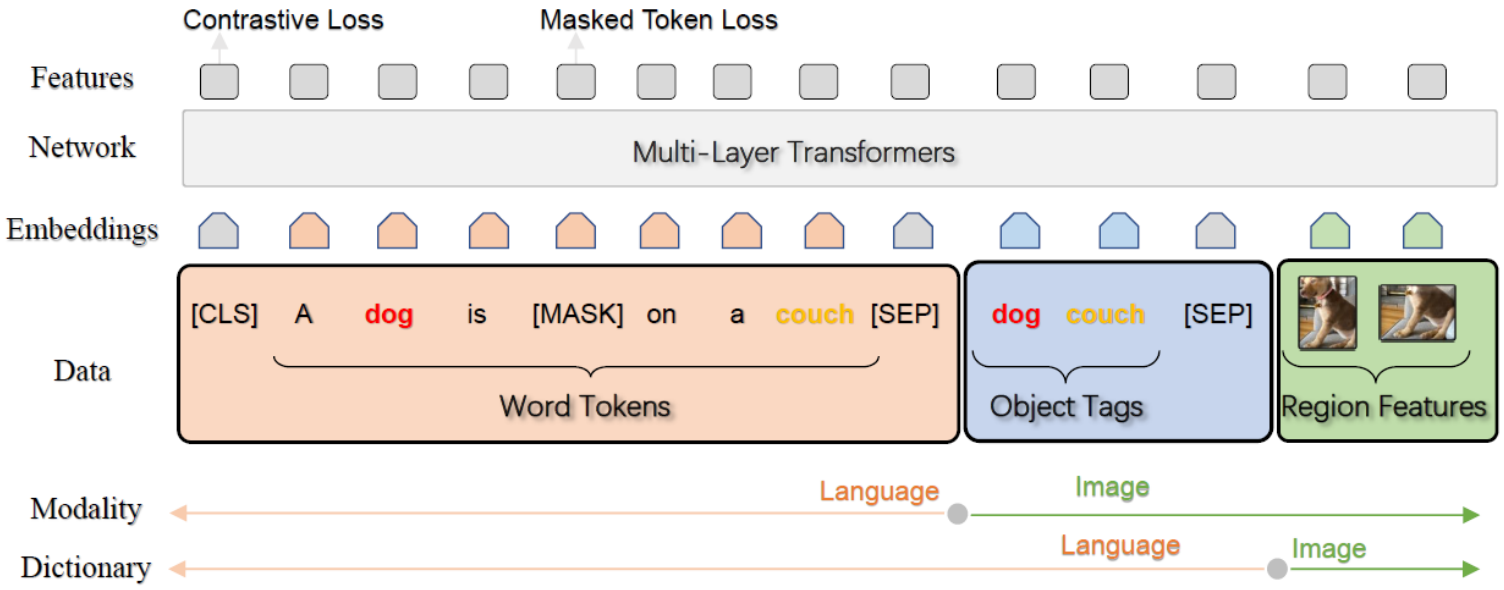
Semantics-Aligned Pre-training for V+L Tasks

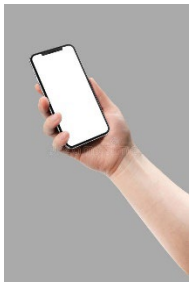
- **Oscar**: **O**bject-**S**emanti**c**s **A**ligned P**r**e-training for Vision-Language Tasks
  - Training:
    - Inputs: triplets of caption-tag-region
    - Objectives: Masked token loss for words & tags + Contrastive loss tags and others
  - Fine-tuning:
    5 vision & language tasks (image captioning, NOC, VQA, image-text retrieval, etc.)



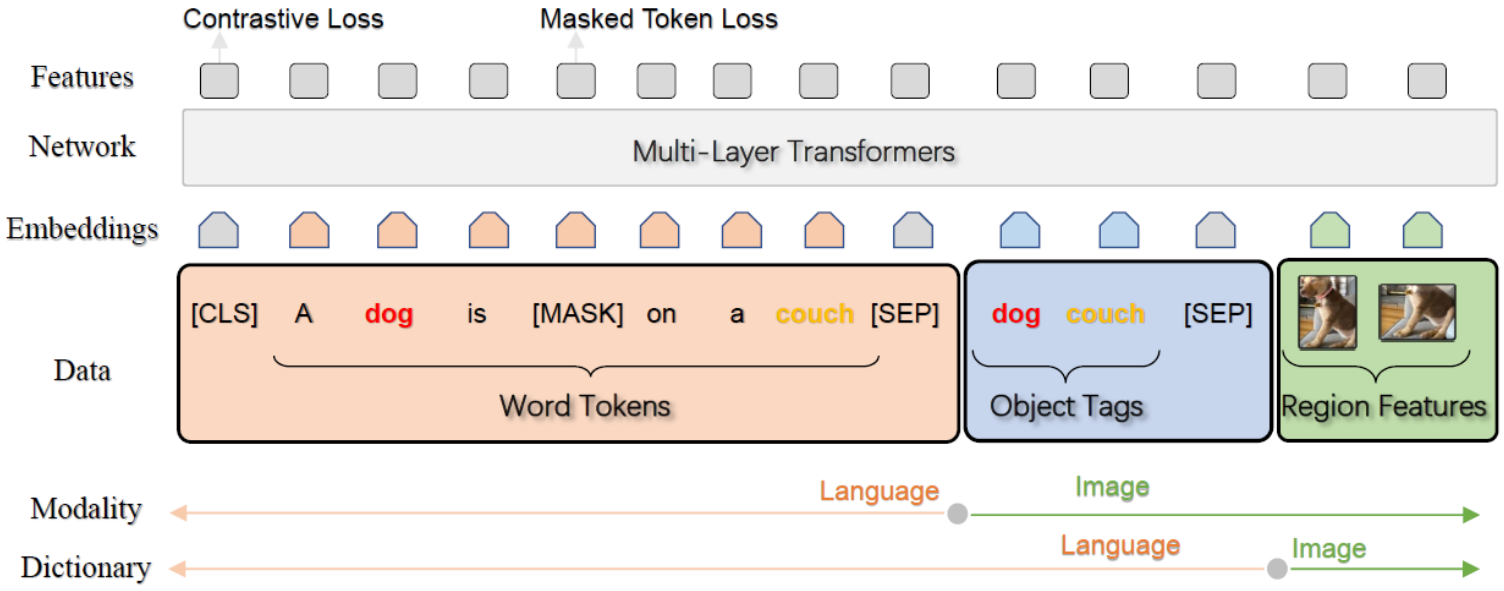
Semantics-Aligned Pre-training for V+L Tasks (cont'd)

- **Oscar**: **O**bject-**S**emanti**c**s **A**ligned P**r**e-training for Vision-Language Tasks (ECCV'20)
  - Training:
    - Inputs: triplets of word-tag-region
    - Objectives: Masked token loss for words & tags + Contrastive loss tags and others
  - Fine-tuning:
    - 5 vision & language tasks (image captioning, NOC, VQA, image-text retrieval, etc.)



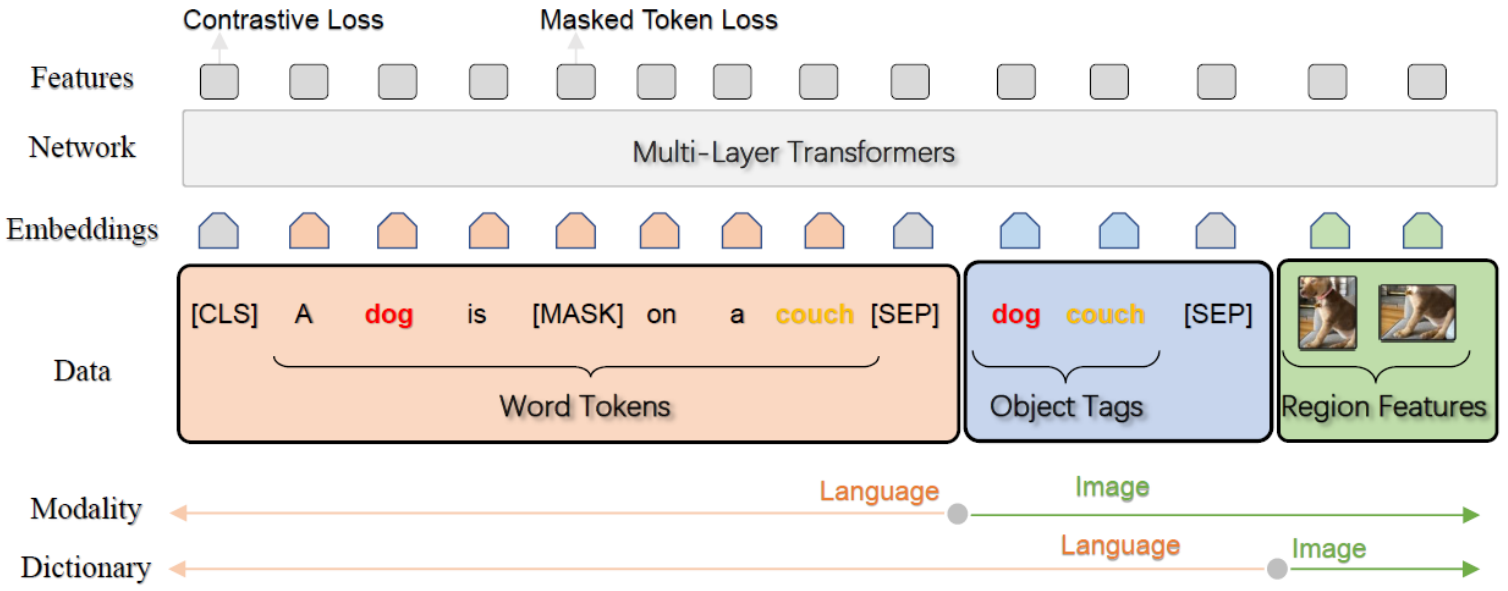
Holding an apple   or 

- **Oscar** (cont'd)
  - Fine-tuning:
    5 vision & language tasks (image captioning, NOC, VQA, image-text retrieval, etc.)
  - Take **image-text retrieval** as an example
    - Training: aligned/mis-aligned image-text pairs as positive/negative input pairs, with **[CLS]** for binary classification (1/0)
    - Inference: for either image or text retrieval, calculate classification score of **[CLS]** for the input query



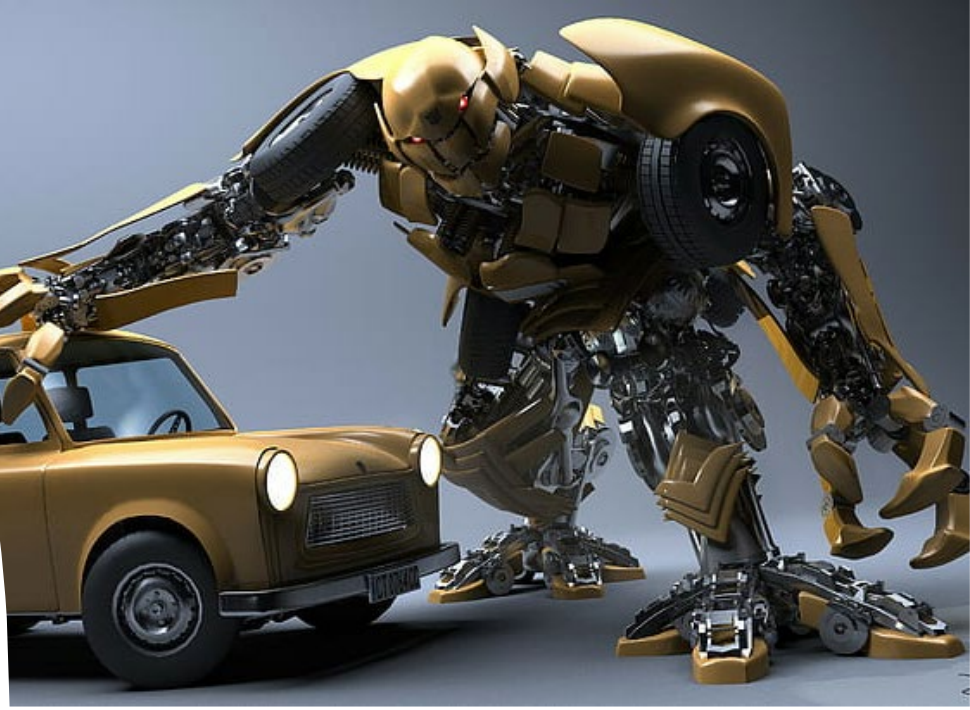
Semantics-Aligned Pre-training for V+L Tasks (cont'd)

- **Oscar**: **O**bject-**S**emanti**c**s **A**ligned P**r**e-training for Vision-Language Tasks (ECCV'20)
  - Fine-tuning:
    5 vision & language tasks (image captioning, NOC, VQA, image-text retrieval, etc.)
  - Take **image captioning** as an example
    - Training: triplets of image regions features + object tags + captions as inputs;
      caption tokens with full attention on image regions/tags but not the other way around
    - Inference: image regions, tags and **[CLS]** as inputs,
      with **[MASK]** tokens sequentially added/predicted



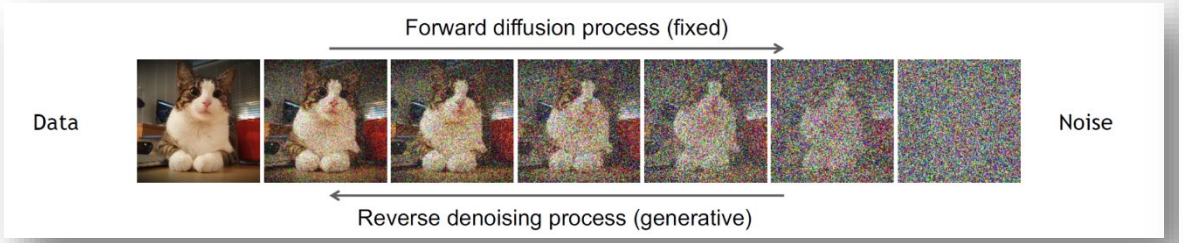
What to Be Covered?

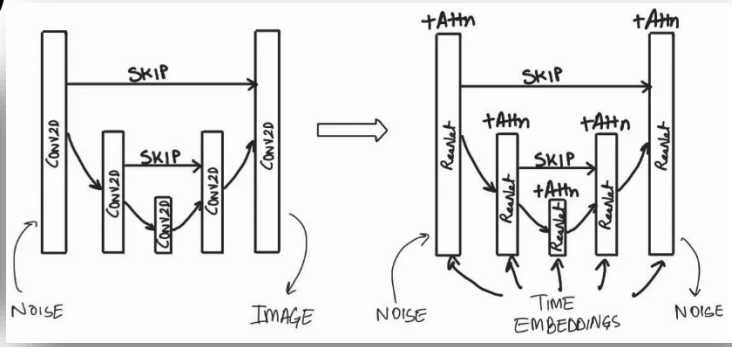
- Transformer
- Transformer for Visual Analysis
  - Vision Transformer (ViT)
  - SSL & Beyond
- Vision-Language Model
  - Image2Text
  - Text2Image
- Pretraining vs. Finetuning
  - Parameter-Efficient Finetuning (PEFT)



Recap: Denoising Diffusion Probabilistic Models (DDPM)

- Training:
  - Forward/reverse diffusion & denoising process
    - learns to generate/restore data by denoising
    - typically implemented via a **conditional U-net**)





- Pseudo Code for Training/Inference (Sampling):

Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \text{Uniform}(\{1, \dots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

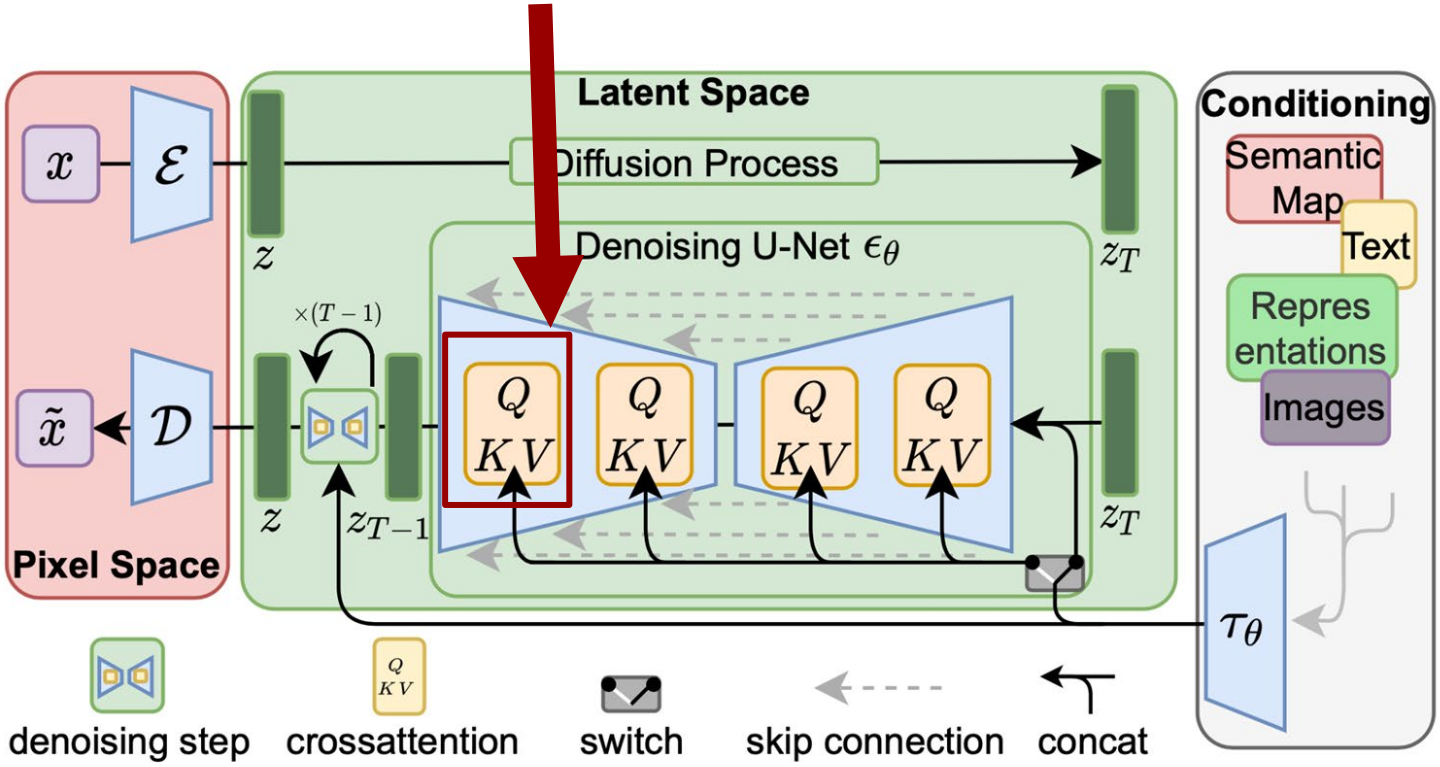
Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \dots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

Ho et al., Denoising Diffusion Probabilistic Models, NeurIPS 2020
Song et al., Score-Based Generative Modeling through Stochastic Differential Equations, ICLR 2021

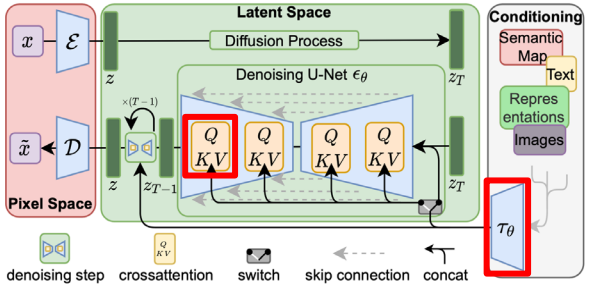
From Unconditional to Conditional Latent Diffusion Model

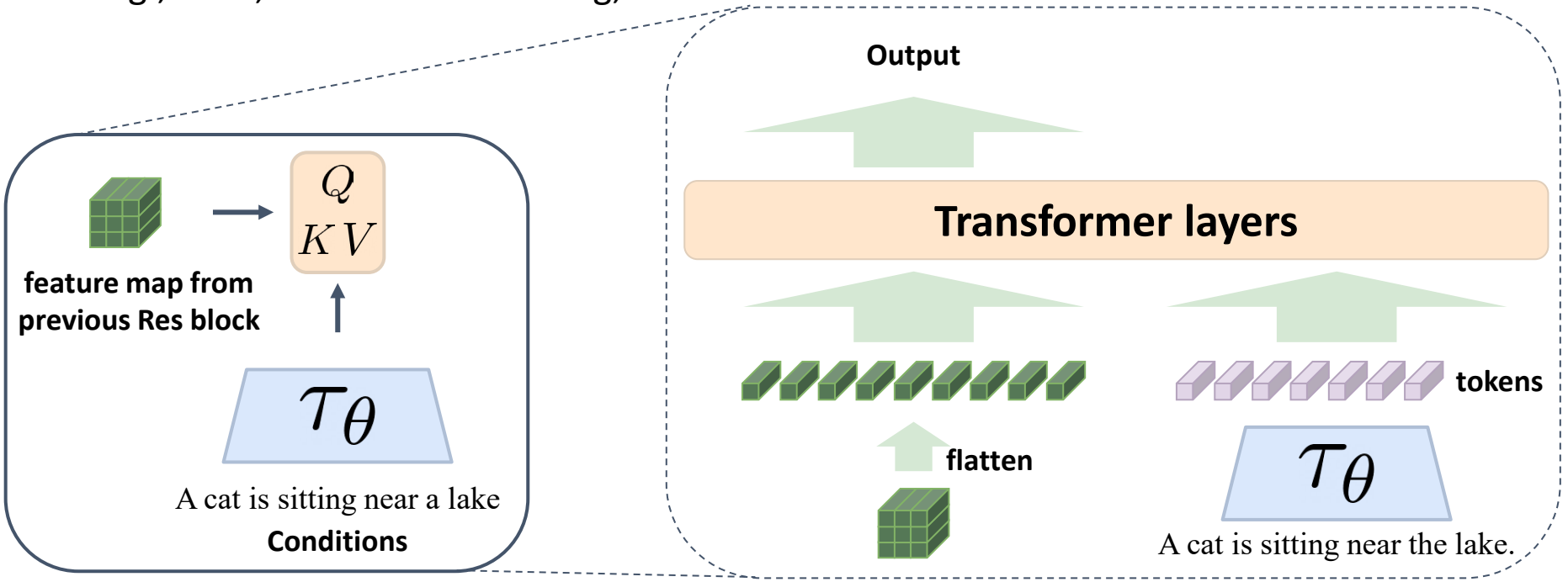
- Latent diffusion model (LDM), CVPR'22: DDPM in latent space
- Condition mechanism: cross-attention at transformer layers



From Unconditional to Conditional Latent Diffusion Model (cont'd)

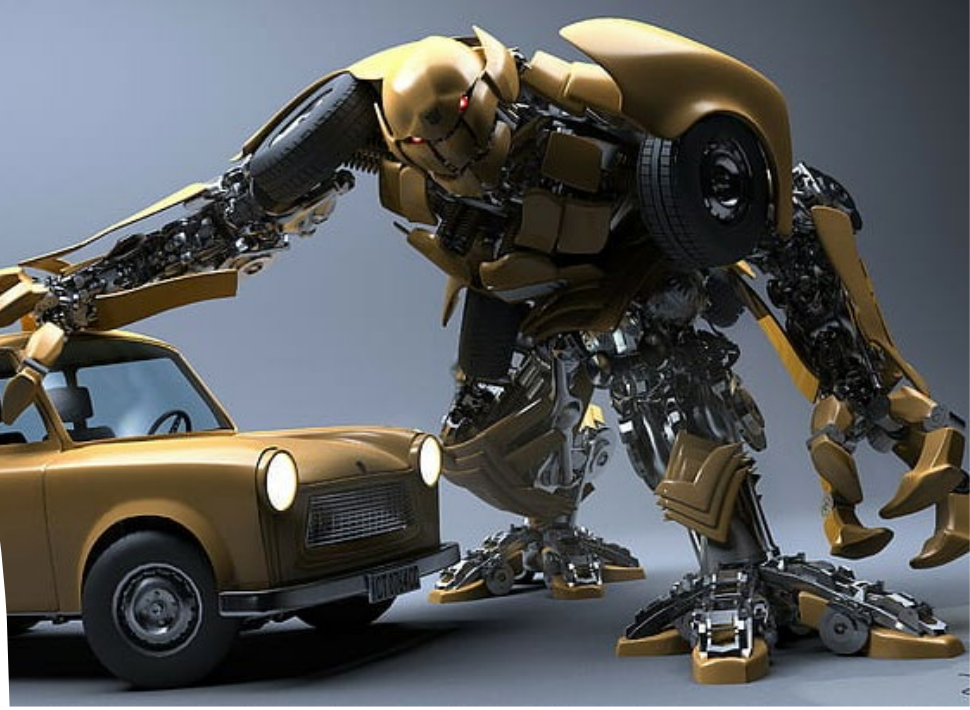
- **Condition mechanism:** using transformer layers
- τ_θ is the embedding module for conditions
  e.g., BERT, CLIP text embedding, etc.





What to Be Covered?

- Transformer
- Transformer for Visual Analysis
  - Vision Transformer (ViT)
  - SSL & Beyond
- Vision-Language Model
  - Image2Text
  - Text2Image
- Pretraining vs. Finetuning
  - Parameter-Efficient Finetuning (PEFT)



Pretrain & Finetune LLM/VLM/MLLM

Stage 1

Pre-training by
self-supervised learning
or supervised learning

Stage 2

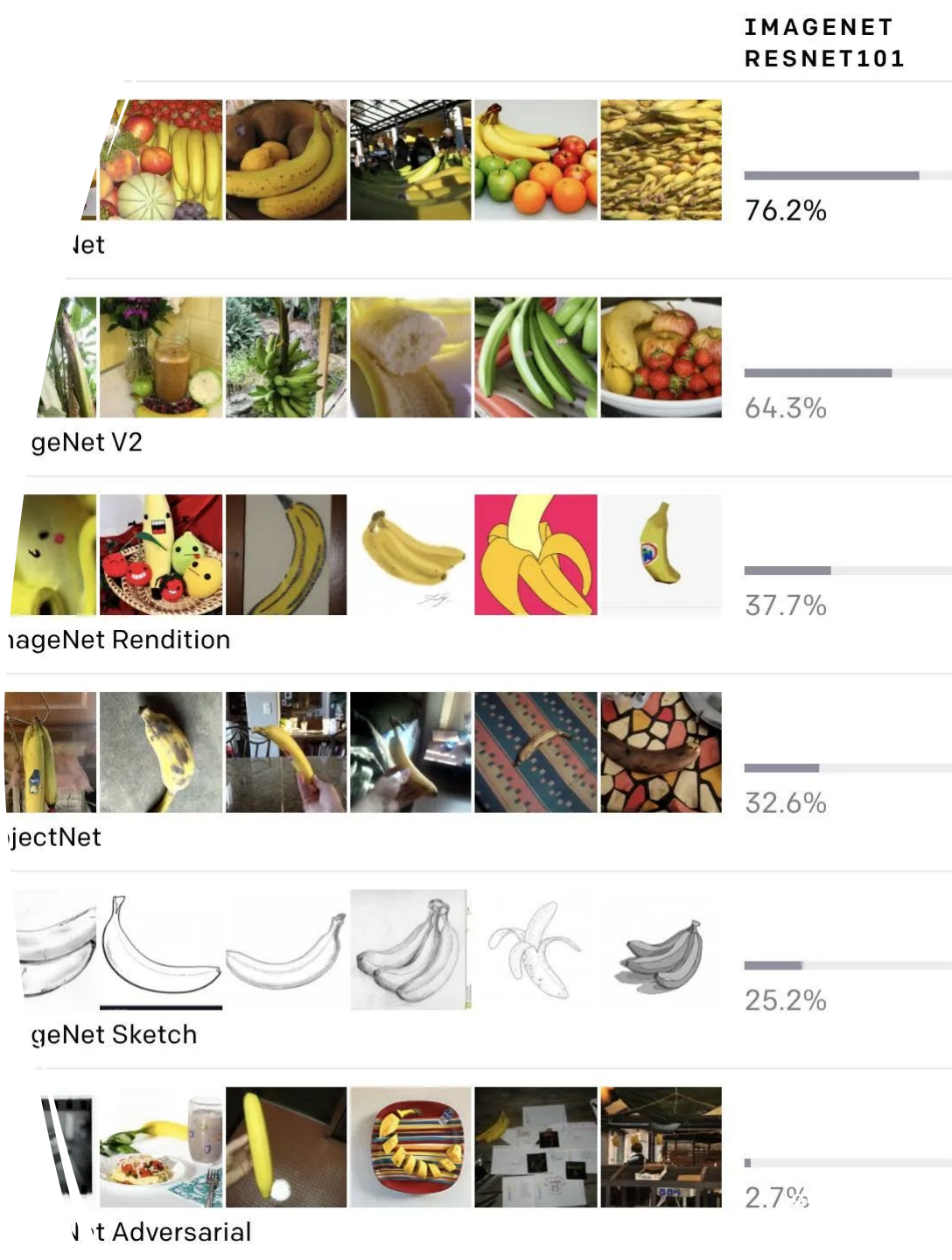
Finetuning
by downstream tasks
in target domains

Stage 3

RLHF - Reinforcement Learning
with Human Feedback
(will not cover details)

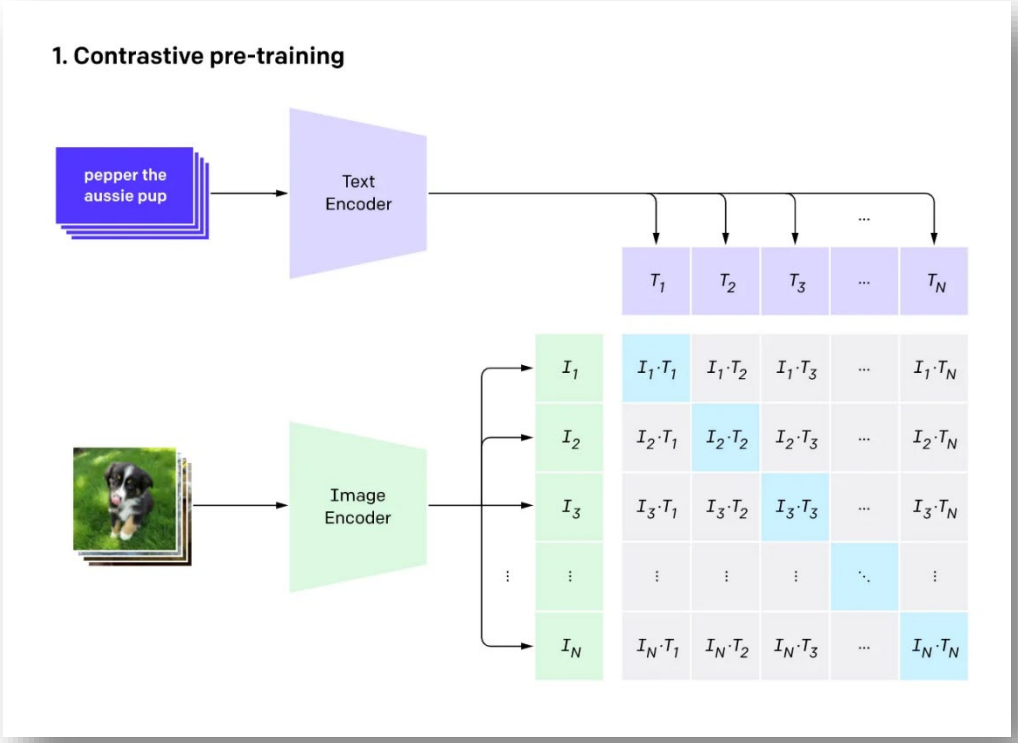
CLIP: Contrastive Language-Image Pretraining

- OpenAI, *Learning Transferable Visual Models From Natural Language Supervision*, NeurIPS WS 2021 (w/ 9000+ citations)
- Why DL/CNN not good enough?
  - Require annotated data for training image classification
  - Domain gap between closed-world and open-world domain data
  - Lack of ability for **zero-shot classification**



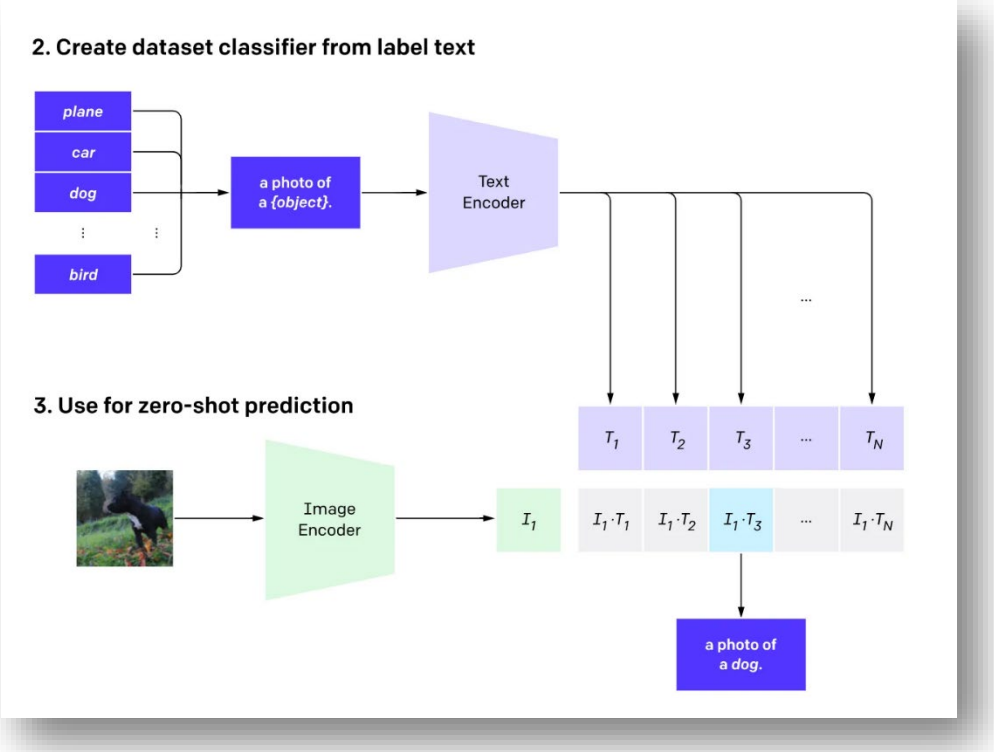
CLIP (cont'd)

- Why DL/CNN not good enough?
  - Require annotated data for training image classification
  - Domain gap between closed-world and open-world domain data
  - Lack of ability for zero-shot classification
- Motivation/Objectives
  - Cross-domain contrastive learning from large-scale image-language data



CLIP (cont'd)

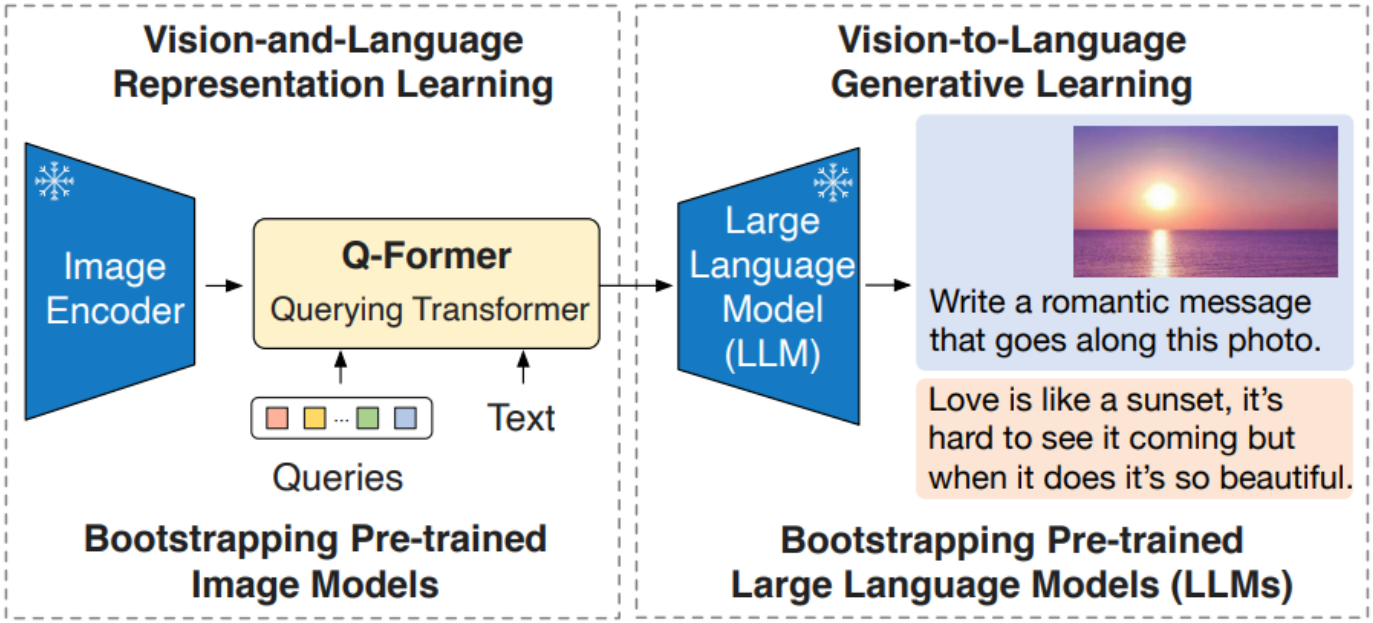
- (Zero-shot) Inference:



- Limitation
  - Fine-grained description ability
  - Any examples?

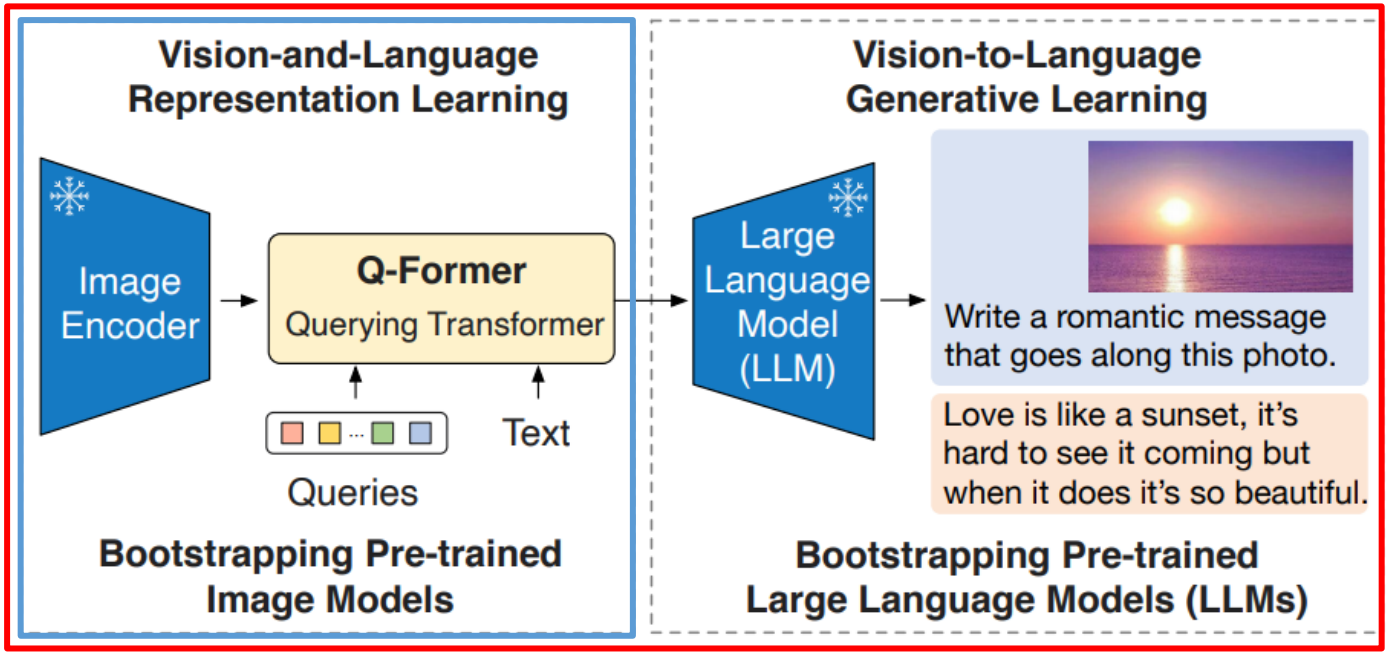
BLIP-2 (ICML'23) (if time permits…)

- **BLIP:**
  Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation, Salesforce Research, NeurIPS 2021
- **Goal:**
  Bridge the modality gap between off-the-shelf **frozen pre-trained image encoders** and **frozen large language models** with a lightweight Querying Transformer (Q-Former).
- **Advantages:**
  1. No need to train from scratch
  2. Avoid catastrophic forgetting (w/ fixed VLM & LLM)



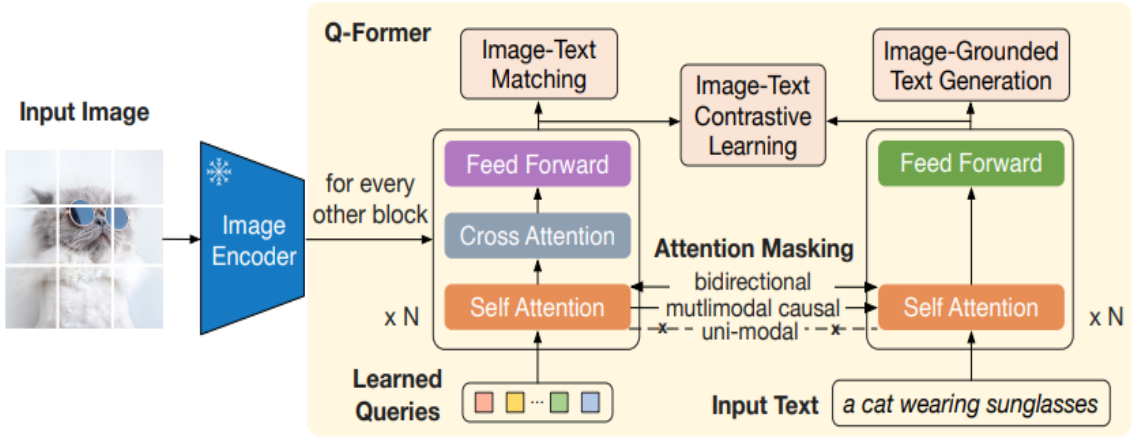
Pre-training of BLIP-2

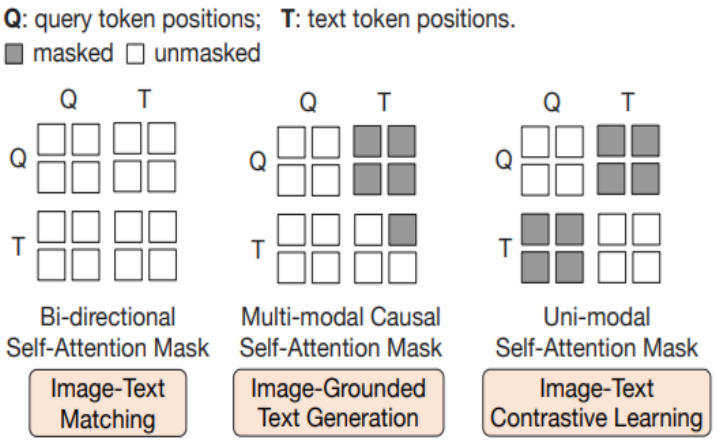
- A **two-stage** pre-training strategy
  - **Stage 1**: Representation Learning
    - enforce **Q-Former** to learn **visual representation** that is most relevant to the text description
  - **Stage 2**: Generative Learning
    - make the output representation of **Q-Former** to be understood by **LLM**



Pre-training Stage 1 - VL Representation Learning

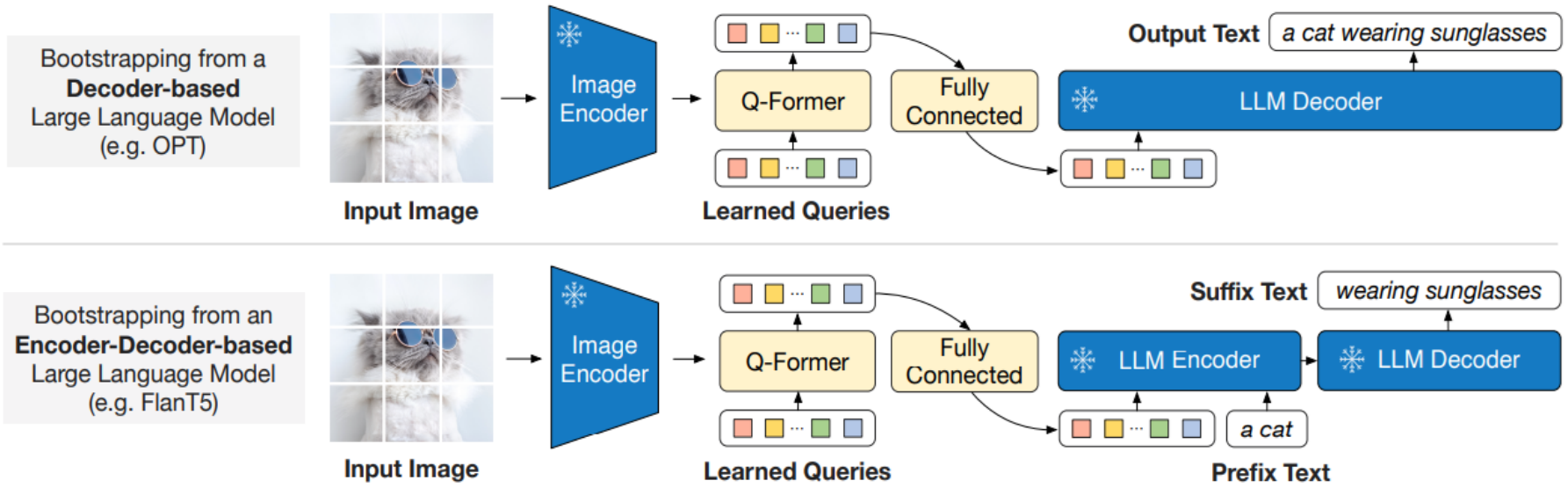
- **Goal:** enforce **Q-Former** to extract visual representation relevant to text
- **Method:** three pre-training tasks
  - **Image-Text Contrastive Learning (ITC)**:
    self-attn in Q/T, followed by max (sim(Q, T)) -> can be viewed as CLIP training
  - **Image-Text Matching (ITM)**:
    for each learnable query -> linear classifier for binary decision
  - **Image-grounded Text Generation (ITG)**:
    self-attn in Q for encoder training; T->Q for image-to-text generation





Pre-training Stage 2 - VL Generative Learning

- **Goal:**
  Learning with LLM guidance
  i.e., make the output representation of **Q-Former** to be understood by **LLMs**.
- **Method:**
  pre-training with Image-grounded Text Generation (ITG)



Quantitative Results

- Comparison on zero-shot visual question answering (VQA)

Models	#Trainable Params	#Total Params	VQAv2 val	VQAv2 test-dev	OK-VQA test	GQA test-dev
VL-T5$_{\text{no-vqa}}$	224M	269M	13.5	-	5.8	6.3
FewVLM (Jin et al., 2022)	740M	785M	47.7	-	16.5	29.3
Frozen (Tsimpoukelli et al., 2021)	40M	7.1B	29.6	-	5.9	-
VLKD (Dai et al., 2022)	406M	832M	42.6	44.5	13.3	-
Flamingo3B (Alayrac et al., 2022)	1.4B	3.2B	-	49.2	41.2	-
Flamingo9B (Alayrac et al., 2022)	1.8B	9.3B	-	51.8	44.7	-
Flamingo80B (Alayrac et al., 2022)	10.2B	80B	-	56.3	**50.6**	-
BLIP-2 ViT-L OPT$_{2.7B}$	104M	3.1B	50.1	49.7	30.2	33.9
BLIP-2 ViT-g OPT$_{2.7B}$	107M	3.8B	53.5	52.3	31.7	34.6
BLIP-2 ViT-g OPT$_{6.7B}$	108M	7.8B	54.3	52.6	36.4	36.4
BLIP-2 ViT-L FlanT5$_{XL}$	103M	3.4B	62.6	62.3	39.4	44.4
BLIP-2 ViT-g FlanT5$_{XL}$	107M	4.1B	63.1	63.0	40.7	44.2
BLIP-2 ViT-g FlanT5$_{XXL}$	108M	12.1B	**65.2**	**65.0**	45.9	**44.7**

Quantitative Results (cont'd)

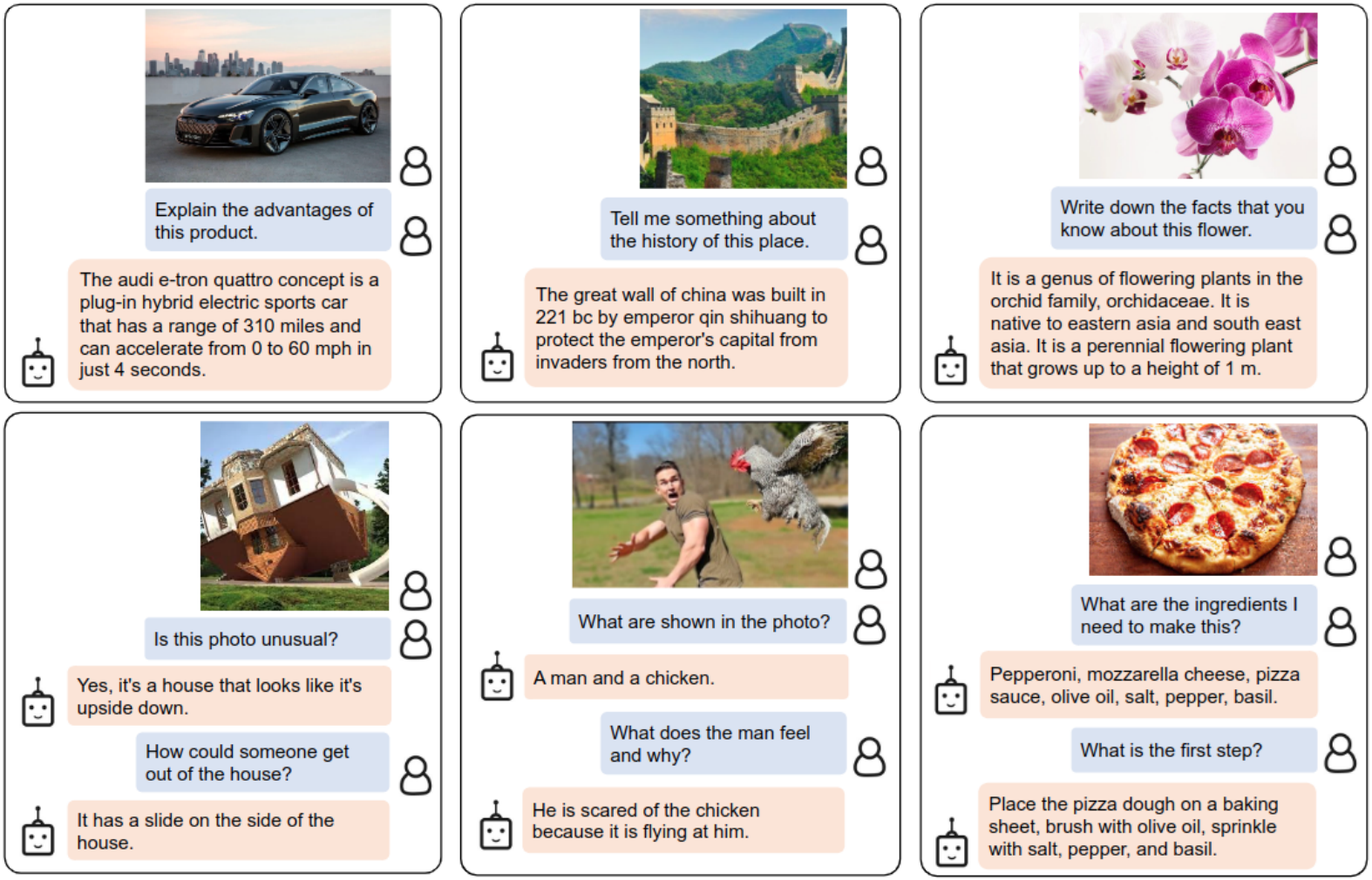
- Comparison on image captioning

Models	#Trainable Params	NoCaps Zero-shot (validation set)								COCO Fine-tuned Karpathy test	
		in-domain		near-domain		out-domain		overall			
		C	S	C	S	C	S	C	S	B@4	C
OSCAR (Li et al., 2020)	345M	-	-	-	-	-	-	80.9	11.3	37.4	127.8
VinVL (Zhang et al., 2021)	345M	103.1	14.2	96.1	13.8	88.3	12.1	95.5	13.5	38.2	129.3
BLIP (Li et al., 2022)	446M	114.9	15.2	112.1	14.9	115.3	14.4	113.2	14.8	40.4	136.7
OFA (Wang et al., 2022a)	930M	-	-	-	-	-	-	-	-	**43.9**	145.3
Flamingo (Alayrac et al., 2022)	10.6B	-	-	-	-	-	-	-	-	-	138.1
SimVLM (Wang et al., 2021b)	~1.4B	113.7	-	110.9	-	115.2	-	112.2	-	40.6	143.3
BLIP-2 ViT-g $OPT_{2.7B}$	1.1B	123.0	15.8	117.8	15.4	123.4	**15.1**	119.7	15.4	43.7	**145.8**
BLIP-2 ViT-g $OPT_{6.7B}$	1.1B	**123.7**	15.8	119.2	15.3	124.4	14.8	121.0	15.3	43.5	145.2
BLIP-2 ViT-g $FlanT5_{XL}$	1.1B	**123.7**	**16.3**	**120.2**	**15.9**	**124.8**	**15.1**	**121.6**	**15.8**	42.4	144.5

C: CIDEr S: SPICE B@4:BLEU@4

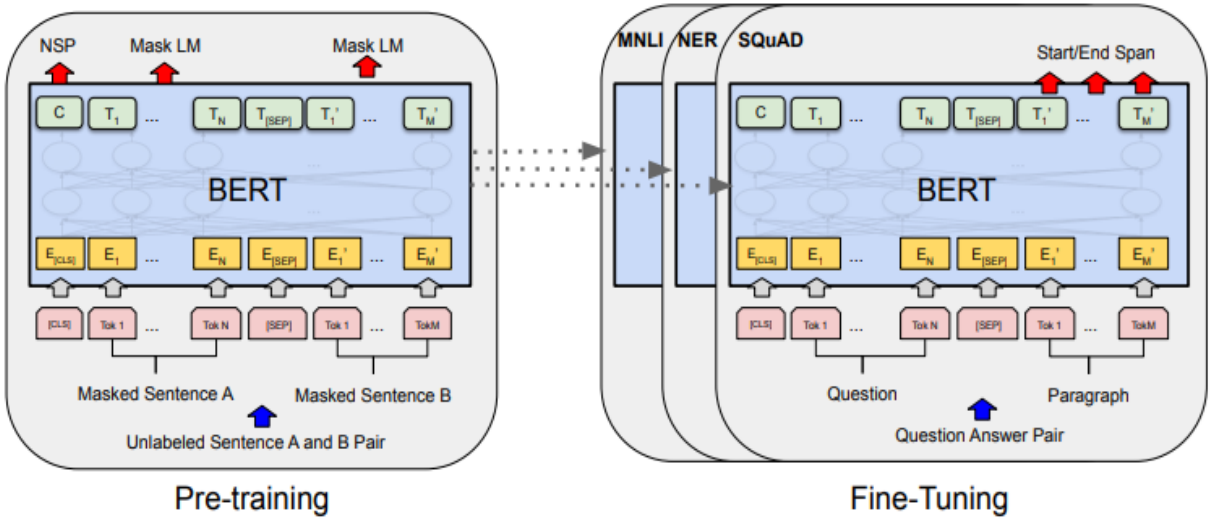
Visualization

- Instructed zero-shot image-to-text generation examples



Take BERT as an example

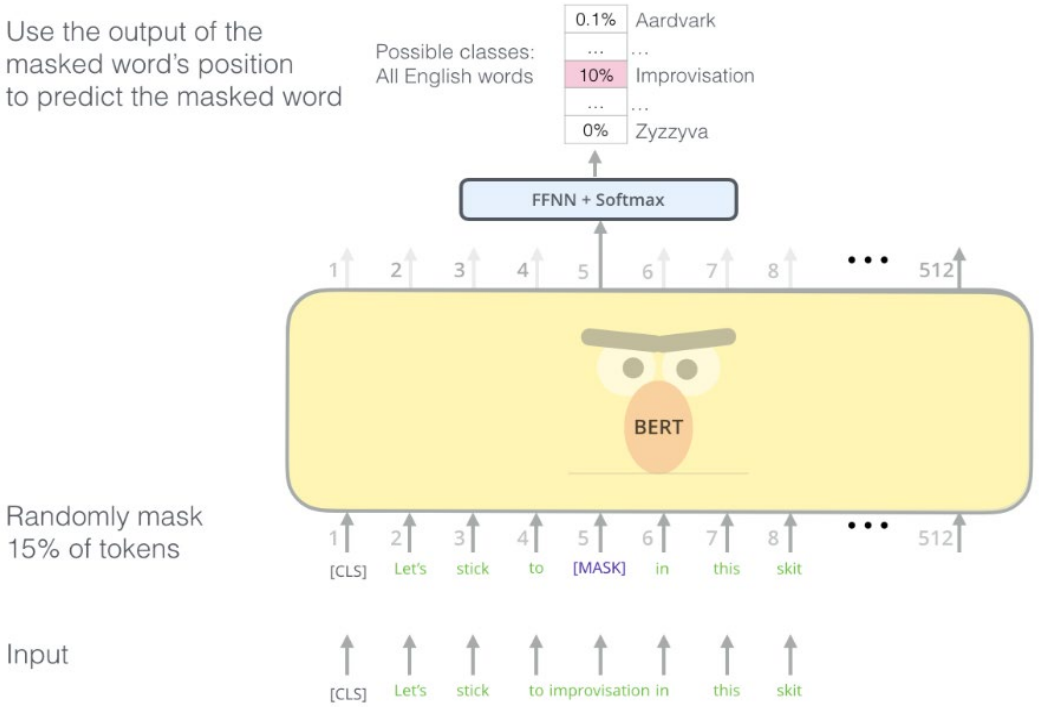
- BERT, short for Bidirectional Encoder Representations from Transformers



BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, Google, arxiv 2018

Take BERT as an example (cont'd)

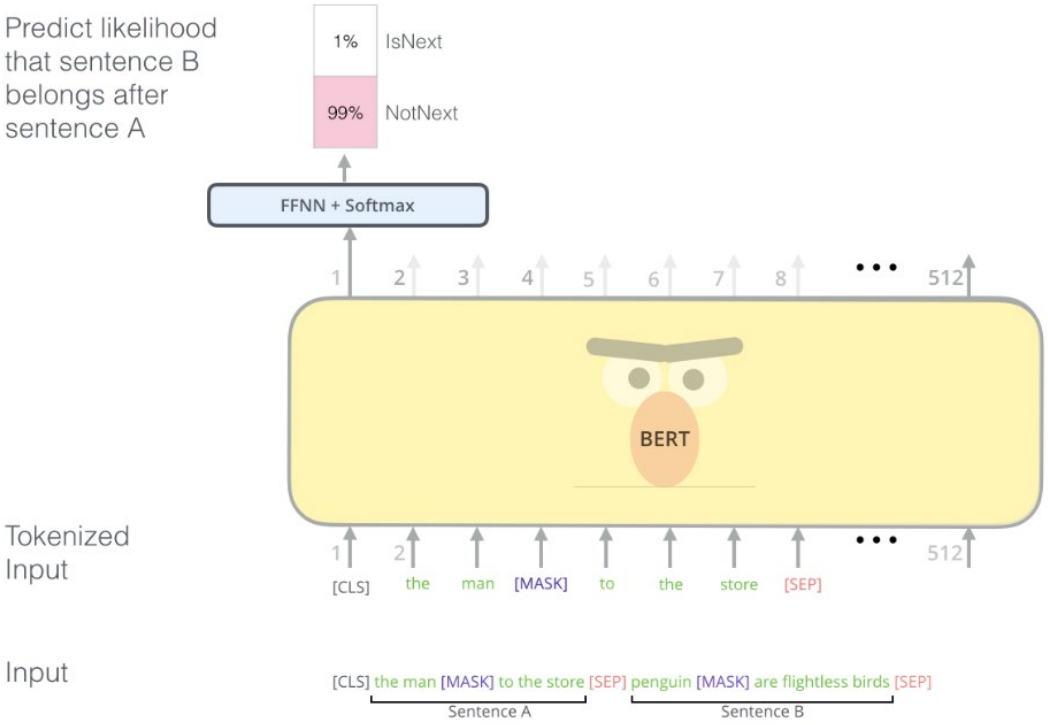
- Pre-training strategy #1:
  Masked Token Prediction (or Masked Language Modeling)
  - Mask out 15% of the input text and predict the masked outputs



BERT's clever language modeling task masks 15% of words in the input and asks the model to predict the missing word.

Take BERT as an example (cont'd)

- Pre-training strategy #2: Next Sentence Prediction
  - Given two sentences A and B, enforce model to learn their relationship
  - Beneficial to QA-type downstream tasks



The second task BERT is pre-trained on is a two-sentence classification task. The tokenization is oversimplified in this graphic as BERT actually uses WordPieces as tokens rather than words --- so some words are broken down into smaller chunks.

Pretrain & Finetune LLM/VLM/MLLM

Stage 1

Pre-training by
self-supervised learning
or supervised learning

Stage 2

Finetuning
by downstream tasks
in target domains

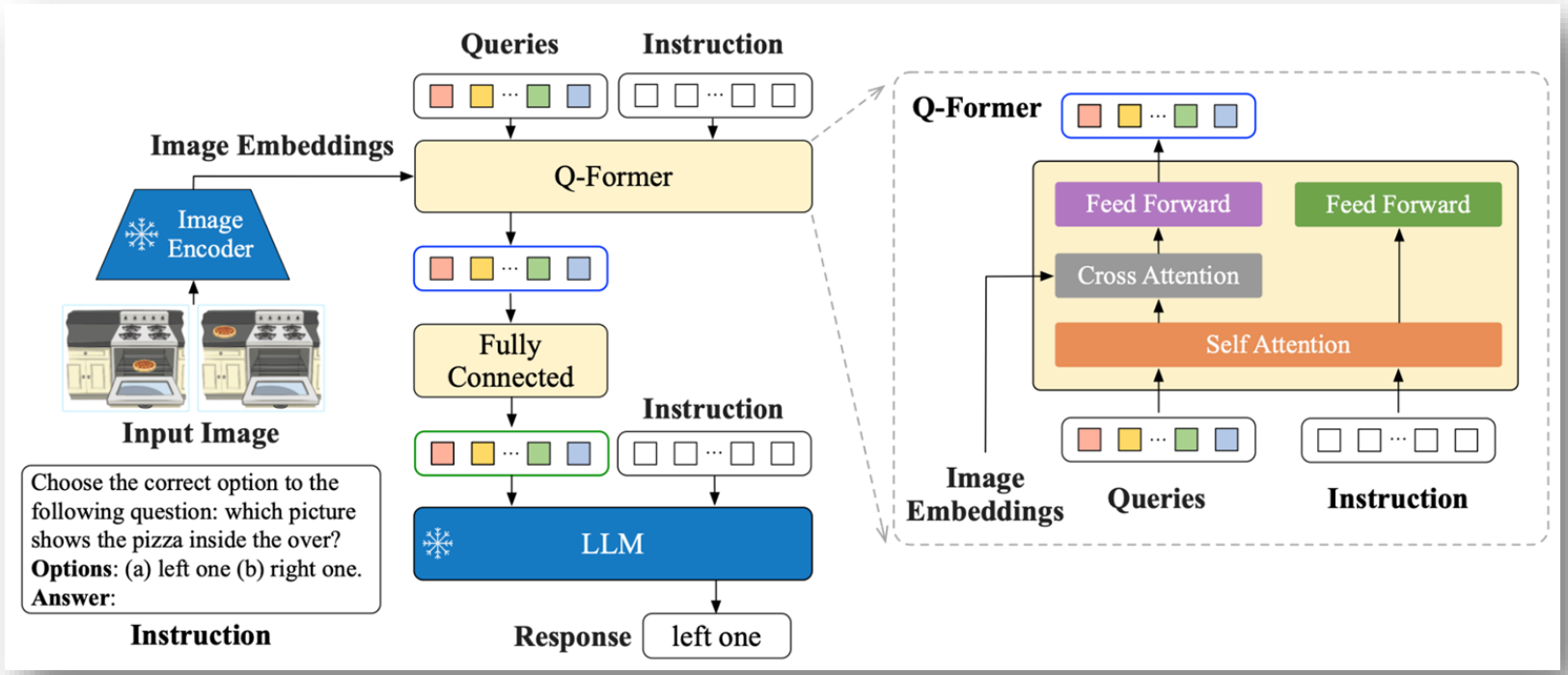
Stage 3

RLHF - Reinforcement Learning
with Human Feedback
(will not cover details)

InstructBLIP (NeurIPS'23)

- Enable **BLIP2** to understand instruction following tasks
- Collect **existing** vision-language datasets (**held-in** / **held-out**)

for training **for zero-shot**



Examples of instruction tuning templates

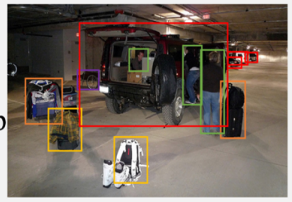
Task	Instruction Template
Image Captioning	<Image>A short image caption: <Image>A short image description: <Image>A photo of <Image>An image that shows <Image>Write a short description for the image. <Image>Write a description for the photo. <Image>Provide a description of what is presented in the photo. <Image>Briefly describe the content of the image. <Image>Can you briefly explain what you see in the image? <Image>Could you use a few words to describe what you perceive in the photo? <Image>Please provide a short depiction of the picture. <Image>Using language, provide a short account of the image. <Image>Use a few words to illustrate what is happening in the picture.
VQA	<Image>{Question} <Image>Question: {Question} <Image>{Question} A short answer to the question is <Image>Q: {Question} A: <Image>Question: {Question} Short answer: <Image>Given the image, answer the following question with no more than three words. {Question} <Image>Based on the image, respond to this question with a short answer: {Question}. Answer: <Image>Use the provided image to answer the question: {Question} Provide your answer as short as possible: <Image>What is the answer to the following question? "{Question}" <Image>The question "{Question}" can be answered using the image. A short answer is
VQG	<Image>Given the image, generate a question whose answer is: {Answer}. Question: <Image>Based on the image, provide a question with the answer: {Answer}. Question: <Image>Given the visual representation, create a question for which the answer is "{Answer}". <Image>From the image provided, craft a question that leads to the reply: {Answer}. Question: <Image>Considering the picture, come up with a question where the answer is: {Answer}. <Image>Taking the image into account, generate an question that has the answer: {Answer}. Question:

Visual Instruction Tuning (NeurIPS'23)

- **LLaVA**: **L**arge **L**anguage **a**nd **V**ision **A**ssistant
- Data source: **generated** by GPT-4

Context type 1: Captions

A group of people standing outside of a black vehicle with various luggage.
Luggage surrounds a vehicle in an underground parking area
People try to fit all of their luggage in an SUV.
The sport utility vehicle is parked in the public garage, being packed for a trip
Some people with luggage near a van that is transporting it.



Context type 2: Boxes

person: [0.681, 0.242, 0.774, 0.694], person: [0.63, 0.222, 0.686, 0.516], person: [0.444, 0.233, 0.487, 0.34], backpack: [0.384, 0.696, 0.485, 0.914], backpack: [0.755, 0.413, 0.846, 0.692], suitcase: [0.758, 0.413, 0.845, 0.69], suitcase: [0.1, 0.497, 0.173, 0.579], bicycle: [0.282, 0.363, 0.327, 0.442], car: [0.786, 0.25, 0.848, 0.322], car: [0.783, 0.27, 0.827, 0.335], car: [0.86, 0.254, 0.891, 0.3], car: [0.261, 0.101, 0.787, 0.626]

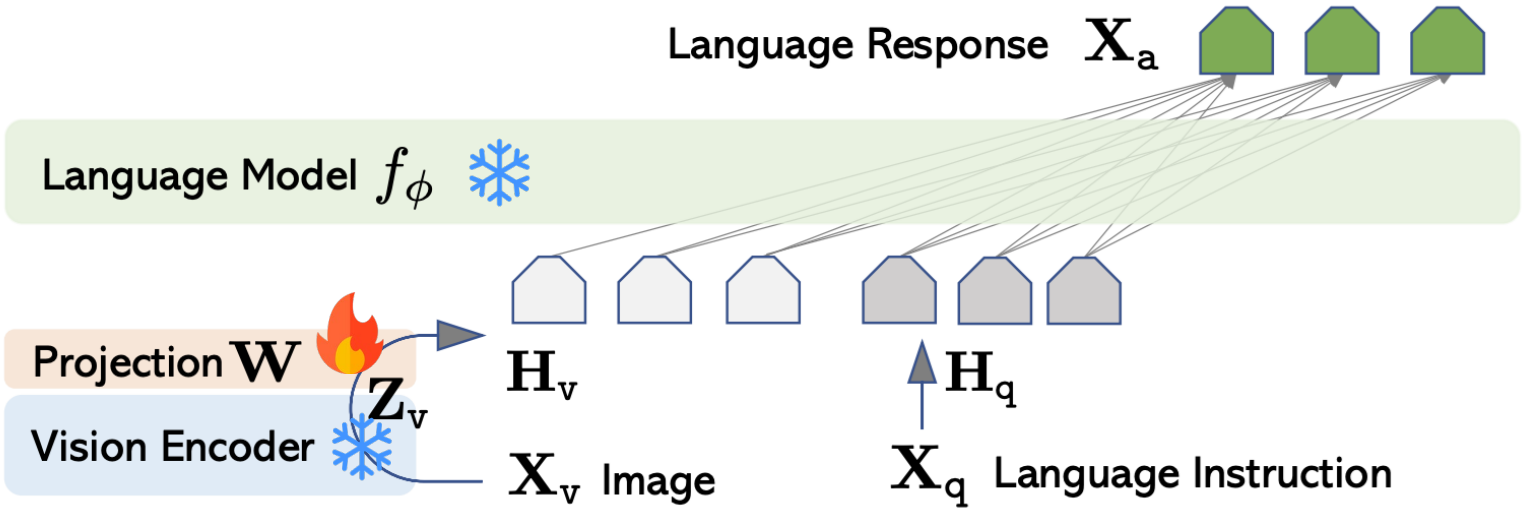
GPT-4

type 1: conversation **type 2: detailed desciption** **type 3: complex reasoning**

LLaVA

Stage I: Re-Training for Feature Alignment

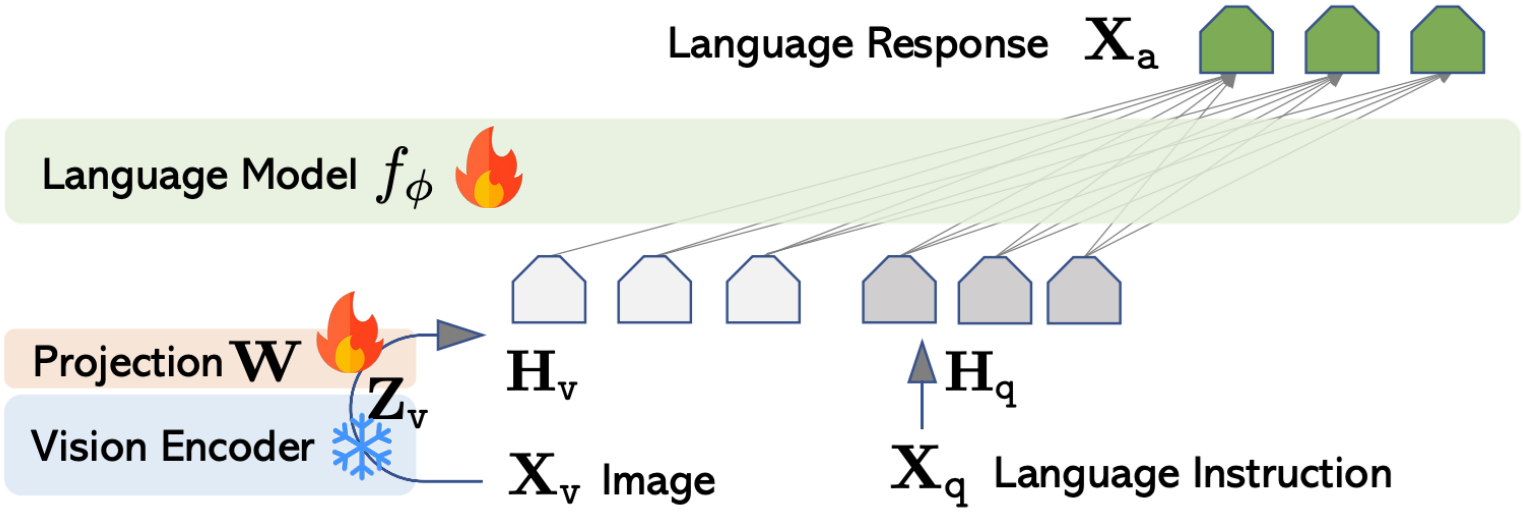
- Finetune projection layer by **image-text pairs** with **instruction tuning**



LLaVA

Stage II: End-to-End Finetuning

- Instruction tuning LLM and
  projection layers with **vision-language complex reasoning data** from GPT-4



ScienceQA Results

- LLaVA outperforms GPT-4 on ScienceQA dataset

Method	Subject			Context Modality			Grade		Average
	NAT	SOC	LAN	TXT	IMG	NO	G1-6	G7-12	
Representative & SoTA methods with numbers reported in the literature									
Human [30]	90.23	84.97	87.48	89.60	87.50	88.10	91.59	82.42	88.40
GPT-3.5 [30]	74.64	69.74	76.00	74.44	67.28	77.42	76.80	68.89	73.97
GPT-3.5 w/ CoT [30]	75.44	70.87	78.09	74.68	67.43	79.93	78.23	69.68	75.17
LLaMA-Adapter [55]	84.37	88.30	84.36	83.72	80.32	86.90	85.83	84.05	85.19
MM-CoT$_{Base}$ [57]	87.52	77.17	85.82	87.88	82.90	86.83	84.65	85.37	84.91
MM-CoT$_{Large}$ [57]	95.91	82.00	90.82	95.26	88.80	92.89	92.44	90.31	91.68
Results with our own experiment runs									
GPT-4	84.06	73.45	87.36	81.87	70.75	90.73	84.69	79.10	82.69
LLaVA	90.36	95.95	88.00	89.49	88.00	90.66	90.93	90.90	90.92

NAT: Natural Science

SOC: Social Science

LAN: Language Science

TXT: text context

IMG: image context

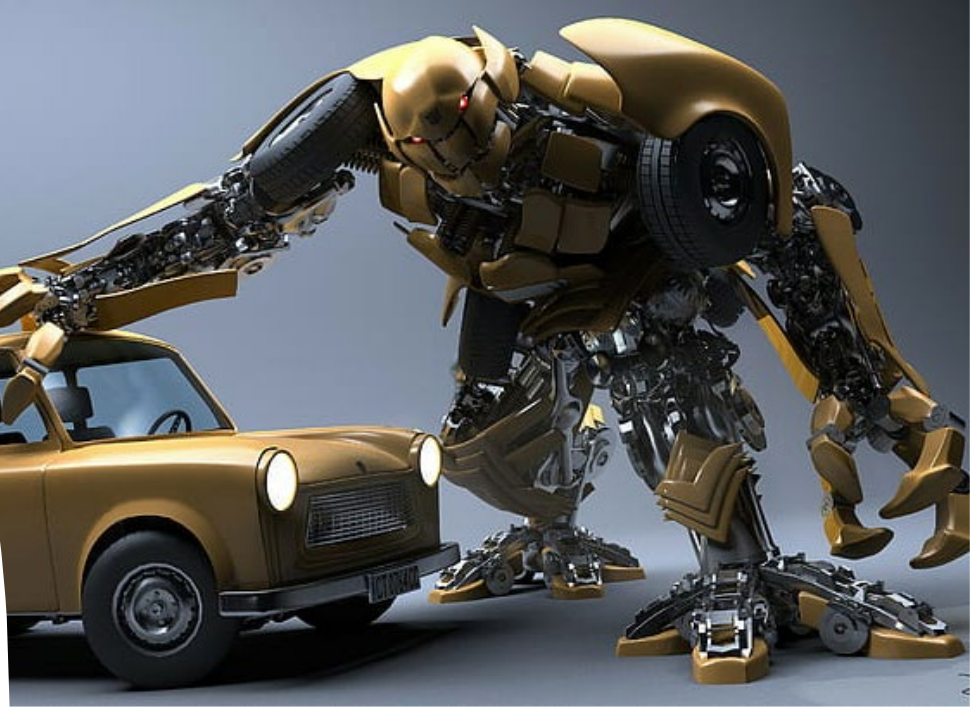
NO: no context

G1-6: grades 1-6

G7-12: grads 7-12

What to Be Covered?

- Transformer
- Transformer for Visual Analysis
  - Vision Transformer (ViT)
  - SSL & Beyond
- Vision-Language Model
  - Image2Text
  - Text2Image
- Pretraining vs. Finetuning
  - Parameter-Efficient Finetuning (PEFT)



Pretrain & Finetune LLM/VLM/MLLM

Stage 1

Pre-training by
self-supervised learning
or supervised learning

Stage 2

Finetuning
by downstream tasks
in target domains

Stage 3

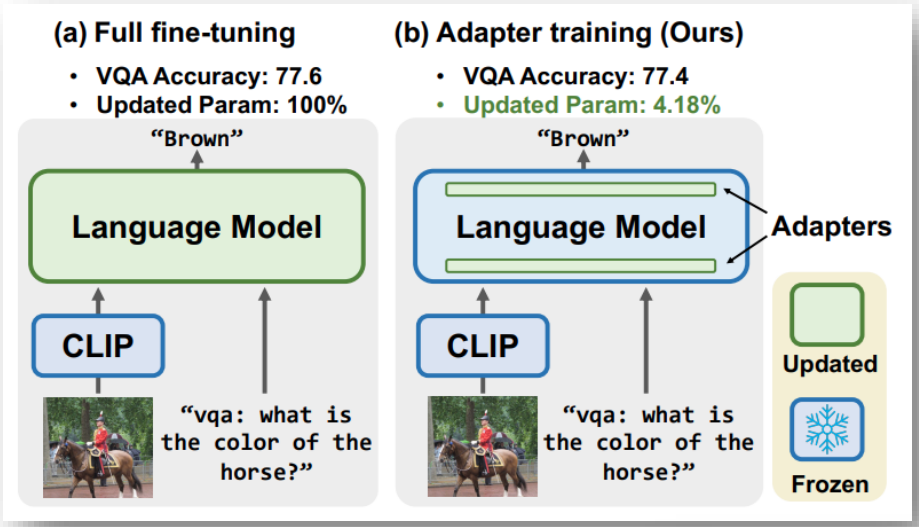
RLHF - Reinforcement Learning
with Human Feedback
(will not cover details)

Can we do FT
in a more efficient way?

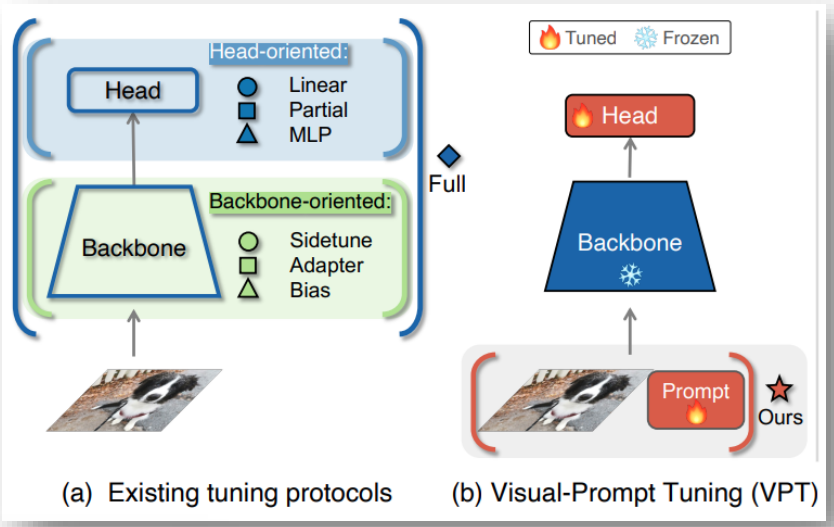
Parameter-Efficient Fine-Tuning (PEFT)

- **Prompt Tuning** (訓練一個提詞器)
  - Visual Prompt Tuning (ECCV, 2022)
- **Adapter** (訓練一個小抄帶在身上)
  - VL-ADAPTER: Parameter-Efficient Transfer Learning for Vision-and-Language Tasks (CVPR, 2022)
- **LoRA** (訓練一個小抄帶在身上)
  - LoRA: **Lo**w-**R**ank **A**daptation of Large Language Models (ICLR, 2022)

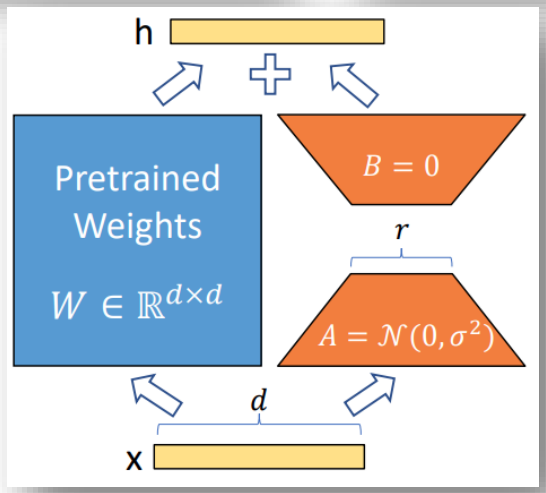
Parameter Efficient Fine Tuning



Adapter



Prompt Tuning

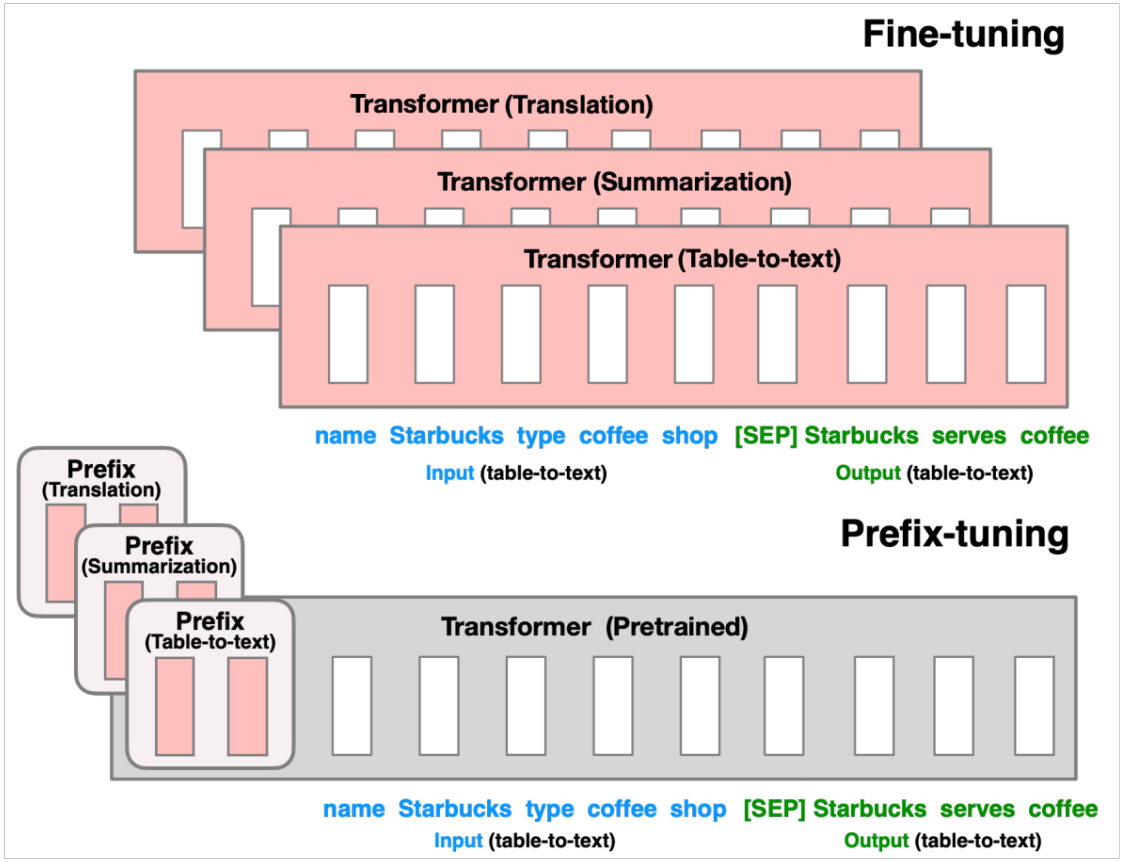


LoRA

Pros & Cons for each?
Will discuss later…

PEFT (1/3): Prefix Tuning (Prompt Tuning)

Train個提詞機



Prefix-Tuning: Optimizing Continuous Prompts for Generation, Stanford, ACL 2021

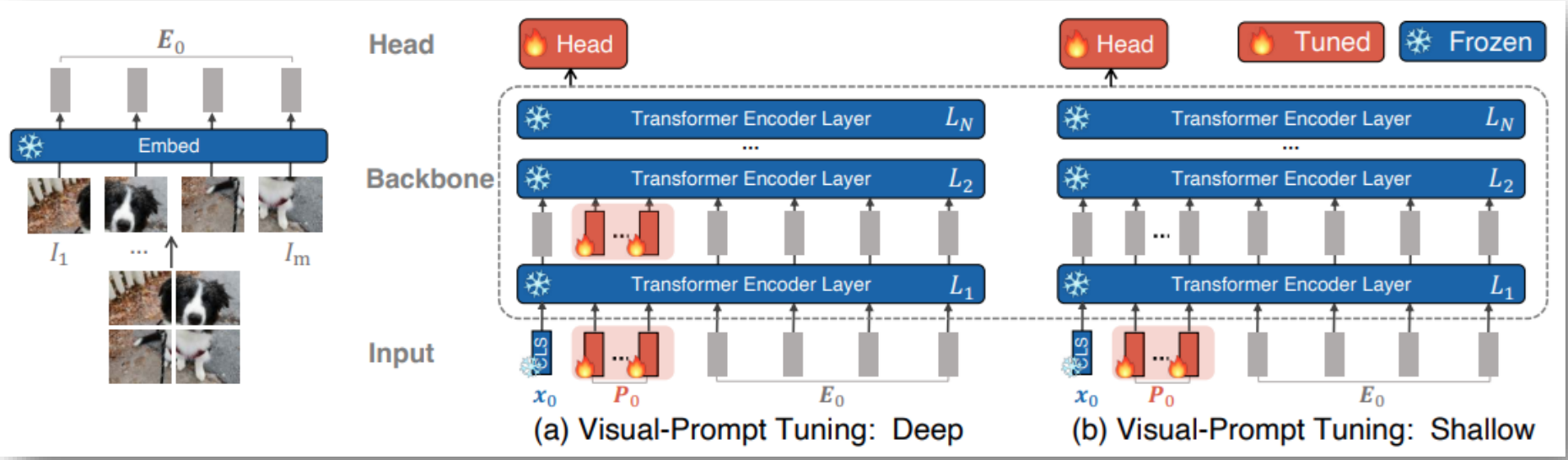
Prompt Tuning

- Shallow:

$$[\mathbf{x}_1, \mathbf{Z}_1, \mathbf{E}_1] = L_1([\mathbf{x}_0, \mathbf{P}, \mathbf{E}_0]) \tag{4}$$
$$[\mathbf{x}_i, \mathbf{Z}_i, \mathbf{E}_i] = L_i([\mathbf{x}_{i-1}, \mathbf{Z}_{i-1}, \mathbf{E}_{i-1}]) \qquad i = 2, 3, \ldots, N \tag{5}$$
$$\mathbf{y} = \texttt{Head}(\mathbf{x}_N) \ , \tag{6}$$

- Deep:

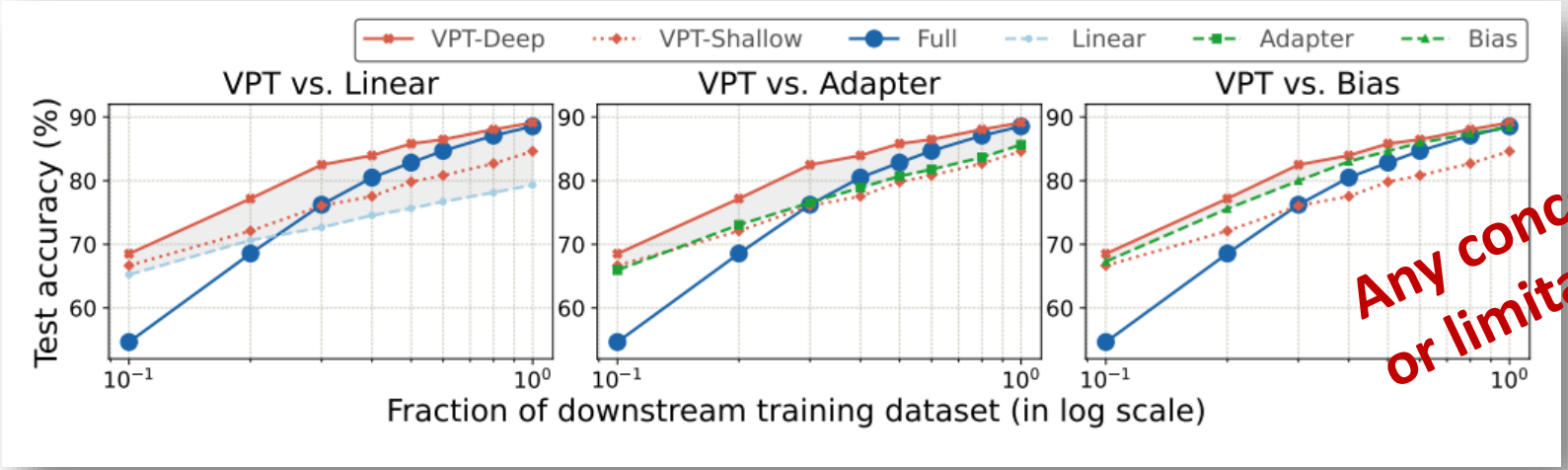
$$[\mathbf{x}_i, \_, \mathbf{E}_i] = L_i([\mathbf{x}_{i-1}, \mathbf{P}_{i-1}, \mathbf{E}_{i-1}]) \qquad i = 1, 2, \ldots, N \tag{7}$$
$$\mathbf{y} = \texttt{Head}(\mathbf{x}_N) \ . \tag{8}$$



Visual Prompt Tuning, Meta AI, ECCV 2022

Visual Prompt Tuning

	ViT-B/16 (85.8M)	Total params	Scope Input	Scope Backbone	Extra params	FGVC	VTAB-1k Natural	VTAB-1k Specialized	VTAB-1k Structured
	Total # of tasks					5	7	4	8
(a)	FULL	24.02×		✓		88.54	75.88	83.36	47.64
(b)	LINEAR	1.02×				79.32 (0)	68.93 (1)	77.16 (1)	26.84 (0)
	PARTIAL-1	3.00×				82.63 (0)	69.44 (2)	78.53 (0)	34.17 (0)
	MLP-3	1.35×			✓	79.80 (0)	67.80 (2)	72.83 (0)	30.62 (0)
(c)	SIDETUNE	3.69×		✓	✓	78.35 (0)	58.21 (0)	68.12 (0)	23.41 (0)
	BIAS	1.05×		✓		88.41 (3)	73.30 (3)	78.25 (0)	44.09 (2)
	ADAPTER	1.23×		✓	✓	85.66 (2)	70.39 (4)	77.11 (0)	33.43 (0)
(ours)	VPT-SHALLOW	1.04×	✓		✓	84.62 (1)	76.81 (4)	79.66 (0)	46.98 (4)
	VPT-DEEP	1.18×				**89.11 (4)**	**78.48 (6)**	**82.43 (2)**	**54.98 (8)**

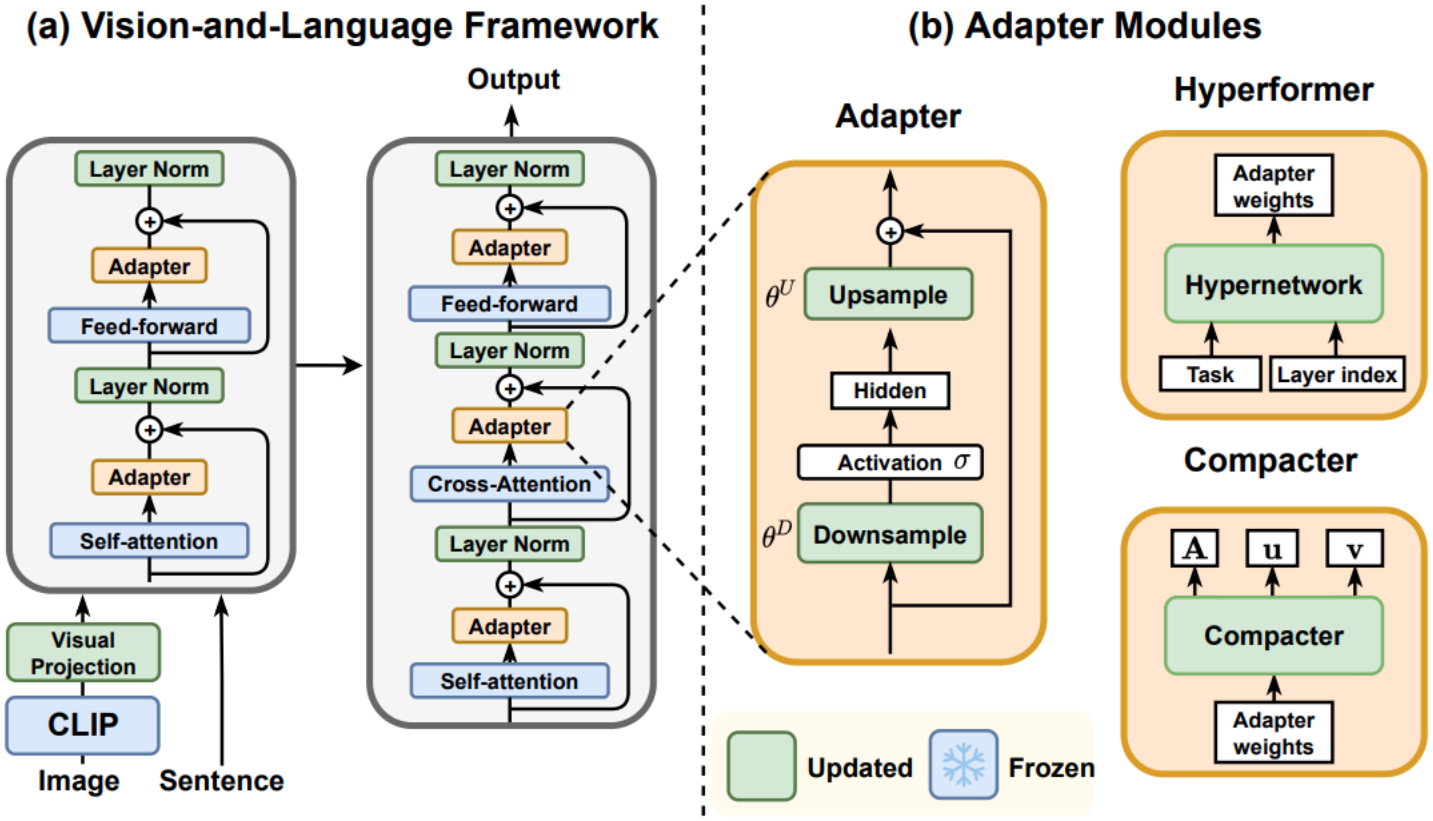


Any concern or limitation?

Train個小抄
帶著考試

PEFT (2/3): VL-ADAPTER

Parameter-Efficient Transfer Learning for Vision-and-Language Tasks



PEFT (2/3): VL-ADAPTER

Parameter-Efficient Transfer Learning for Vision-and-Language Tasks (cont'd)

- Variants for adapter modules
  - Adapter

$$h = f_{\theta^U}(\sigma(f_{\theta^D}(\boldsymbol{x}))) + \boldsymbol{x}$$

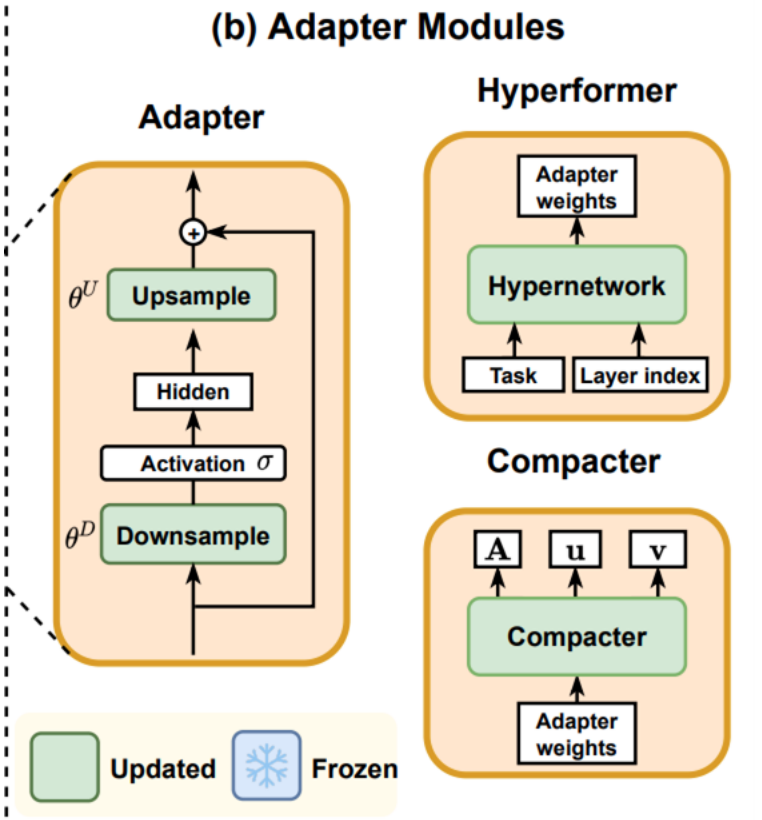
  - Hyperformer

$$[\theta^D, \theta^U] = f_{\theta^H}(f_{\theta^T}([\boldsymbol{t}_j, \boldsymbol{l}_i]))$$

  - Compacter

$$\theta^D = \sum_{i=1}^{k} A_i \otimes B_i = \sum_{i=1}^{k} A_i \otimes (\boldsymbol{u}_i \boldsymbol{v}_i)$$

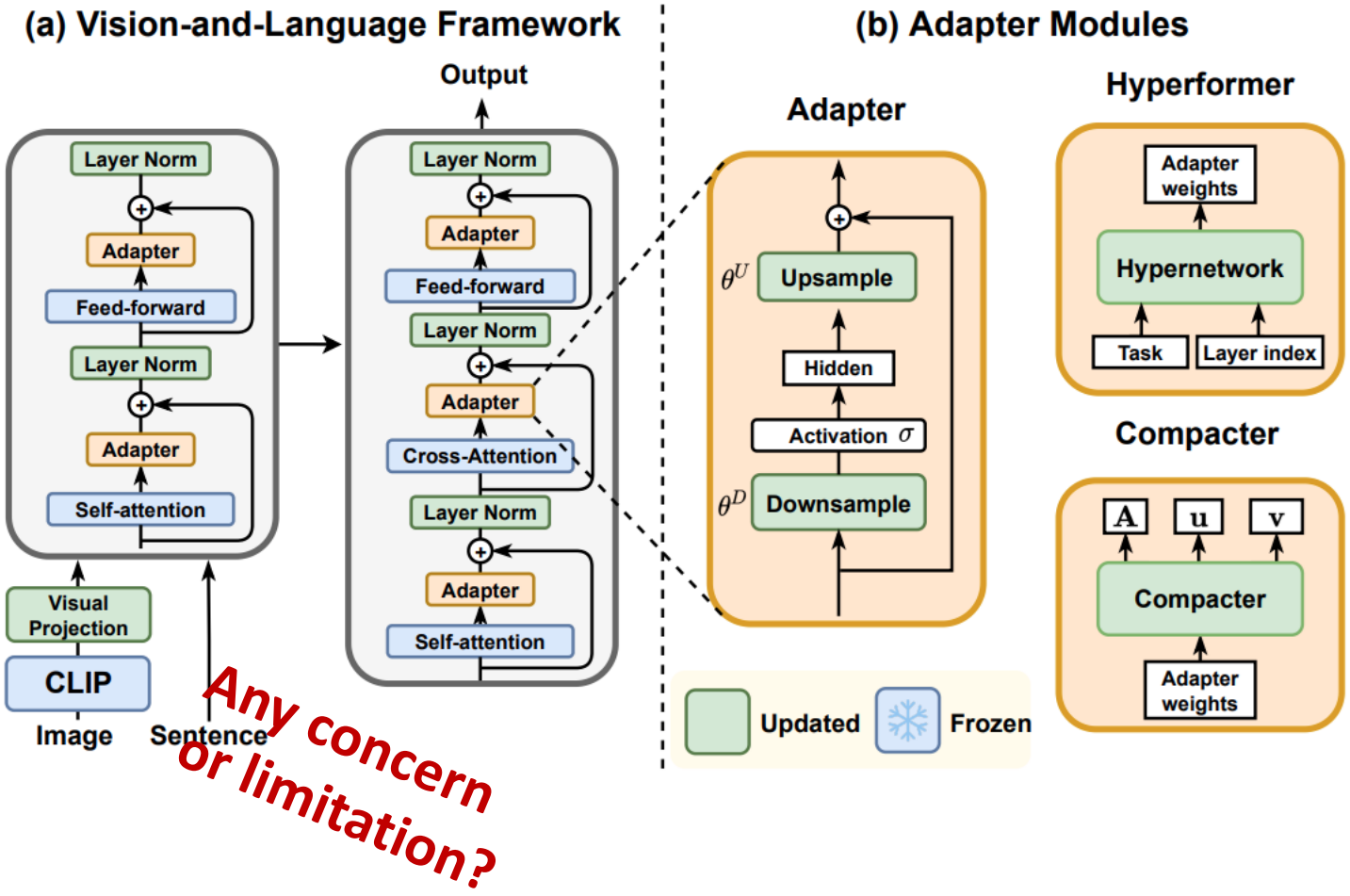
Any concern or limitation?



Train個小抄
帶著考試

PEFT (2/3): VL-ADAPTER

Parameter-Efficient Transfer Learning for Vision-and-Language Tasks



VL-Adapter: Parameter-Efficient Transfer Learning for Vision-and-Language Tasks, UNC, CVPR 2022

PEFT (3/3): LoRA
Low-Rank Adaptation of Large Language Models

- Previous problems
  - Adapter Layers introduce extra inference latency
  - Directly optimizing the prompt may not be easy

- LoRA

$$W_0 \in \mathbb{R}^{d \times k}$$

$$W_0 + \Delta W = W_0 + BA$$

$$B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k}$$

$$\text{rank } r \ll \min(d, k)$$

$$h = W_0 x + \Delta W x = W_0 x + BAx$$

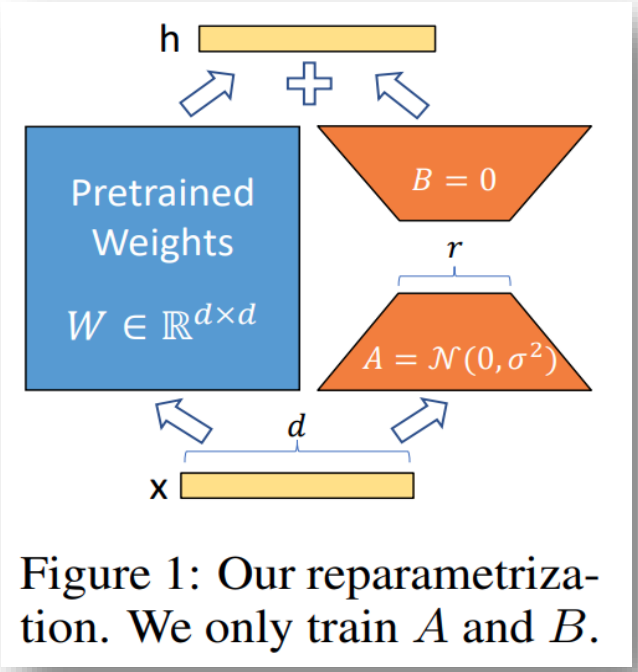


Figure 1: Our reparametrization. We only train A and B.

LoRA: Low-Rank Adaptation of Large Language Models, Microsoft, ICLR 2022

LoRA: Low-Rank Adaptation of LLMs (cont'd)

Model & Method	# Trainable Parameters	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	Avg.
RoB_{base} (FT)*	125.0M	**87.6**	94.8	90.2	**63.6**	92.8	**91.9**	78.7	91.2	86.4
RoB_{base} (BitFit)*	0.1M	84.7	93.7	**92.7**	62.0	91.8	84.0	81.5	90.8	85.2
RoB_{base} (Adpt^{D})*	0.3M	$87.1_{\pm.0}$	$94.2_{\pm.1}$	$88.5_{\pm1.1}$	$60.8_{\pm.4}$	$93.1_{\pm.1}$	$90.2_{\pm.0}$	$71.5_{\pm2.7}$	$89.7_{\pm.3}$	84.4
RoB_{base} (Adpt^{D})*	0.9M	$87.3_{\pm.1}$	$94.7_{\pm.3}$	$88.4_{\pm.1}$	$62.6_{\pm.9}$	$93.0_{\pm.2}$	$90.6_{\pm.0}$	$75.9_{\pm2.2}$	$90.3_{\pm.1}$	85.4
RoB_{base} (LoRA)	0.3M	$87.5_{\pm.3}$	$\mathbf{95.1}_{\pm.2}$	$89.7_{\pm.7}$	$63.4_{\pm1.2}$	$\mathbf{93.3}_{\pm.3}$	$90.8_{\pm.1}$	$\mathbf{86.6}_{\pm.7}$	$\mathbf{91.5}_{\pm.2}$	**87.2**
$\text{RoB}_{\text{large}}$ (FT)*	355.0M	90.2	**96.4**	**90.9**	68.0	94.7	**92.2**	86.6	92.4	88.9
$\text{RoB}_{\text{large}}$ (LoRA)	0.8M	$\mathbf{90.6}_{\pm.2}$	$96.2_{\pm.5}$	$\mathbf{90.9}_{\pm1.2}$	$\mathbf{68.2}_{\pm1.9}$	$\mathbf{94.9}_{\pm.3}$	$91.6_{\pm.1}$	$\mathbf{87.4}_{\pm2.5}$	$\mathbf{92.6}_{\pm.2}$	**89.0**
$\text{RoB}_{\text{large}}$ (Adpt^{P})†	3.0M	$90.2_{\pm.3}$	$96.1_{\pm.3}$	$90.2_{\pm.7}$	$\mathbf{68.3}_{\pm1.0}$	$\mathbf{94.8}_{\pm.2}$	$\mathbf{91.9}_{\pm.1}$	$83.8_{\pm2.9}$	$92.1_{\pm.7}$	88.4
$\text{RoB}_{\text{large}}$ (Adpt^{P})†	0.8M	$\mathbf{90.5}_{\pm.3}$	$\mathbf{96.6}_{\pm.2}$	$89.7_{\pm1.2}$	$67.8_{\pm2.5}$	$\mathbf{94.8}_{\pm.3}$	$91.7_{\pm.2}$	$80.1_{\pm2.9}$	$91.9_{\pm.4}$	87.9
$\text{RoB}_{\text{large}}$ (Adpt^{H})†	6.0M	$89.9_{\pm.5}$	$96.2_{\pm.3}$	$88.7_{\pm2.9}$	$66.5_{\pm4.4}$	$94.7_{\pm.2}$	$92.1_{\pm.1}$	$83.4_{\pm1.1}$	$91.0_{\pm1.7}$	87.8
$\text{RoB}_{\text{large}}$ (Adpt^{H})†	0.8M	$90.3_{\pm.3}$	$96.3_{\pm.5}$	$87.7_{\pm1.7}$	$66.3_{\pm2.0}$	$94.7_{\pm.2}$	$91.5_{\pm.1}$	$72.9_{\pm2.9}$	$91.5_{\pm.5}$	86.4
$\text{RoB}_{\text{large}}$ (LoRA)†	0.8M	$\mathbf{90.6}_{\pm.2}$	$96.2_{\pm.5}$	$\mathbf{90.2}_{\pm1.0}$	$68.2_{\pm1.9}$	$\mathbf{94.8}_{\pm.3}$	$91.6_{\pm.2}$	$\mathbf{85.2}_{\pm1.1}$	$\mathbf{92.3}_{\pm.5}$	**88.6**
DeB_{XXL} (FT)*	1500.0M	91.8	**97.2**	92.0	72.0	**96.0**	92.7	93.9	92.9	91.1
DeB_{XXL} (LoRA)	4.7M	$\mathbf{91.9}_{\pm.2}$	$96.9_{\pm.2}$	$\mathbf{92.6}_{\pm.6}$	$\mathbf{72.4}_{\pm1.1}$	$\mathbf{96.0}_{\pm.1}$	$\mathbf{92.9}_{\pm.1}$	$\mathbf{94.9}_{\pm.4}$	$\mathbf{93.0}_{\pm.2}$	**91.3**

Model & Method	# Trainable Parameters	E2E NLG Challenge				
		BLEU	NIST	MET	ROUGE-L	CIDEr
GPT-2 M (FT)*	354.92M	68.2	8.62	46.2	71.0	2.47
GPT-2 M ($\text{Adapter}^{\text{L}}$)*	0.37M	66.3	8.41	45.0	69.8	2.40
GPT-2 M ($\text{Adapter}^{\text{L}}$)*	11.09M	68.9	8.71	46.1	71.3	2.47
GPT-2 M ($\text{Adapter}^{\text{H}}$)	11.09M	$67.3_{\pm.6}$	$8.50_{\pm.07}$	$46.0_{\pm.2}$	$70.7_{\pm.2}$	$2.44_{\pm.01}$
GPT-2 M (FT^{Top2})*	25.19M	68.1	8.59	46.0	70.8	2.41
GPT-2 M (PreLayer)*	0.35M	69.7	8.81	46.1	71.4	2.49
GPT-2 M (LoRA)	0.35M	$\mathbf{70.4}_{\pm.1}$	$\mathbf{8.85}_{\pm.02}$	$\mathbf{46.8}_{\pm.2}$	$\mathbf{71.8}_{\pm.1}$	$\mathbf{2.53}_{\pm.02}$
GPT-2 L (FT)*	774.03M	68.5	8.78	46.0	69.9	2.45
GPT-2 L ($\text{Adapter}^{\text{L}}$)	0.88M	$69.1_{\pm.1}$	$8.68_{\pm.03}$	$46.3_{\pm.0}$	$71.4_{\pm.2}$	$\mathbf{2.49}_{\pm.0}$
GPT-2 L ($\text{Adapter}^{\text{L}}$)	23.00M	$68.9_{\pm.3}$	$8.70_{\pm.04}$	$46.1_{\pm.1}$	$71.3_{\pm.2}$	$2.45_{\pm.02}$
GPT-2 L (PreLayer)*	0.77M	70.3	8.85	46.2	71.7	2.47
GPT-2 L (LoRA)	0.77M	$\mathbf{70.4}_{\pm.1}$	$\mathbf{8.89}_{\pm.02}$	$\mathbf{46.8}_{\pm.2}$	$\mathbf{72.0}_{\pm.2}$	$2.47_{\pm.02}$

DoRA: Weight-Decomposed Low-Rank Adaptation

- Previous problems
  - Adapter layers introduce extra inference latency
  - Directly optimizing the prompt may not be sufficient
  - LoRA still exhibits performs gap (vs. FT)

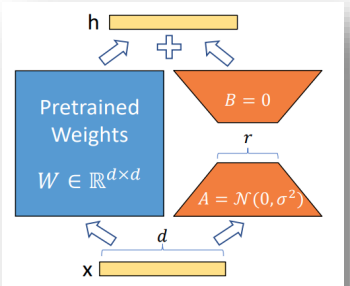
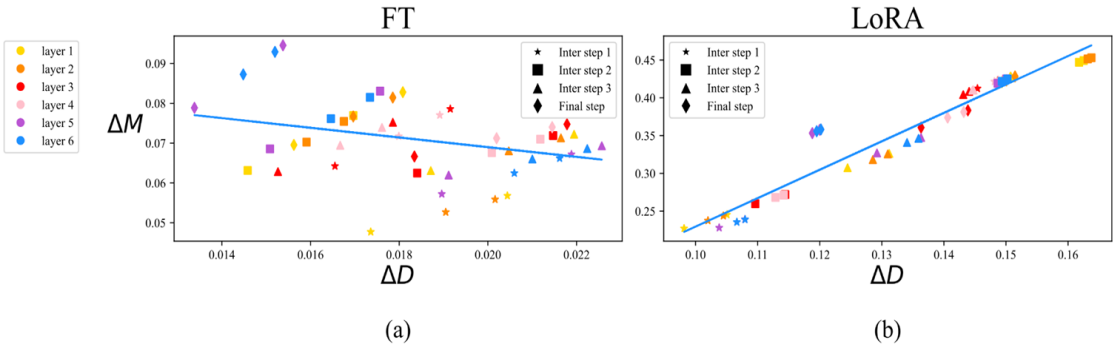


Figure 1: Our reparametrization. We only train A and B.

- Observation…



DoRA: Weight-Decomposed Low-Rank Adaptation
NVIDIA (Taiwan), ICML 2024 (Google Scholar citation #: 700)

What We've Covered?

- Transformer
- Transformer for Visual Analysis
  - Vision Transformer (ViT)
  - SSL & Beyond
- Vision-Language Model
  - Image2Text
  - Text2Image
- Pretraining vs. Finetuning
  - Parameter-Efficient Finetuning (PEFT)
- No class next Wed (10/22) due to ICCV 2025